



ROBOTICS

Application manual

OmniCore App SDK 1.6.0

This page is intentionally left blank

Application manual
OmniCore App SDK 1.6.0
Document revision: A, en

The information in this manual is subject to change without notice and should not be construed as a commitment by ABB. ABB assumes no responsibility for any errors that may appear in this manual.

Except as may be expressly stated anywhere in this manual, nothing herein shall be construed as any kind of guarantee or warranty by ABB for losses, damages to persons or property, fitness for a specific purpose or the like.

In no event shall ABB be liable for incidental or consequential damages arising from use of this manual and products described herein.

This manual and parts thereof must not be reproduced or copied without ABB's written permission.

Additional copies of this manual may be obtained from ABB.

The original language for this publication is English. Any other languages that are supplied have been translated from English.

© Copyright 2020-2026 ABB. All rights reserved.

ABB AB, Robotics

Robotics and Discrete Automation

Hydrovägen 10

SE-721 68 Västerås

Sweden

Table of contents

1	About This Manual	7
2	Introduction	8
3	Setting Up	10
4	Robot Web Services Client Library	13
4.1	Return Values	13
4.2	App Lifetime Callbacks	15
4.3	RWS.Controller Namespace.....	16
4.3.1	Monitor Object.....	24
4.4	RWS.Rapid Namespace.....	26
4.4.1	Monitor Object.....	35
4.4.2	Task Object.....	39
4.4.3	ServiceRoutine Object	44
4.4.4	Module Object.....	45
4.4.5	Data Object	46
4.5	RWS.IO Namespace.....	55
4.5.1	Network Object.....	58
4.5.2	Device Object.....	59
4.5.3	Signal Object.....	61
4.6	RWS.CFG Namespace.....	65
4.6.1	Domain Object.....	69
4.6.2	Type Object.....	72
4.6.3	Instance Object	75
4.7	RWS.FileSystem Namespace	76
4.7.1	Directory Object.....	78
4.7.2	File Object	81
4.8	RWS.Elog Namespace	84
4.8.1	Event Object.....	86
4.8.2	Domain Object.....	88
4.9	RWS.UAS Namespace	91
4.10	RWS.ControlStation Namespace	92
4.10.1	Monitor Object.....	97
4.11	RWS.Mastership Namespace.....	98
4.11.1	Monitor Object.....	100

1 About This Manual

4.12 RWS.MotionMastership Namespace	101
4.12.1 Monitor Object	103
5 Components Library	104
5.1 Introduction	104
5.2 Common Resources	105
5.3 Standard Colors	106
5.4 Button	108
5.5 Checkbox	110
5.6 Dropdown	112
5.7 Input	116
5.8 Radio Button	119
5.9 Switch Button	121
5.10 Toggle	123
5.11 Slider	127
5.12 Context Menu	131
5.13 Digital Indicator	136
5.14 Analog Level Meter	138
5.15 Line Chart	140
5.16 Pie Chart	148
5.17 Menu	155
5.18 Fold-in	162
5.19 Popup Dialog	164
5.20 File System Dialog	169
5.21 Tab Container	172
5.22 Hamburger Menu	177
5.23 On-screen Keyboard	181
5.24 Console Log Overlay	184
6 AppMaker	187

1 About This Manual

About this manual

This manual describes how to build and deploy custom OmniCore FlexPendant user interfaces. Custom user interfaces are developed using common web technology, i.e. HTML, CSS and JavaScript.

This manual also describes how to connect your custom user interface to the OmniCore controller using Robot Web Services (RWS) through a JavaScript RWS client library supplied by ABB.

There is also a components library for creating “widgets” (e.g., buttons, drop-downs, switches, ...) for a look-and-feel that is similar in appearance to what is used in the native FlexPendant shell. This library is also documented in this manual.

Who should read this manual?

This manual is intended for:

- System integrators
- Function package developers
- Advanced end-customers

Prerequisites

The reader should have a basic knowledge of:

- General web development (HTML, CSS, JavaScript)
- The OmniCore controller

References

References	Document ID
-	-

2 Introduction

General information

It is common for various robot cell function packages and solutions to add their own user interfaces to the robot controller's teach pendant. ABB's previous-generation IRC5 controller had a C#/.NET SDK for building such user interfaces. The OmniCore FlexPendant takes a step forward and introduces the possibility to create user interfaces in the form of web applications, using common web technologies (HTML, CSS, JavaScript).

In practice, this means that each custom user interface on OmniCore FlexPendant is a web page/application running in an embedded browser within the FlexPendant shell.

Since the embedded browser is a general web browser, there are few limitations on third party frameworks or libraries used for developing the application. Since most FlexPendant applications are relatively small and simple, some application developers prefer to use plain HTML/CSS/JavaScript – which has grown quite powerful over the years. Others may opt to use one or more of the many tools available from the web development community.

It is beyond the scope of this document to describe general web development. It is assumed that the reader has knowledge in these areas. If not, it is recommended to start by going through one of the many introductions available from various sources on the Internet.

RobotWare support

Support for web-based user interfaces and OmniCore App SDK was added in RobotWare 7.1 and the corresponding FlexPendant software.

Browser engine

The embedded browser is using the legacy pre-Chromium Microsoft Edge engine, version 18.19044.

The engine may be replaced in future versions of the FlexPendant software.

Robot Web Services Client Library

The application will most likely need to communicate with the controller. This is done by using the *Robot Web Services* REST API, commonly referred to as *RWS*, hosted on the controller.

While it is fully possible for the application to use the built-in capabilities of JavaScript to communicate directly with RWS, a client library is also available for more user-friendly access.

The Robot Web Services Client Library is distributed as JavaScript files that should be deployed together with the application itself. More on this in section 4.

Components Library

The Components Library is a collection of re-usable interface components, such as buttons, input fields, menus, etc.

The Components Library is distributed as JavaScript/CSS files that should be deployed together with the application itself. More on this in section 5.

Regarding AppMaker

Previous versions of OmniCore App SDK included *AppMaker*, a visual tool for quickly designing and generating code for extensible OmniCore App SDK-based apps.

As of release 1.5, AppMaker has been discontinued.

If this causes any inconvenience, please reach out to ABB support to discuss your options for moving forward.

3 Setting Up

Setting up

This section describes the minimal requirements for an app running on the OmniCore FlexPendant, and some other general information such as accessing the app from an external browser.

Application file structure

Each application needs to meet a few requirements in order to be properly installed. The source files for a very simple application named `MyApp` could look like this:

```
$HOME
└─ WebApps
    └─ MyApp
        ├── appinfo.xml
        ├── myapp.html
        └─ icon.png
```

Each application can be deployed in a separate subdirectory under `$HOME/WebApps/`. Hence, if another application is added, the file structure would be

```
$HOME
└─ WebApps
    └─ MyApp
        ├── appinfo.xml
        ├── myapp.html
        └─ icon.png
    └─ MyOtherApp
        ├── appinfo.xml
        ├── myotherapp.html
        └─ icon.png
```

The FlexPendant will automatically discover and add launchers for available applications, providing they are located in a correct directory path on the controller. See more below on how to package the application within a RobotWare Addin.

In the examples above, each application consists of three files.

The `myapp.html` file is the HTML page implementing the application. An application usually contains many more files than this file, it could be other HTML pages, CSS files, JavaScript files, or images.

If the RWS Library or the Components Library is used, they are placed here as well. More information about these files can be found in the relevant sections in this document.

Each application should also contain an icon image file that will be used for its launcher in the FlexPendant Applications menu. This image will be rendered at 100x100 pixels resolution and will automatically be scaled.

The `appinfo.xml` file is the application manifest file. For the simple application above, it looks like this:

```
<?xml version="1.0" encoding="UTF-8"?>

<WebApp>
    <name>MyWebApp</name>
    <icon>myicon.png</icon>
    <path>myappl.html</path>
</WebApp>
```

The `name` element is used to give the application a name, this name is used in the FlexPendant Application menu, and as an identifier for navigation purposes, e.g. when using the `UIDisplay RAPID` routine. Note that there is no requirement for this name to be equal to the app's directory name.

The `icon` element is used to refer to the image file that should be used for the launcher.

The `path` element is a reference to where the start page of the application is located.

Both the icon and start page paths are relative to the directory where the `appinfo.xml` file is located.

Distributing the app within a RobotWare Add-in

For advanced users, it is also possible to place a `WebApps` directory in an option directory of a RobotWare Add-In. This directory can host applications in the same way as when located under `$HOME`. I.e. `<option>/WebApps/<app>`.

Information about the general structure of RobotWare Addins is beyond the scope of this manual. However, to register an option directory, use the following line in the add-in's `install.cmd` file:

```
register -type option -description my_app -path $BOOTPATH
```

In this example, the FlexPendant will search in `$BOOTPATH/WebApps/` for app directories, provided the `WebApps` directory exists.

The web-root

The *web-root* is the base directory for the application when loading additional resources from the application. For each application, the web-root is the directory on the controller that contains the `appinfo.xml` file.

The HTML image tag

```
</img>
```

will load the SomePicture.png image from the root directory on the controller.

Using an external browser

It is possible to use an external browser (such as Chrome, Edge or Firefox) for connecting to a controller and running an application. This is especially useful during development, since most modern browsers have a plethora of powerful development tools.

To connect to a virtual controller on the same machine, assuming there is a web application at `$HOME/WebApps/MyApp/myapp.html`, use the following address:

```
https://127.0.0.1:80/fileservice/$home/WebApps/MyApp/myapp.html
```

The network port (80) is not guaranteed on a virtual controller, since the port may be in use by another virtual controller or other process on the same machine, or otherwise blocked by a firewall or operating system policy.

From RobotWare 7.3 and forward, if port 80 cannot be used for any reason, the virtual controller will instead try to find an available port from the following list:

- 5466
- 9403
- 9805
- 11622
- 19985
- 31015
- 34250
- 40129
- 45003

Since the virtual controller is using a self-signed certificate, the browser may issue a warning, and a permission to proceed anyway must be given.

The browser will ask for user credentials, which are the same as when logging into the system normally. Unless UAS (*User Authorization System*) has been configured, the default user is "**Default User**" and the corresponding password is "**robotics**".

If using a real controller, replace `127.0.0.1` with the IP address of the controller. The port is **443**, which can be omitted since it is the default port for HTTPS communication.

Note that due to limitations in Robot Web Services and RobotWare, it is not supported to use several tabs in the same browser for connecting to the same controller.

Example WebApps

The OmniCore App SDK distribution contains a set of simple application examples.

One such example is the `components-example` application, which showcases how to use some components from the Components Library. Another example, `rws-example`, shows how to use the RWS Library to set up simple read and write communication with RAPID data and I/O signals, as well as use these values to update the user interface of an application.

4 Robot Web Services Client Library

General Information

The Robot Web Services Library provides a JavaScript API to access the controller through use of Robot Web Services (RWS.) It uses pure JavaScript and does not require any knowledge of other frameworks. It does not preclude the use of other frameworks or techniques, that is left entirely up to the app developer.

The Robot Web Services Library uses only JavaScript functionality that is compatible with Microsoft's Edge browser as that is used by the WebView component on the FlexPendant. That is a limitation that also should be adhered to by users of the library.

The API uses JavaScript promises for handling asynchronous calls. It is strongly recommended that the user of the API knows how to best use the advantages of promises, however, most examples in this chapter use the `await` instruction to make it easier to understand the API.

Terminology

Throughout this chapter we will use the term namespace. In the Robot Web Services library, we build hierarchies by nesting objects. To visualize such hierarchies, we refer to them in the format; `RWS.Rapid`, this syntax we call namespaces. As a namespace refers to an object it may have methods as any other object in JavaScript.

RWS supports subscriptions to events that occur on the controller. Most subscriptions are related to a specific object, e.g. a IO signal or RAPID variable, but some are general and have no objects created. To facilitate subscriptions to such events there are special objects implemented in the namespaces. These objects are called Monitors and they are implemented specifically for the namespaces and thus have different behavior depending of the functionality of the subscriptions they support.

Installation

The Robot Web Services Library files should be placed in a subdirectory under the application web root (where the `appinfo.xml` file is located) named `rws-api`.

omnicore-rws.js

The `omnicore-rws` script is required and contains the RWS namespace. This is where all essential functionality regarding controller interaction via Robot Web Services resides.

To use the script, it must be loaded into the application. The most straight-forward method is to add it to the HTML file's head section.

```
<script src="rws-api/omnicore-rws.js"></script>
```

omnicore-app.js

This script contains the App namespace and is imported in the same way as the `omnicore-rws.js` file in the previous section.

```
<script src="rws-api/omnicore-app.js"></script>
```

rapiddata-rws.js

This script existed in earlier releases and has been merged into `omnicore-rws.js`.

4.1 Return Values

Asynchronous operations

Typically functions that will take time to execute, e.g. functions that communicate with the controller through RWS, will return a Promise. This allows the API user to execute them asynchronously, or synchronously by using `await`. It is strongly recommended that the API user knows how Promises in JavaScript work and can select the appropriate way to handle them. Running asynchronously will result in significantly improved UI performance.

When a Promise is resolved the returned data will be as defined for the specific method in this manual, however when a Promise is rejected a status object will be returned. This object will simply be referred to as Status in this document. The object will at least contain a message string but may also contain additional information (note that the user is responsible to check if the additional information exists before use.)

Structure of the status object:

```
status = {
  (string) message,      descriptive text
  ({} ) httpStatus,      HTTP status object, see description below
  ({} ) controllerStatus controller status object, see description below
}
```

```
httpStatus = {
  (number) code,          HTTP status, e.g. 400
  (string) text           HTTP text, e.g. 'BadRequest'
}
```

```
controllerStatus = {
  (string) name,          status name, e.g.
                          'SYS_CTRL_E_RAPID_SYNTAX_ERROR'
  (string) code,          status code, e.g. '-1073442803'
  (string) severity,      'Error' or 'Success'
  (string) description    status description, e.g.
                          'Operation not allowed due to syntax error(s).'
```

Example

```
var x = 42;
RWS.Rapid.setDataValue('T_ROB1', 'TestModule', 'TestData', x.toString())
  .catch(status => {
    let m = status.message;
    let h = '';
    if (status.hasOwnProperty('httpStatus') === true){
```

```
        let code = status.httpStatus.code;
        let text = status.httpStatus.text;
        h = `${code} ${text}`;
    }
    let c = '';
    if (status.hasOwnProperty('controllerStatus') === true) {
        c = status.controllerStatus.description;
    }
    console.log(`${m}\n${h}\n${c}`);
});
```

4.2 App Lifetime Callbacks

OmniCore App SDK callbacks

OmniCore App SDK support several lifetime callbacks – i.e., custom functions that will be called in some situations when running on the FlexPendant.

To enable a callback, simply declare a function in global scope (the `window` object) with a matching name.

User Implemented Functions

`appActivate`

Signature: `appActivate()`

Requires the `omnicore-app.js` script file.

This function will be called by the App SDK when the app is brought to the foreground after being in the background, i.e. being "activated".

This function is not called by the SDK when the app is started.

This function is expected to return a Promise that resolves if the execution of the function was successful and rejects otherwise. The contents of the promise are not relevant as the return value that is sent back to the FlexPendant will always be a boolean, true or false, depending on if the promise resolves or rejects.

`appDeactivate`

Signature: `appDeactivate()`

Requires the `omnicore-app.js` script file.

This function will be called by the App SDK when the app is brought to the background, i.e. being "deactivated".

This function is expected to return a Promise that resolves if the execution of the function was successful and rejects otherwise. The contents of the promise are not relevant as the return value sent back to the FlexPendant will always be a boolean, true or false, depending on if the promise resolves or rejects.

This function can be used to disable user-controlled subscriptions and persist any volatile state that otherwise can be lost if the FlexPendant shuts down.

`appCleanUp`

Signature: `appCleanUp()`

Requires the `omnicore-rws.js` script file.

This function will be called by the App SDK when the app is closed on the FlexPendant.

This callback will be aborted by the FlexPendant after a few seconds if not yet finished. The exact time available can differ since the time slot is shared by some cleanup from the App SDK itself.

The rule of thumb is to keep the callback as short as possible.

This function is expected to return a Promise that resolves if the execution of the function was successful and rejects otherwise. The contents of the promise are not relevant as the return value sent back to the FlexPendant will always be a boolean, true or false, depending on if the promise resolves or rejects.

This function can be used to disable user-controlled subscriptions and persist any volatile state that otherwise can be lost if the FlexPendant shuts down.

JavaScript "load" Event

Code that should be executed when starting the app should be placed in a standard JavaScript load event, to ensure everything is loaded before starting using the APIs.

Example:

```
<html>
  <head>
    <script src="rws-api/omnicore-rws.js"></script>
    <script src="rws-api/omnicore-app.js"></script>
    <script>
      window.addEventListener("load", ()=>{
        // Do initialization here
      });
    </script>
  </head>
</html>
```

4.3 RWS.Controller Namespace

Description

The controller namespace, `RWS.Controller`, contains basic functionality for manipulating the controller, e.g. get system information, create/restore backup, set motors on/off, restart controller, etc. To facilitate easy subscription to controller states there is a Monitor object.

Methods

getMonitor

Signature: `getMonitor(string resource)`

`resource` the resource to subscribe

Creates a new Monitor (see section 4.3.1) for subscribing to the provided resource. Valid resources are:

<code>'controller-state'</code>	changes to controller's state
<code>'operation-mode'</code>	changes to the controller's operation mode

The enum `RWS.Controller.MonitorResources` provides an easy way to safely use supported resource strings.

The returned Monitor has no callbacks set nor subscription enabled.

Return value

A Monitor object.

isVirtualController

Signature: `isVirtualController()`

Gets the controller's type. I.e., whether it is a virtual or physical controller.

Return value

A promise with a boolean, *true* for virtual controller, *false* for physical controller.

getControllerState

Signature: `getControllerState()`

Gets the controller's system state.

Return value

A promise with a string containing the controller state. Possible states are:

<code>'initializing'</code>	the controller is starting up
<code>'motors_on'</code>	motors on state
<code>'motors_off'</code>	motors off state
<code>'guard_stop'</code>	stopped due to safety
<code>'emergency_stop'</code>	emergency stop state
<code>'emergency_stop_resetting'</code>	emergency stop state resetting
<code>'system_failure'</code>	system failure state

Example

```
var state = await RWS.Controller.getControllerState();
if (state === 'emergency_stop' || state === 'system_failure'){
    mySetErrorState();
}
```

```
}
```

setMotorsState

Signature: `setMotorsState(string state)`

state the new state of the motors

Sets the controller's motor state. Possible states are:

'motors_on'	motors on state
'motors_off'	motors off state

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

getOperationMode

Signature: `getOperationMode()`

Gets the controller's operation mode.

Return value

A promise with a string containing the operation mode. Possible modes are:

'initializing'	controller is starting up, but not yet ready
'automatic_changing'	automatic mode requested
'manual_full_changing'	manual full speed mode requested
'manual_reduced'	manual reduced speed mode
'manual_full'	manual full speed mode
'automatic'	automatic mode
'undefined'	undefined

setOperationMode

Signature: `setOperationMode(string mode)`

mode the new operation mode

Sets the controller's operation mode. Possible modes are:

'automatic'	automatic mode
'manual'	manual mode
'manual_full'	manual full speed mode

NOTE! Automatic and manual full speed mode must be acknowledged by the user. This is done in a message box that is presented on the FlexPendant.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

restartController

Signature: `restartController(string mode)`

`mode` the restart mode, default is 'restart'

Restarts the controller. The parameter mode indicates what kind of restart is requested. Possible modes are:

'restart'	normal warm start
'shutdown'	shut down the controller
'boot_application'	start boot application
'reset_system'	reset system
'reset_rapid'	reset Rapid
'revert_to_auto'	revert to last auto-save

NOTE! An enum `'RWS.Controller.RestartModes'` with the valid values is provided for ease of use.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

getEnvironmentVariable

Signature: `getEnvironmentVariable(string variable)`

`variable` the environment variable to get

Gets any of the predefined system environment variables, e.g. "\$HOME", "\$TEMP", etc.

Return value

A promise with a string containing the contents of an environment variable.

getTime

Signature: `getTime()`

Gets the system time. The time is returned as a string formatted as 'year-month-day T hour:minute:second', e.g. '2020-06-11 T 15:04:33'.

Return value

A promise with a string containing the system time.

Example

```
var timeString = await RWS.Controller.getTime();
try {
```

```
// Convert controller time string to JavaScript Date
var temp = timeString.replace(/[T]/g, '-');
temp = temp.replace(/[ ]/g, '');
temp = temp.replace(/[:]/g, '-');
var s = temp.split('-');
var t = new Date(s[0], s[1] - 1, s[2], s[3], s[4], s[5]);
var h = t.getHours();
if (h >= 6 && h < 9){
    console.log('Good morning!');
}
} catch(error) {
    console.error(`not a valid date: ${error}`);
}
```

getTimezone

Signature: `getTimezone()`

Gets the system time zone.

NOTE! Not supported on virtual controller.

Return value

A promise with a string containing the system time zone.

getIdentity

Signature: `getIdentity()`

Gets the controller's identity (name.)

Return value

A promise with a string containing the controller's identity.

getNetworkSettings

Signature: `getNetworkSettings()`

Gets the controller's network settings.

NOTE! Not supported on virtual controller.

Return value

A promise with a list of objects containing the network settings for each adapter, defined as:

<code>(string) id</code>	adapter identification string
<code>(string) logicalName</code>	logical name of adapter
<code>(string) network</code>	network interface name
<code>(string) address</code>	IP address

(string) mask	network interface mask
(string) primaryDNS	primary DNS IP address
(string) secondaryDNS	secondary DNS IP address
(string) DHCP	use DHCP, valid values: 'enabled' or 'disabled'
(string) gateway	gateway IP address

getNetworkConnections

Signature: `getNetworkConnections()`

Gets the controller's network connections. This collection of information will provide information on which interfaces are currently available.

NOTE! Not supported on virtual controller.

Return value

A promise with a list of objects containing the network connections:

(string) id	adapter identification string
(string) MACAddress	adapter MAC address
(boolean) connected	adapter media status
(boolean) enabled	adapter execution status
(string) speed	adapter network speed, e.g. '100 Mbps'

verifyOption

Signature: `verifyOption(string option)`

option the name of the option

Verifies if an option is available on the system.

NOTE! The option string is case sensitive.

Return value

A promise with a boolean.

createBackup

Signature: `createBackup(string path, number timeout)`

path the path where the backup will be created, including the name of the backup

timeout the maximum time (in seconds) to wait for completion, default is 60 s

Creates a backup of the system. The Promise returned by the function will settle when the backup is either completed or fails. As the process of creating a backup can take a long time, and the time is heavily dependent of the system, there is a possibility that the wait for completion is terminated before completion. By adjusting the timeout value; it is possible to ensure that the process gets a reasonable time to finish.

NOTE! Even if the wait for completion is aborted, the controller can still continue the process and the backup may or may eventually be completed.

Return value

A promise with the status code from the backup process or error status text.

verifyBackup

Signature: `verifyBackup(string path, {})`

`path` the path to the backup

`{}` options for the verify process, containing:

<code>(string) ignoreMismatches</code>	ignore mismatches between system and backup, valid values: 'all', 'system-id', 'template-id' or 'none'
<code>(boolean) includeControllerSettings</code>	include controller settings, true or false
<code>(boolean) includeSafetySettings</code>	include safety settings, true or false
<code>(string) include</code>	included components, 'cfg', 'modules' or 'all'

Verifies a backup. The return value is a Promise containing a list of errors. If the list is empty the verification did not find any faults and backup will be usable for the given properties.

There are two enum objects that can be used to set values to the `ignoreMismatches` and `include` parameters; `BackupIgnoreMismatches` and `BackupInclude`. These enums can be used to avoid spelling errors and other use of illegal values.

Return value

A promise with a list of objects containing errors. An error object contains:

<code>(string) status</code>	the status of the verification, possible values: 'ok' 'system_id_mismatch' 'template_id_mismatch' 'file_or_directory_missing' 'cfg_file_corrupt'
<code>(string) path</code>	optionally a path to the file or folder with issues

Example

```
await RWS.Controller.createBackup('$BACKUP/myBackup', 120);
var statuses = await RWS.Controller.verifyBackup('$BACKUP/myBackup');
if (statuses.length > 0){
    for (let i = 0; i < statuses.length; i++){
```

```

        if (statuses[i].path !== ''){
            var s = statuses[i].status;
            var p = statuses[i].path;
            console.log(`${s} in '${p}'`);
        } else {
            console.log(statuses[i].status);
        }
    }
} else {
    console.log('No problems.');
```

restoreBackup

Signature: `restoreBackup(string path, {})`

`path` the path to the backup

`{}` options for the restore process, containing:

<code>(string) ignoreMatches</code>	ignore mismatches between system and backup, valid values: 'all', 'system-id', 'template-id' or 'none'
<code>(boolean) deleteDir</code>	indicates if backup directory should be deleted afterwards, true or false
<code>(boolean) includeSafetySettings</code>	include safety settings, true or false
<code>(string) include</code>	included components, 'cfg', 'modules' or 'all'

Restores a backup with the given settings.

There are two enum objects that can be used to set values to the `ignoreMatches` and `include` parameters; `BackupIgnoreMatches` and `BackupInclude`. These enums can be used to avoid spelling errors and other use of illegal values.

NOTE! This operation will restart the controller.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

saveDiagnostics

Signature: `saveDiagnostics(string path, number timeout)`

`path` the path where the diagnostics will be placed, including file name

`timeout` the maximum time (in seconds) to wait for completion, default is 60 s

Creates a system diagnostics log. The Promise returned by the function will settle when the process is either completed or fails. As the process of saving a diagnostics log can take a long time, and the time is heavily dependent of the system, there is a possibility

that the wait for completion is terminated before completion. By adjusting the timeout value; it is possible to ensure that the process gets a reasonable time to finish.

NOTE! Even if the wait for completion is aborted, the controller can still continue the process and the diagnostics may or may not be completed.

NOTE! Not supported on virtual controller.

Return value

A promise with the status code from the process or an error status text.

4.3.1 Monitor Object

Description

The Monitor object is used to subscribe to resources that have no explicit object that can be used for subscription. The Monitor supports subscriptions to the controller's state and operation mode.

To create a Monitor object use the factory method `RWS.Controller.getMonitor(resource)`.

Each instance of the Monitor object can subscribe to one resource.

Resources

controller-state

Changes to the state of the controller, e.g. guard stop, motors on, etc. The event data is a string and can be one of the following values:

'initializing'	the robot is starting up
'motors_off'	standby, no power to motors
'motors_on'	ready to move
'guard_stop'	stopped due to safety run chain is open
'emergency_stop'	stopped due to emergency stop active
'emergency_stop_resetting'	emergency stop deactivated but not yet confirmed
'system_failure'	system failure state

operation-mode

Changes to the operation mode of the controller, e.g. auto, man, man full speed, etc. The event data is a string and can be one of the following values:

'initializing'	the robot is starting up
'automatic_changing'	automatic mode requested
'manual_full_changing'	manual full speed mode requested
'manual_reduced'	manual reduced speed mode
'manual_full'	manual full speed mode
'automatic'	automatic mode

'undefined'

undefined

Methods

getTitle

Signature: `getTitle()`

Gets the title (part of the URL) of the subscribed resource.

Return value

A string containing the title.

getResourceString

Signature: `getResourceString()`

Gets a URL that is unique for the resource and used for subscription.

Return value

A string containing a unique resource URL (as used by RWS.)

addCallbackOnChanged

Signature: `addCallbackOnChanged(function callback)`

`callback` a callback function to run when event occurs

Adds a callback function which will be called when the subscribed resource's state changes.

The callback routine can have one in-parameter, which is the new value of the subscribed resource. Contents of the value depends on the resource subscribed to.

Return value

No return value.

Example

```
var controllerMonitor = RWS.Controller.getMonitor('controller-state');
controllerMonitor.addCallbackOnChanged((eventData) => {
    console.log(`Controller state has changed to '${eventData}'.`);
});
```

onchanged

This is a function for internal use only. Do not call from external scripts.

subscribe

Signature: `subscribe(boolean raiseInitial = false)`

`raiseInitial` flag indicating whether an initial event is raised when subscription is registered

Starts the subscription to the resource defined when the Monitor object was created.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

Example

```
var controllerMonitor = RWS.Controller.getMonitor('operation-mode');
try {
    controllerMonitor.addCallbackOnChanged((eventData) => {
        console.log(`Operation mode changed to '${eventData}'.`);
    });
    await controllerMonitor.subscribe();
} catch(error) {
    console.error(`Subscription to 'operation-mode' failed. >>> ${error}`);
}
```

unsubscribe

Signature: `unsubscribe()`

Stops the subscription to the resource defined when the Monitor object was created.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

4.4 RWS.Rapid Namespace

Description

The Rapid namespace, `RWS.Rapid`, contains methods to manipulate Rapid data and execution of programs and service routines.

The namespace contains the following objects; Monitor, Task, Module and Data. These are described closer later in this section.

Methods

getMonitor

Signature: `getMonitor(string resource, string taskName)`

`resource` the resource to subscribe

`taskName` the name of the task

Creates a new Monitor (see section 4.4.1) for subscription on the provided resource. Valid resources are:

'execution'	changes to the Rapid program execution state
'program-pointer'	changes to the program pointer
'motion-pointer'	changes to the motion pointer
'uiinstr'	events from a Ui instruction such as TPWrite

NOTE! The parameter `taskName` is not valid for the 'execution' resource as it only applies for all tasks simultaneously.

NOTE! The returned Monitor has no callbacks set nor is the subscription enabled.

Return value

A Monitor object.

getTasks

Signature: `getTasks()`

Gets the available Rapid tasks on the controller.

Return value

A promise with a list of Task (see section 4.4.2) objects.

getTask

Signature: `getTask(string taskName)`

`taskName` the name of the task

Gets the Task object for Rapid task named `taskName`.

Return value

A promise with a Task object.

getProgramInfo

Signature: `getProgramInfo(string taskName)`

`taskName` the name of the task

Gets the program information for a task. The function returns an object with the information.

Return value

A promise with an object containing:

<code>(string) name</code>	program name
<code>(string) entrypoint</code>	name of function that is the program entry point

getModuleNames

Signature: `getModuleNames(string taskName)`

`taskName` the name of the task

Gets module names for a task. This will return both program modules and system modules.

Return value

A Promise with an object containing:

<code>([string]) programModules</code>	list of the names of all program modules
<code>([string]) systemModules</code>	list of the names of all system modules

Example

```
var modules = await RWS.Rapid.getModuleNames('T_ROB1');
if (modules.hasOwnProperty('programModules') === true){
    for (let i = 0; i < modules['programModules'].length; i++){
        console.log(modules['programModules'][i]);
    }
}
if (modules.hasOwnProperty('systemModules') === true){
    for (let i = 0; i < modules['systemModules'].length; i++){
        console.log(modules['systemModules'][i]);
    }
}
```

getModule

Signature: `getModule(string taskName, string moduleName)`

`taskName` the name of the task

`moduleName` the name of the module

Gets a module. This will retrieve the properties for the module from the controller and initialize the object.

Return value

A Promise with an initialized Module (see section 4.4.4) object.

getData

Signature: `getData(string taskName, string moduleName, string symbolName)`

`taskName` the name of the task

`moduleName` the name of the module

`symbolName` the name of the symbol

Gets a Data object for symbol. This will retrieve the properties for the symbol from the controller and initialize the object.

Return value

A Promise with an initialized Data (see section 4.4.5) object.

setDataValue

Signature: setDataValue(string taskName, string moduleName, string symbolName, string value, ...number indexes)

taskName	the name of the task
moduleName	the name of the module
symbolName	the name of the symbol
value	the new value
indexes	an optional list of numbers that specifies an element in arrays and records

Sets the value of a symbol. This requires the value string to be on the RWS format, and as that can be a bit complicated it is recommended that the `setValue` function on an instance of `RWS.Rapid.Data` is used. However, as this call is slightly faster in some cases, it can be used when timing is essential.

By providing indexes, in the in-parameter `indexes`, a specific element in an array (multidimensional) or record can be set independent of the other elements in the structure. The indexes are 1-based as in Rapid.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

Example

```
var x = 42;
await RWS.Rapid.setDataValue('T_ROB1', 'TestModule', 'TestData', x.toString());
```

Example

Lets assume that a Record is declared as follows:

```
Record TestRecord
    num Num1;
    num Num2;
    string String1;
EndRecord

PERS TestRecord TestSimple := [0,1,"OK"];
```

Then its string component can be set as follows:

```
var str1 = "Testing raw write";
await RWS.Rapid.setDataValue('T_ROB1', 'TestModule', 'TestSimple', str1, 3);
```

Example

Lets assume that an Array is declared as follows:

```
PERS num TestArray25{2,5} := [[0,0,0,0,0], [0,0,0,0,0]];
```

Then the element in position [2,3] can be set as follows:

```
var e23 = '100';  
await RWS.Rapid.setDataValue('T_ROB1', 'TestModule', 'TestArray25', e23, 2, 3);
```

getSharedData

Signature: `getSharedData(string symbolName)`

`symbolName` the name of the symbol

Gets a Data object for symbol that is declared in a module that is shared across all tasks. This will retrieve the properties for the symbol from the controller and initialize the object.

Return value

A Promise with an initialized Data object.

setSharedDataValue

Signature: `setSharedDataValue(string symbolName, string value)`

`symbolName` the name of the symbol

`value` the new value

Sets the value of a symbol that is declared in a module that is shared across all tasks. This requires the value string to be on the RWS format, and as that can be a bit complicated it is recommended that the `setValue` function on an instance of `RWS.Rapid.Data` is used instead.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

getExecutionState

Signature: `getExecutionstate()`

Gets the current execution state of the controller.

Return value

A Promise with a string containing the execution state.

Possible values: 'running' or 'stopped'

resetPP

Signature: `resetPP()`

Moves the program pointer to Main (defined entrypoint) in all tasks.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

startExecution

Signature: `startExecution({})`

`{}` object containing;

<code>(string) regainMode</code>	regain mode, valid values: 'continue', 'regain', 'clear' or 'enter_consume'
<code>(string) executionMode</code>	execution mode, valid values: 'continue', 'step_in', 'step_over', 'step_out', 'step_backwards', 'step_to_last' or 'step_to_motion'
<code>(string) cycleMode</code>	cycle mode, valid values: 'forever', 'as_is' or 'once'
<code>(string) condition</code>	condition, valid values: 'none' or 'callchain'
<code>(boolean) stopAtBreakpoint</code>	stop at breakpoint
<code>(boolean) enableByTSP</code>	all tasks according to task selection panel

Starts the Rapid execution with the settings given in the parameter object. All or any of the defined parameters can be supplied, if a value is omitted a default value will be used. The default values are:

```
regainMode = 'continue'
execMode = 'continue'
cycleMode = 'forever'
condition = 'none'
stopAtBreakpoint = true
enableByTSP = true
```

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

Example

```
try {
  await RWS.Rapid.resetPP();
  await RWS.Controller.setMotorsState('motors_on');
  await RWS.Rapid.startExecution({
    regainMode: 'continue',
    executionMode: 'continue',
    cycleMode: 'once',
    condition: 'none',
```

```

        stopAtBreakpoint: false,
        enableByTSP: true
    });
} catch(error) {
    myErrorHandler(error);
}

```

stopExecution

Signature: stopExecution({})

{}

(string) stopMode	stop mode, valid values: 'cycle', 'instruction', 'stop' or 'quick_stop'
(string) useTSP	use task selection panel, valid values: 'normal' or 'all_tasks'

Stops the Rapid execution with the settings given in the parameter object. All or any of the defined parameters can be supplied, if a value is omitted a default value will be used. The default values are:

```

stopMode = 'stop'
useTSP = 'normal'

```

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

Example

```

var state = await RWS.Rapid.getExecutionState();
if (state === 'running'){
    await RWS.Rapid.stopExecution({
        stopMode: 'cycle',
        useTSP: 'normal'
    });
    await setTimeout(() => {}, 2000);
    state = await RWS.Rapid.getExecutionState();
    if (state === 'stopped'){
        console.log('We stopped Rapid.');

```

searchSymbols

Signature: searchSymbols({} properties, string dataType, string regexp)

properties search properties (see getDefaultSearchProperties below)

`dataType` the name of the data type to look for, can be empty
`regexp` regular expression, can be empty

Searches for Rapid symbols as defined in the search parameters. The method can be used to search for variables, routines and data types.

If the data type is declared in a module that is installed or shared, only the data type's name should be provided in the `dataType` parameter, otherwise the full path is required. The flags "isInstalled" and "isShared" in the `properties` parameter must also be set accordingly.

Return value

A Promise with list of objects. Each object contains:

<code>(string) name</code>	name of the data symbol
<code>([string]) scope</code>	symbol scope
<code>(string) symbolType</code>	type of the symbol, e.g. 'pers '
<code>(string) dataType</code>	type of the data, e.g. 'num '

Example

```
var properties = RWS.Rapid.getDefaultSearchProperties();
properties.searchURL = 'RAPID/T_ROB1/TestModule';
properties.types = RWS.Rapid.SymbolTypes.rapidData;
properties.isInUse = false;

var hits = await RWS.Rapid.searchSymbols(properties);
if (hits.length > 0){
    for (let i = 0; i < hits.length; i++){
        if (hits[i].name === 'TestData'){
            console.log(JSON.stringify(hits[i]));
        }
    }
}
```

Example

```
var properties = RWS.Rapid.getDefaultSearchProperties();
properties.searchURL = 'RAPID/T_ROB1/TestModule';
properties.types = RWS.Rapid.SymbolTypes.rapidData;
properties.isInUse = false;

var hits = await RWS.Rapid.searchSymbols(properties, 'num', 'Test[a-zA-Z]*');
if (hits.length <= 0) {
    console.log('Could not find test data.');
```

getDefaultSearchProperties

Signature: `getDefaultSearchProperties()`

Returns an object with a default set of properties for searching Rapid symbols.

NOTE! Symbols in a shared module will reside directly under 'RAPID', i.e. use 'Rapid' as `searchURL`.

NOTE! Symbols in an installed module require the flag "#SYS" instead of module name, i.e. use a string like 'Rapid/T_ROB1/#SYS' as `searchURL`.

The fields are:

<code>(SearchMethods) method</code>	search methods (see enum <code>SearchMethods</code>)
<code>(string) searchURL</code>	URL to the starting point
<code>(SymbolTypes) types</code>	symbol type (see enum <code>SymbolTypes</code>)
<code>(boolean) isInstalled</code>	is the module containing the datatype installed or loaded, true if an installed module
<code>(boolean) isShared</code>	is the module containing the searched datatype declared in a shared module, true if a shared module
<code>(boolean) recursive</code>	search recursively, i.e. nested symbols like components for a RECORD type will be returned
<code>(boolean) skipShared</code>	skip all results that are declared in shared modules
<code>(boolean) isInUse</code>	only return symbols that are used in a Rapid program, i.e. a type declaration that has no declared variable will not be returned when this flag is set true.

The default values are:

```
method = SearchMethods.block
searchURL = 'RAPID'
types = SymbolTypes.any
isInstalled = false
isShared = false
recursive = true
skipShared = false
isInUse = false
```

Return value

An object with default search properties.

SearchMethods

Supported search method in searchSymbols. Specifies the direction of the search, which can be block (down) or scope (up). This means that if the search uses “block” all symbols matching the search criteria in the path declared in “searchURL” and forward will be returned. If using “scope”, all symbols matching the search criteria that can be accessed from the path declared in “searchURL” will be returned.

The fields are as follows:

block: 1	block mode (down from searchURL)
scope: 2	scope mode (up from searchURL)

SymbolTypes

Valid symbol types to search for using searchSymbols. This is a flag style enum, i.e. fields can be combined by adding the values of several fields.

NOTE! Not all combinations are relevant, but no errors will be issued if the value is senseless.

The fields are as follows:

undefined: 0	value not set
constant: 1	Rapid constant
variable: 2	Rapid var declared variable
persistent: 4	Rapid pers declared variable
function: 8	Rapid function
procedure: 16	Rapid procedure
trap: 32	Rapid trap
module: 64	Rapid module
task: 128	Rapid task
routine: 8 + 16 + 32	function, procedure or trap
rapidData: 1 + 2 + 4	constant, variable or persistent
any: 255	any of the above

4.4.1 Monitor Object

Description

Monitor is an object used to subscribe to resources that have no other object that can be subscribed.

To create an Monitor object use the factory method `getMonitor(resource, task)` in the `RWS.Rapid` namespace.

The Monitor object can subscribe to changes in rapid execution, program pointer, motion pointer and UI instructions.

Each instance of the Monitor object can subscribe to one resource.

NOTE! The program pointer and motion pointer are due to limitations in other parts of the controller software not usable to follow program execution when Rapid is executing.

Resources

execution

Changes to the state of the Rapid execution. The events will return a string as:

'running'	Rapid is running
'stopped'	Rapid is stopped

program-pointer

Changes to the program pointer.

NOTE! Due to limitations in the underlying architecture events for all changes may not be received. Changes faster than 200 ms will be filtered.

The events will return a program pointer object with the properties:

(string) moduleName	name of the module
(string) routineName	name of the routine
(string) beginPosition	position of start in program file, e.g. '89,9' - row = 89 and column = 9
(string) endPosition	position of end in program file
(boolean) hasValue	flag indicating if data in this object is valid

motion-pointer

Changes to the motion pointer.

NOTE! Due to limitations in the underlying architecture events for all changes may not be received. Changes faster than 200 ms will be filtered.

The events will return a motion pointer object with the properties:

(string) moduleName	name of the module
(string) routineName	name of the routine
(string) beginPosition	position of start in program file, e.g. '89,9' - row = 89 and column = 9
(string) endPosition	position of end in program file
(boolean) hasValue	flag indicating if data in this object is valid

uiinstr

Events when UI instructions' states change. UI instructions are all RAPID instructions that present information in the operator messages view, and may or may not require operator responses. Typical UI instructions are *TPWrite*, *TPReadFK*, *UIMessageBox*, etc.

The events will return an object with the properties:

(string) instruction	instruction type
----------------------	------------------

(string) event	event type, valid values: 'send' 'post' 'abort'
(string) task	RAPID task that executed the instruction
(string) message	message of the instruction
(string) executionLevel	execution level, valid values: 'user' 'normal'
(string) id	unique id of the instruction call, a 'send' event can be matched with an 'abort' event using this id
({}) parameters	an object containing the instructions parameters, which varies depending of instruction and what parameters are provided in the instruction call

Methods

getTitle

Signature: `getTitle()`

Used internally. Gets the title (part of a URL) of the subscribed resource.

Return value

A string containing the title.

getResourceString

Signature: `getResourceString()`

Gets a unique resource URL for subscription.

Return value

A string containing a unique resource URL (as used by RWS.)

addCallbackOnChanged

Signature: `addCallbackOnChanged(function callback)`

`callback` a callback function to run when event occurs

Adds a callback function which will be called when the subscribed resource's state changes.

The callback routine can have one in-parameter, which is the new value of the subscribed resource. Contents of the value depends on the resource subscribed to (see section 4.4.1).

Return value

No return value.

Example

```
var counter = 0;
var executionMonitor = RWS.Rapid.getMonitor('execution');
executionMonitor.addCallbackOnChanged(eventData => {
    console.log(`New execution state: ${eventData}.`);
});
```

onchanged

This is a function for internal use only. Do not use from external scripts.

subscribe

Signature: `subscribe(boolean raiseInitial = false)`

`raiseInitial` flag indicating whether an initial event is raised when subscription is registered

Starts the subscription to the resource defined at instantiation.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

Example

```
var counter = 0;
var executionMonitor = RWS.Rapid.getMonitor('execution');
executionMonitor.addCallbackOnChanged((newValue) => {
    var c = (counter++).toString();
    console.log(`Execution state changed: ${newValue} (${c}).`);
});

try {
    await executionMonitor.subscribe();
} catch(error) {
    console.error(`Subscription to 'execution' failed. >>> ${error}`);
}
```

unsubscribe

Signature: `unsubscribe()`

Stops the subscription to the resource defined at instantiation.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

4.4.2 Task Object

Description

The Task object corresponds to a Rapid task on the controller. It provides functionality to access modules, data and service routines.

To get a Task object use the factory method `getTask()` directly in the `RWS.Rapid` namespace.

Nested Objects

The Task object contains a nested object, `ServiceRoutine`, which is used to handle service routines.

Methods

`getName`

Signature: `getName()`

Gets the name of the task.

Return value

A string containing the name of the task.

`getProperties`

Signature: `getProperties()`

Gets a set of properties of the task. The function returns an object with the properties.

Note that all properties are not necessarily available at all times. The user should check if the property exists (see example.)

Return value

A promise with an object containing:

<code>(string) name</code>	task name
<code>(string) taskType</code>	task type, valid values: 'normal' 'static' 'semistatic' 'unknown'
<code>(string) taskState</code>	task state, valid values: 'empty' 'initiated'

	'linked'
	'loaded'
	'uninitialized'
(string) executionState	execution state, valid values:
	'ready'
	'stopped'
	'started'
	'uninitialized'
(string) activeState	active state, valid values:
	'on'
	'off'
(boolean) isMotionTask	flag indicating if task is a motion task
(string) trustLevel	trust level, valid values:
	'sys_fail'
	'sys_halt'
	'sys_stop'
	'none'
(number) id	identifier
(string) executionLevel	current execution level, valid values:
	'none'
	'normal'
	'trap'
	'user'
	'unknown'
(string) executionMode	current execution mode, valid values:
	'continuous'
	'step_over'
	'step_in'
	'step_out_of'
	'step_back'
	'step_last'
	'stepwise'
	'unknown'
(string) executionType	current execution type, valid values:
	'none'
	'normal'
	'interrupt'

	'external_interrupt'
	'user_routine'
	'event_routine'
	'unknown'
(string) progEntrypoint	program entry point (the configured routine that will be activated by PP to Main calls)
(boolean) bindRef	flag indicated if task is bound
(string) taskInForeground	name of task in foreground

Example

```
var task = await RWS.Rapid.getTask('T_ROB1');
var properties = task.getProperties();
if (properties.hasOwnProperty('executionType')) {
    console.log(`Execution type = ${properties['executionType']}`);
}
```

getServiceRoutines

Signature: `getServiceRoutines()`

Gets a list of `ServiceRoutine` (see 4.4.3) objects for the service routines on the task.

Return value

A Promise with a list of available `ServiceRoutine` objects.

Example

```
// Move program pointer to first service routine
var task = await RWS.Rapid.getTask('T_ROB1');
var serviceRoutines = await task.getServiceRoutines();
if (serviceRoutines.length > 0) {
    serviceRoutines[0].setPP();
}
```

getData

Signature: `getData(string moduleName, string symbol)`

`moduleName` the name of the module

`symbol` the name of the symbol

Gets the Data object (described later in this chapter) for a symbol. The call will try to get information from the controller to initialize the object.

Return value

A Promise with a Data object

Example

```
var task = await RWS.Rapid.getTask('T_ROB1');  
var data = await task.getData('TestModule', 'TestData');  
data.setValue('Test String');
```

getProgramInfo

Signature: `getProgramInfo()`

Gets program information, program name and entrypoint, for the task.

Return value

A promise with an object containing:

<code>(string) name</code>	program name
<code>(string) entrypoint</code>	name of function that is currently the entry point (i.e. the routine where execution will start)

getModuleNames

Signature: `getModuleNames()`

Gets module names for the task. This will return both program modules and system modules.

Return value

A Promise with an object containing:

<code>([string]) programModules</code>	list of the names of all program modules
<code>([string]) systemModules</code>	list of the names of all system modules

getModule

Signature: `getModule(string moduleName)`

`moduleName` the name of the module

Gets a module on the task.

Return value

A Promise with a Module object

getPointers

Signature: `getPointers()`

Gets information about the program pointer and the motion pointer. The pointers may not have values and the corresponding `hasValue` flag should always be checked before use.

Return value

A Promise with an object containing:

<code>({}) programPointer</code>	an object with pointer information
<code>({}) motionPointer</code>	an object with pointer information

Pointer information contains:

<code>(string) moduleName</code>	name of the module
<code>(string) routineName</code>	name of the routine
<code>(string) beginPosition</code>	position of start in program file, e.g. '89,9' - row = 89 and column = 9
<code>(string) endPosition</code>	position of end in program file
<code>(boolean) hasValue</code>	flag indicating if value in this object is valid

Example

```
var task = await RWS.Rapid.getTask('T_ROB1');
var pointers = await task.getPointers();
if (pointers.programPointer.hasValue === true) {
    var r = pointers.programPointer['routineName'];
    console.log(`PP is in routine '${r}'`);
}
```

movePPToRoutine

Signature: `movePPToRoutine(string routineName, boolean userLevel = false, string moduleName = undefined)`

<code>routineName</code>	the name of the routine
<code>userLevel</code>	flag indicating if execution is to be on user level
<code>moduleName</code>	optional module name

Moves the program pointer for the task to a routine. Optionally the execution level can be set to “user”, i.e., execute as a service routine.

Optionally a module name can be specified.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

abortServiceRoutine

Signature: `abortServiceRoutine()`

Aborts the current execution level of the task, under the condition that the execution level is “user”, i.e., a service routine.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

4.4.3 ServiceRoutine Object

Description

The ServiceRoutine object is nested in the Task object. It provides a basic interface to facilitate executing service routines.

ServiceRoutine objects cannot be explicitly created, they are returned as results of other functions.

The execution of the service routine is controlled with methods in the `RWS.Rapid` namespace, i.e. `startExecution` and `stopExecution`.

Methods

`getName`

Signature: `getName()`

Gets the name of the service routine.

Return value

A string containing the name of the service routine.

`getUrl`

Signature: `getUrl()`

Gets an RWS URL for the service routine. Used internally by api.

Return value

A string containing the URL.

`setPP`

Signature: `setPP()`

Set program pointer to the service routine.

After the program pointer has been set; use `RWS.Rapid.startExecution()` to run the routine.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

4.4.4 Module Object

Description

The Module object corresponds to a Rapid module on the controller. It provides functionality to access Rapid data.

To get an Module object use the factory method `getModule()` in the `RWS.Rapid` namespace.

Methods

`getName`

Signature: `getName()`

Gets the name of the module.

Return value

A string containing the module name.

`getTaskName`

Signature: `getTaskName()`

Gets the name of the task that contains the module.

Return value

A string containing the task name.

`getProperties`

Signature: `getProperties()`

Gets the properties of the module.

Return value

A Promise with an object containing:

<code>(string) taskName</code>	task's name
<code>(string) moduleName</code>	module's name
<code>(string) fileName</code>	the name of the file the module is saved as
<code>(string) attributes</code>	attributes

Example

```
var module = await RWS.Rapid.getModule('T_ROB1', 'TestModule');
var properties = await module.getProperties();
console.log(`Module file name is '${properties.fileName}'`);
```

getData

Signature: `getData(string symbolName)`

`symbolName` the name of the symbol

Gets the Data object for a symbol. The call the controller to initialize the object with current properties. This will verify that the symbol exists. Note that the value of the is not read and must be read explicitly.

Return value

A Promise with a Data object.

Example

```
var module = await RWS.Rapid.getModule('T_ROB1', 'TestModule');
var data = await module.getData('TestData');
var testValue = data.getValue();
```

4.4.5 Data Object

Description

The Data object corresponds to a Rapid data on the controller. It provides functionality to access read, write and subscribe to the data.

To get an Data object use the factory method `getData()` in the `RWS.Rapid` namespace, or the Task and Module objects.

Methods

getTitle

Signature: `getTitle()`

Used internally. Gets the title (part of a URL) of the subscribed resource.

Return value

A string containing the title.

getProperties

Signature: `getProperties()`

Gets the basic properties of the data. A call to this method will refresh the contents of the object.

NOTE! Calling the individual property accessors will return the values from the last call to this method and are consequently not necessarily the actual values.

Return value

A Promise with an object containing:

(string) taskName	task's name
(string) moduleName	module's name
(string) symbolName	symbol's name
(string) dataType	symbol's data type
(string) symbolType	the declaration type of the data, valid values: 'constant' 'variable' 'persistent'
([number]) dimensions	list of dimensions for arrays
(string) scope	the data's scope, valid values: 'local' 'task' 'global'
(string) dataTypeURL	RWS URL to the data's type symbol

Example

```
var data = await RWS.Rapid.getData('T_ROB1', 'TestModule', 'TestData');  
var properties = await data.getProperties();  
console.log(`Data type is '${properties.dataType}'`);
```

getName

Signature: `getName()`

Gets the name of the symbol.

Return value

A string containing the symbol name.

getModuleName

Signature: `getModuleName()`

Gets the name of the module containing the symbol.

Return value

A string containing the module name.

getTaskName

Signature: `getTaskName()`

Gets the name of the task containing the symbol.

Return value

A string containing the task name.

getDataType

Signature: `getDataType()`

Gets the data type of the symbol, i.e. Rapid data types *num*, *string*, etc.

Return value

A promise with a string containing the data type.

getSymbolType

Signature: `getSymbolType()`

Gets the symbol declaration type of the data.

Return value

A promise with a string containing the symbol type.

Possible values: `'constant'`, `'variable'` or `'persistent'`

getDimensions

Signature: `getDimensions()`

Gets the dimensions of the symbol, if the symbol is an array or multi-dimensional matrix. The method returns a list of number which corresponds to the dimensions in order. If the symbol is atomic (i.e. has no dimensions) the return value will be an empty list.

Return value

A promise with a list of numbers.

getScope

Signature: `getScope()`

Gets the data's scope. The value indicates how accessible the data is, i.e. local, global or task.

Return value

A promise with a string.

Possible values: `'local'`, `'task'` or `'global'`

getTypeURL

Signature: `getTypeURL()`

Gets the RWS URL to the data type's definition of the symbol. For complex data types, e.g. Record types, this is the entry point for when evaluating the value of the symbol.

Return value

A promise with a string containing the data type URL.

getValue

Signature: `getValue()`

Gets the value of the symbol. This will return the latest retrieved value. If value has not been retrieved yet, it will be retrieved by this call. Use the `fetch` function to update the value before subsequent calls to this method.

The return value will be parsed and returned as a JavaScript value corresponding to the Rapid value, i.e. a num will be returned as number, an array will be returned as an JavaScript array (`[]`), a Record will be returned as an object with properties named as the record components in Rapid, etc.

Return value

A promise with a parsed data value.

Example

Lets assume that a Record is declared as follows:

```
Record TestRecord
    num Num1;
    num Num2;
    string String1;
EndRecord
```

Then it can be read as follows:

```
var data = await RWS.Rapid.getData('T_ROB1', 'TestModule', 'TestRecord');
var testValue = await data.getValue();
if (testValue.Num1 > 17) {
    console.log(`Num1 = '${testValue.Num1}' is greater than 17.`);
}
console.log(testValue.String1);
```

getArrayItem

Signature: `getArrayItem(...number indexes)`

`indexes` a parameter list of numbers

Gets the value of an element in an array specified by the provided indexes. If the array is multidimensional an index can be provided for each dimension. If the number of indexes or the indexes are greater than the arrays size in that dimension, the call will be rejected.

The indexes are 1-based as in Rapid.

Return value

A promise with a parsed data value.

Example

Lets assume that an array is declared as follows:

```
PERS num TestArray52{5,2} := [[0,1],[0,4],[0,9],[1,2],[2,5]];
```

Then its elements can be read as follows:

```
var array = await RWS.Rapid.getData('T_ROB1','TestModule','TestArray52');
var number = await array.getItem(3,2);
if (number === 9) console.log('Correctly read number from array.');
```

```
var tuple = await array.getItem(3);
```

```
if (tuple === [0,9]) console.log('Correctly read tuple from array.');
```

getRecordItem

Signature: `getRecordItem(...string components)`

`components` a parameter list of component names

Gets the value of an element in a record specified by the provided component names. If the record contains nested records, several names can be provided to get a value deeper in the hierarchy. If the list of component names cannot be traversed, the call will be rejected.

Return value

A promise with a parsed data value.

Example

Lets assume Records declared as follows:

```
RECORD TestRecord
    num Code;
    num State;
    string Status;
ENDRECORD
```

```
RECORD NestedTestRecord
    TestRecord InnerTest;
    TestRecord OuterTest;
ENDRECORD
```

```
PERS NestedTestRecord TestNested := [[0,1,"OK"],[0,2,"Error"]];
```

Then the components can be read as follows:

```
var rec = await RWS.Rapid.getData('T_ROB1','TestModule',' TestNested ');
```

```
var status = await rec.getRecordItem('InnerTest', 'Status');
if (status === 'OK') console.log('Inner test is OK.');
```

```
status = await rec.getRecordItem('OuterTest', 'Status');
if (status === 'OK') console.log('Outer test is OK.');
```

getRawValue

Signature: `getRawValue()`

Gets the RWS string value of the symbol. This will return the latest retrieved value. If the value has not been retrieved yet, it will be retrieved by this call. Use the `fetch` function to update the value before subsequent calls to this method.

NOTE! Use this function only if you know the format of the RWS string and need to optimize for speed.

Return value

A promise with a string containing the data value.

fetch

Signature: `fetch()`

Retrieves the latest value from the controller. The value is stored in the object and can be accessed using a call to `getValue()`.

NOTE! If the Data object has an active subscription to the value, it is not necessary to call `fetch`. The value will be updated by the events.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

Example

```
var testValue1 = '';
var testValue2 = '';
var data1 = await RWS.Rapid.getData('T_ROB1', 'TestModule', 'TestData1');
var data2 = await RWS.Rapid.getData('T_ROB1', 'TestModule', 'TestData2');

...

async function updateValues() {
  try {
    await data1.fetch();
    await data2.fetch();
    testValue1 = await data1.getValue();
    testValue2 = await data2.getValue();
  } catch (error) {
    console.error(`We have a problem. ${error}`);
  }
}
```

setValue

Signature: `setValue({} value)`

value object or atomic with the new value

Sets the value of the symbol to the value provided.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

Example

Lets assume that a Record is declared as follows:

```
Record TestRecord
    num Num1;
    num Num2;
    string String1;
EndRecord
```

Then it can be set as follows:

```
var data = {
    Num1: 17,
    Num2: 42,
    String1: 'Some random numbers'
};
var record = await RWS.Rapid.getData('T_ROB1', 'TestModule', 'TestRecord');
await record.setValue(data);
```

setArrayItem

Signature: `setArrayItem(any value, ...number indexes)`

value object or atomic with the new value

indexes a parameter list of numbers

Sets the value of an element in an array specified by the provided indexes. If the array is multidimensional an index can be provided for each dimension. If the number of indexes or the indexes are greater than the arrays size in that dimension, the call will be rejected.

The indexes are 1-based as in Rapid.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

Example

Lets assume that an array is declared as follows:

```
PERS num TestArray25{2,5} := [[0,1,0,4,0],[9,1,2,2,5]];
```

Then its elements can be written as follows:

```
var array = await RWS.Rapid.getData('T_ROB1','TestModule','TestArray25');
await array.setArrayItem(42,2,3); // will write the value 42 to position 2,3
```

setRecordItem

Signature: setRecordItem(any value, ...string components)

value object or atomic with the new value

components a parameter list of component names

Sets the value of an element in a record specified by the provided component names. If the record contains nested records, several names can be provided to get a value deeper in the hierarchy. If the list of component names cannot be traversed, the call will be rejected.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

Example

Lets assume Records declared as follows:

```
RECORD TestRecord
    num Code;
    num State;
    string Status;
ENDRECORD

RECORD NestedTestRecord
    TestRecord InnerTest;
    TestRecord OuterTest;
ENDRECORD

PERS NestedTestRecord Test := [[0,1,"OK"],[0,2, "Error"]];
```

Then the components can be written as follows:

```
var rec = await RWS.Rapid.getData('T_ROB1','TestModule','Test');
await rec.setRecordItem('OK', 'OuterTest', 'Status');
await rec.setRecordItem(7, 'OuterTest', 'State');
var inner = {Code:0, State: 2, Status: 'Warning'};
await rec.setRecordItem(inner, 'InnerTest');
```

setRawValue

Signature: setRawValue(string value, ...number indexes)

`value` the new value

`indexes` an optional list of numbers that specify an element in arrays and records

Sets the value of the symbol to the value provided. This requires the value string to be on the RWS format, and as that can be a bit complicated it is recommended that the `setValue` function is used instead.

By providing indexes, in the in-parameter `indexes`, a specific element in an array (multidimensional) or record can be set independent of the other elements in the structure. The indexes are 1-based as in Rapid.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

getResourceString

Signature: `getResourceString()`

Gets a URL to the resource. This URL is used to identify the resource when subscribing.

Return value

A string containing a unique resource URL (as used by RWS.)

addCallbackOnChanged

Signature: `addCallbackOnChanged(function callback)`

`callback` a callback function to run when event occurs

Adds a callback function which will be called when the subscribed symbol's value changes.

The callback can have one argument, which will contain the data value. The value will be the same as received when `getRawValue` is called. This means that the value will be in raw RWS format. To get a more friendly parsed value, use the `getValue` function available on the data object from within the callback function.

Return value

No return value.

onchanged

This is a function for internal use only. Do not use from external scripts.

subscribe

Signature: `subscribe(boolean raiseInitial = false)`

`raiseInitial` flag indicating whether an initial event is raised when subscription is registered

Subscribes to changes on the value of the symbol.

IMPORTANT! Note that only persistent data variables (`PERS`) support subscriptions.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

Example

```
var data = await RWS.Rapid.getData('T_ROB1', 'TestModule', 'TestData');
data.addCallbackOnChanged((newValue) => {
    console.log(`Testdata value changed: ${newValue}.`);
});

try {
    await data.subscribe();
} catch(error) {
    var resource = data.getResourceString();
    console.error(`Subscription to '${resource}' failed. >>> ${error}`);
}
```

unsubscribe

Signature: `unsubscribe()`

Stops the subscription to changes on the value of the symbol.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

4.5 RWS.IO Namespace

Description

The IO namespace, `RWS.IO`, contains methods to manipulate I/O signals.

The namespace contains the following objects; Network, Device and Signal. These are described closer in this section.

Methods**getNetwork**

Signature: `getNetwork(string networkName)`

`networkName` the name of the network

Gets a Network object. The presence of the network is verified and if it does not exist the call will be rejected.

Return value

A Promise with a Network object.

getDevice

Signature: `getDevice(string networkName, string deviceName)`

`networkName` the name of the network

`deviceName` the name of the device

Gets a Device object. The presence of the device is verified and if it does not exist the call will be rejected.

Return value

A Promise with a Device object.

getSignal

Signature: `getSignal(string signal)`

`signal` the name of the signal

Gets a Signal object. The presence of the signal is verified and if it does not exist the call will be rejected. The basic properties for the signal are retrieved from the controller, however the value is not retrieved until any of the methods `getValue()` or `fetch()` is called.

Return value

A Promise with a Signal object.

Example

```
var signal = await RWS.IO.getSignal('TestDI');
var testValue = await signal.getValue();
if (testValue === 1){
    mySetSomeIndicator(1);
}
```

setSignalValue

Signature: `setSignalValue(string signal, number value)`

`signal` the name of the signal

`value` the new value of the signal

Sets a value on a signal. The call will fail if the value string is not valid for the signal type.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

searchSignals

Signature: `searchSignals({} filter)`

`filter` an object with filter information:

(string) name	signal name
(string) device	device name
(string) network	network name
(string) category	category string
(string) category-pon	
(string) type	type of signal, valid values: 'DI', 'DO', 'AI', 'AO', 'GI' or 'GO'
(boolean) invert	inverted signals
(boolean) blocked	blocked signals

Search system for signals matching the filter. The signal `name` does not have to be an exact match, any signal which name begins with the string will be returned.

The `type` filter can contain several types by grouping the values within '[' and ']'. The signal types should be separated by comma characters, e.g. '[DI,DO]'.

Return value

A Promise with a list of Signal objects that match the filters.

Example

```
var filter = {
  name: 'TestDI',
}
var signals = await RWS.IO.searchSignals(filter);
if (signals.length > 0){
  for (let i = 0; i < signals.length; i++){
    var testValue = await signals[i].getValue();
    console.log(`${signals[i].getName()} = ${testValue}`);
  }
}
```

Example

```
var filter = {
  name: 'Test',
  type: '[DI,DO]'
}
var signals = await RWS.IO.searchSignals(filter);
if (signals.length > 0){
  for (let i = 0; i < signals.length; i++){
    var testValue = await signals[i].getValue();
    console.log(`${signals[i].getName()} = ${testValue}`);
  }
}
```

4.5.1 Network Object

Description

The Network object represents an IO network and can be used to check the states of the network and access signals.

Network contains two states, physical state and logical state, whose values are not updated unless explicitly ordered by calling `fetch()`. The first time any of the states are accessed, `fetch()` is called implicitly and both states are updated.

Methods

`getName`

Signature: `getName()`

Gets the name of the Network.

Return value

A string containing the network's name.

`getPhysicalState`

Signature: `getPhysicalState()`

Gets the physical state of the Network. If the Network instance has not been accessed before, it will retrieve current states from the controller and then return the physical state. If the Network instance has been accessed, the `fetch()` function must be called first to ensure that the states are up to date.

Return value

A promise with a string containing the physical state.

Possible values: 'halted', 'running', 'error', 'startup', 'init' or 'unknown'

Example

```
var device = await RWS.IO.getDevice('TestNet', 'TestDevice');
var state = await device.getPhysicalState();
if (state !== 'running') {
    myHandleDeviceState('TestDevice', state);
}
```

`getLogicalState`

Signature: `getLogicalState()`

Gets the logical state of the Network. If the Network instance has not been accessed before, it will retrieve current states from the controller and then return the logical state. If

the Network instance has been accessed, the `fetch()` function must be called first to ensure that the states are up to date.

Return value

A promise with a string containing the logical state.

Possible values: 'stopped', 'started' or 'unknown'

fetch

Signature: `fetch()`

Retrieves the current states from the controller. The values are stored in the object and can be accessed using calls to `getLogicalState()` or `getPhysicalState()`.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

4.5.2 Device Object

Description

The Device object represents an IO device and can be used to check the states of the device and get signals that are configured on the device.

Device contains two states, physical state and logical state, whose values are not updated unless explicitly ordered by calling `fetch()`. The first time any of the states are accessed, `fetch()` is called implicitly and both are values updated.

Methods

getName

Signature: `getName()`

Gets the name of the Device.

Return value

A string containing the device's name.

getNetworkName

Signature: `getNetworkName()`

Gets the name of the Network on which the Device resides.

Return value

A string containing the network's name.

getPhysicalState

Signature: `getPhysicalState()`

Gets the physical state of the Device. If the Device instance has not been accessed before, it will retrieve current states from the controller and then return the physical state. If the Device instance has been accessed, the `fetch()` function must be called first to ensure that the states are up to date.

Return value

A promise with a string containing the physical state.

Possible values: 'deact', 'running', 'error', 'unconnect', 'unconfg', 'startup', 'init' or 'unknown'

getLogicalState

Signature: `getLogicalState()`

Gets the logical state of the Device. If the Device instance has not been accessed before, it will retrieve current states from the controller and then return the logical state. If the Device instance has been accessed, the `fetch()` function must be called first to ensure that the states are up to date.

Return value

A promise with a string containing the logical state.

Possible values: 'disabled', 'enabled' or 'unknown'

getNetwork

Signature: `getNetwork()`

Gets an object for the parent network.

Return value

A Network object.

getSignal

Signature: `getSignal(string signalName)`

`signalName` the name of the signal

Gets a Signal object with properties retrieved from the controller. The signal must reside on the current device. The value is not retrieved until one of the methods `getValue()` or `fetch()` is called.

Return value

A promise with a Signal object.

4.5.3 Signal Object

Description

The Signal object represents an IO signal and can be used to read and set signals value as well as checking some properties.

The signal's properties and value are not updated unless explicitly ordered by calling `fetch()`. The first time any property or value are accessed, `fetch()` is called implicitly and all is updated.

Methods

`getName`

Signature: `getName()`

Gets the name of the Signal.

Return value

A string containing the signal's name.

`getPath`

Signature: `getPath()`

Gets the part of a URL that is the signal's path.

Return value

A string containing the signal's path.

`getNetworkName`

Signature: `getNetworkName()`

Gets the name of the network the Signal resides on.

Return value

A string containing the network's name.

`getDeviceName`

Signature: `getDeviceName()`

Gets the name of the device the Signal resides on.

Return value

A string containing the device's name.

`getTitle`

Signature: `getTitle()`

Used internally. Gets the title (part of a URL) of the subscribed resource.

Return value

A string containing the signal's title.

`getIsSimulated`

Signature: `getIsSimulated()`

Gets the flag indicating whether the signal is simulated. This will update the value at the first call, but subsequent calls must call `fetch()` first.

Return value

A Promise with a boolean.

`getQuality`

Signature: `getQuality()`

Gets the quality of the last update of the signal.

Return value

A Promise with a string.

Possible values: 'bad', 'good' or 'unknown'

`getCategory`

Signature: `getCategory()`

Gets the category field of the signal, as defined in the configuration database.

Return value

A Promise with a string.

`getAccessLevel`

Signature: `getAccessLevel()`

Gets the access level field of the signal, as defined in the configuration database.

Return value

A Promise with a string.

`getWriteAccess`

Signature: `getWriteAccess()`

Gets the write access field of the signal, as defined in the configuration database.

Return value

A Promise with a string.

getSafeLevel

Signature: `getSafeLevel()`

Gets the safe level field of the signal, as defined in the configuration database.

Return value

A Promise with a string.

getType

Signature: `getType()`

Gets the type field of the signal, as defined in the configuration database.

Return value

A Promise with a string.

getDevice

Signature: `getDevice()`

Gets the Device object for the device this signal resides on.

NOTE! This will create a new object and not return an existing one if one exists.

Return value

A Promise with a Device object.

getValue

Signature: `getValue()`

Gets the signals value. This will update the value at the first call, but subsequent calls must call `fetch()` first.

Return value

A Promise with a number.

setValue

Signature: `setValue(number value)`

value the new value

Sets the value of the signal to the value provided.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

addCallbackOnChanged

Signature: `addCallbackOnChanged(function callback)`

`callback` a callback function to run when event occurs

Adds a callback function which will be called when the subscribed signal's value changes.

The callback can have one argument, which will contain the signal value (same as is received when `getValue` is called.)

Return value

No return value.

onchanged

This is a function for internal use only. Do not use from external scripts.

subscribe

Signature: `subscribe(boolean raiseInitial = false)`

`raiseInitial` flag indicating whether an initial event is raised when subscription is registered

Starts the subscription to changes on the value of the signal.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

unsubscribe

Signature: `unsubscribe()`

Stops the subscription to changes on the value of the signal.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

Example

```
var signal = await RWS.IO.getSignal('TestDI');
signal.addCallbackOnChanged((newValue) => {
    console.log(`Test signal value changed: ${newValue}.`);
});

try {
    await signal.subscribe();
} catch(error) {
    var resource = signal.getResourceString();
```

```
        console.error(`Subscribe to '${resource}' failed. >>> ${error}`);
    }

    ...

    try {
        await signal.unsubscribe();
    } catch(error) {
        var resource = signal.getResourceString();
        console.error(`Unsubscribe to '${resource}' failed. >>> ${error}`);
    }
```

4.6 RWS.CFG Namespace

Description

The `RWS.CFG` namespace, contains methods to read and write to the configuration database (the controller's system parameters.)

The namespace contains the following objects; Domain, Type and Instance. These are described closer in this section.

There are also a set of methods that are available directly in the `RWS.CFG` namespace.

NOTE! The identifiers for the domains, types, instances and attributes can easily be determined by creating an instance and then saving that domain to file. In that file the identifiers can be found by looking at the instance created.

Methods

`getDomains`

Signature: `getDomains()`

Gets a list of valid domains in the configuration database.

Return value

A Promise with a list of Domain objects.

`saveConfiguration`

Signature: `saveConfiguration(string domain, string filePath)`

`domain` the domain name, e.g. 'EIO', 'SYS', 'MOC', etc.

`filePath` the path to the file, including file name

Saves the configurations for a domain to file.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

verifyConfigurationFile

Signature: `verifyConfigurationFile(string filePath, string action = 'add')`

`filePath` the path to the file, including file name

`action` the validation method, valid values: 'add', 'replace' and 'add-with-reset'.

Verifies a configuration file. The validation method must match the intended load method.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected. The promise is rejected both in case of an invalid and/or missing file.

loadConfiguration

Signature: `loadConfiguration(string filePath, string action = 'add')`

`filePath` the path to the file, including file name

`action` the validation method, valid values: 'add', 'replace' and 'add-with-reset'.

Loads a configuration file. The validation method indicates in which way duplicates should be handled.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

getTypes

Signature: `getTypes(string domain)`

`domain` the name of the domain

Gets all types on a domain.

Return value

A Promise with a list of Type objects.

getInstances

Signature: `getInstances(string domain, string type)`

`domain` the name of the domain

`type` the name of the type

Gets all instances on a type.

Return value

A Promise with a list of Instance objects.

getInstanceByName

Signature: `getInstanceByName(string domain, string type, string name)`

domain the name of the domain

type the name of the type

name the name of the instance

Gets a specific instance on a type. The instance is identified by its name.

NOTE! Not all instances have names, use `getInstanceById()` or `getInstances()` in that case.

Return value

A Promise with an Instance object.

getInstanceById

Signature: `getInstanceById(string domain, string type, string id)`

domain the name of the domain

type the name of the type

id the identifier of the instance

Gets a specific instance on a type. The instance is identified by the id created by RWS.

Return value

A Promise with an Instance object.

createInstance

Signature: `createInstance(string domain, string type, string name = '')`

domain the domain

type the type

name the name of the instance

Creates a new instance on the controller. The instance will be blank and setting it's attributes by calling `updateAttributesByName()` or `updateAttributesById()` is expected. If the Type does not contain a name attribute the name parameter can be omitted.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

updateAttributesByName

Signature: `updateAttributesByName(string domain, string type, string name, {} attributes)`

domain	the domain
type	the type
name	the name of the instance
attributes	a collection of key-value pairs

Updates the instance's attributes. The attributes Object should only contain valid members, i.e. attributes valid for the instance's specific Type, and the corresponding values.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

Example

```
var attr = {};  
attr['Label'] = 'Test IO Label';  
attr['SignalType'] = 'DI';  
attr['Access'] = 'All';  
await RWS.CFG.updateAttributesByName('EIO', 'EIO_SIGNAL', 'TestDI', attr);
```

updateAttributesById

Signature: updateAttributesById(string domain, string type, string id, {} attributes)

domain	the domain
type	the type
id	the identifier of the instance
attributes	a collection of key-value pairs

Updates the instance's attributes. The attributes Object should only contain valid members, i.e., attributes valid for the instance's specific Type, and the corresponding values.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

deleteInstanceByName

Signature: deleteInstanceByName(string domain, string type, string name)

domain	the domain
type	the type
name	the name of the instance

Deletes an instance on the controller.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

deleteInstanceById

Signature: `deleteInstanceById(string domain, string type, string id)`

`domain` the domain

`type` the type

`id` the identifier of the instance

Deletes an instance on the controller.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

4.6.1 Domain Object

Description

The Domain object represents a configuration domain in the configuration database, e.g. the I/O System domain, Motion domain, etc. Domain objects are used to access configuration information on the specific domain it was instantiated for. Domain objects cannot be explicitly created, they are retrieved with the `getDomains()` function in the `RWS.CFG` namespace.

Methods

getName

Signature: `getName()`

Gets the name of the domain.

Return value

A string containing the name of the domain.

getTypes

Signature: `getTypes()`

Gets a list of all types in the domain.

Return value

A Promise with a list of Type objects.

getInstances

Signature: `getInstances(string type)`

`type` the name of the type which instances to get

Gets all instances on a type.

Return value

A Promise with a list of Instance objects.

getInstanceByName

Signature: `getInstanceByName(string type, string name)`

`type` the name of the type

`name` the name of the instance

Gets a specific instance on a type. The instance is identified by its name.

NOTE! Not all instances have names, use `getInstanceById` or `getInstances` in that case.

Return value

A Promise with an Instance object.

getInstanceById

Signature: `getInstanceById(string type, string id)`

`type` the name of the type

`id` the identifier of the instance

Gets a specific instance on a type. The instance is identified by the id created by RWS.

Return value

A Promise with an Instance object.

createInstance

Signature: `createInstance(string type, string name = '')`

`type` the type

`name` the name of the instance

Creates a new instance on the controller. The instance will be blank and setting it's attributes by calling `updateAttributesByName` or `updateAttributesById` is expected. If the Type does not contain a name attribute the name parameter can be omitted.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

updateAttributesByName

Signature: `updateAttributeByName(string type, string name, {} attributes)`

<code>type</code>	the type
<code>name</code>	the name of the instance
<code>attributes</code>	a collection of key-value pairs

Updates the instance's attributes. The attributes Object should only contain valid members, i.e. attributes valid for the instance's specific Type, and the corresponding values. The function returns a Promise with the updated Instance object.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

updateAttributesById

Signature: `updateAttributeById(string type, string id, {} attributes)`

<code>type</code>	the type
<code>id</code>	the identifier of the instance
<code>attributes</code>	a collection of key-value pairs

Updates the instance's attributes. The attributes Object should only contain valid members (attributes for the specific instance Type) and the corresponding values. The function returns a Promise with the updated Instance object.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

deleteInstanceByName

Signature: `deleteInstanceByName(string type, string name)`

<code>type</code>	the type
<code>name</code>	the name of the instance

Deletes an instance on the controller.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

deleteInstanceById

Signature: `deleteInstanceById(string type, string id)`

<code>type</code>	the type
<code>id</code>	the identifier of the instance

Deletes an instance on the controller.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

saveToFile

Signature: `saveToFile(string filePath)`

`filePath` the path to the file, including file name

Saves the configuration of the domain to a file.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

4.6.2 Type Object

Description

The Type object represents a configuration type in the configuration database, e.g. Signal or System Input in the I/O System domain. Type objects cannot be explicitly created, they are retrieved with the `getTypes()` function in the `RWS.CFG` namespace or a Domain object.

Methods

getName

Signature: `getName()`

Gets the name of the type.

Return value

A string containing the name of the type.

getDomainName

Signature: `getDomainName()`

Gets the name of the type's parent domain.

Return value

A string containing the name of the domain.

getDomain

Signature: `getDomain()`

Gets the type's parent domain.

Return value

A Domain object.

getInstances

Signature: `getInstances()`

Gets all instances on the type.

Return value

A Promise with a list of Instance objects.

getInstanceByName

Signature: `getInstanceByName(string name)`

`name` the name of the instance

Gets a specific instance on the type. The instance is identified by its name.

NOTE! Not all instances have names, use `getInstanceById()` or `getInstances()` in that case.

Return value

A Promise with an Instance object.

getInstanceById

Signature: `getInstanceById(string id)`

`id` the identifier of the instance

Gets a specific instance on the type. The instance is identified by the id created by the configuration database.

Return value

A Promise with an Instance object.

createInstance

Signature: `createInstance(string name = '')`

`name` the name of the instance

Creates a new instance on the controller. The instance will be blank and setting it's attributes by calling `updateAttributesByName()` or `updateAttributesById()` is expected. If the Type does not contain a name attribute the name parameter can be omitted.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

updateAttributesByName

Signature: `updateAttributeByName(string name, {} attributes)`

`name` the name of the instance

`attributes` a collection of key-value pairs

Updates the instance's attributes. The attributes object should only contain valid members, i.e. attributes valid for the instance's specific Type, and the corresponding values. The function returns the updated Instance object.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

updateAttributesById

Signature: `updateAttributeById(string id, {} attributes)`

`id` the identifier of the instance

`attributes` a collection of key-value pairs

Updates the instance's attributes. The attributes object should only contain valid members, i.e. attributes valid for the instance's specific Type, and the corresponding values. The function returns the updated Instance object.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

deleteInstanceByName

Signature: `deleteInstanceByName(string name)`

`name` the name of the instance

Deletes an instance on the controller.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

deleteInstanceById

Signature: `deleteInstanceById(string id)`

`id` the id of the instance

Deletes an instance on the controller.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

4.6.3 Instance Object

Description

The Instance object represents a configuration instance in the configuration database, e.g. a signal instance, a task instance, etc. Instances belong to a configuration type, e.g. signal instances belong to the “Signal” type, which in this API are represented by the Type (see 4.6.2) object.

Methods

getInstanceId

Signature: `getInstanceId()`

Gets the instance identifier. The identifier is read-only and set by the configuration database when a new instance is created.

Return value

A string containing the identifier.

getInstanceName

Signature: `getInstanceName()`

Gets the name of the instance.

Return value

A string containing the name of the instance.

getTypeName

Signature: `getTypeName()`

Gets the name of the instance’s parent type.

Return value

A string containing the name of the type.

getType

Signature: `getType()`

Gets the instance’s parent Type object.

Return value

A Type object.

getAttributes

Signature: `getAttributes()`

Gets the instance's attributes.

Return value

An object containing the attributes as properties.

Example

updateAttributes

Signature: `updateAttributes({} attributes)`

`attributes` a collection of key-value pairs

Updates the instance's attributes. The attributes object should only contain valid members, i.e. attributes valid for the instance's specific Type, and the corresponding values. The function returns a Promise with the updated Instance object.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

Example

```
var attr = {};
attr['Label'] = 'Test IO Label';
attr['SignalType'] = 'DI';
attr['Access'] = 'All';
try{
    var x = await RWS.CFG.getInstanceByName('EIO', 'EIO_SIGNAL', 'TestDI');
    await x.updateAttributes(attr);
} catch(error) {
    console.log(`Failed to write settings.\n${error}`);
}
```

delete

Signature: `delete()`

Deletes the instance on the controller. After calling this method, the Instance object is no longer valid.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

4.7 RWS.FileSystem Namespace

Description

The `RWS.FileSystem` namespace contains methods to access and modify files and directories on the controller's filesystem.

The namespace contains the `Directory` and `File` objects. These are described closer in this section.

There are also some methods available directly in the `RWS.FileSystem` namespace.

Methods

`getDirectory`

Signature: `getDirectory(string directoryPath)`

`directoryPath` the complete path to the directory

Gets a `Directory` object for a path. The contents of the `Directory` will be retrieved when this method is called and can be accessed by calling the `getContents` method on the object.

NOTE! The contents are only refreshed when the `fetch` method is called. If the contents change on the file system after the `Directory` object was created, it will not be reflected in the results received by calling `getContents` until `fetch` has been called.

Return value

A Promise with a `Directory` object.

Example

```
const pathModules = '$HOME/modules';
var directory = await RWS.FileSystem.getDirectory(pathModules);
var contents = await directory.getContents();
console.log(`Contents of '${pathModules}'`);
console.log('Directories:');
for (let item of contents.directories) {
    console.log(`\t${item.name}`);
}
console.log('Files:');
for (let item of contents.files) {
    console.log(`\t${item.name}`);
}
```

`createDirectory`

Signature: `createDirectory(string directoryPath)`

`directoryPath` the complete path to the directory

Creates a new directory in the file system at the given path. A `Directory` object for the new directory will be returned.

NOTE! The entire path will be created if any directory in the path is missing.

Return value

A Promise with a `Directory` object.

getFile

Signature: `getFile(string filePath)`

`filePath` the complete path to the file

Gets a File object for a path. The contents of the file will be retrieved when this method is called.

Return value

A Promise with a File object.

createFileObject

Signature: `createFileObject(string filePath)`

`filePath` the complete path to the file

Creates a new File object for the path. The method does not create the file on the file system. It can be done by adding content to the object by calling `setContents` and then creating the file by calling the `save` method.

NOTE! This does not verify that the file exists, however that can be done by calling the `fileExists` method.

Return value

A Promise with a File object.

4.7.1 Directory Object

Description

The Directory object encapsulates a directory in the controller's filesystem. The object contains the contents of the directory in an object with two lists, as follows;

<code>([]) directories</code>	list of dictionary property objects
<code>([]) files</code>	list of file property objects

The contents of the lists are objects containing the properties of the corresponding directory or file. A directory's property object contains;

<code>(string) name</code>	the dictionary's name
<code>(Date) created</code>	the creation date
<code>(Date) modified</code>	the last modification date

A file's property object contains;

<code>(string) name</code>	the file's name
<code>(Date) created</code>	the creation date
<code>(Date) modified</code>	the last modification date
<code>(number) size</code>	the size of the file in bytes

(boolean) `isReadOnly` flag indicating if file is read only

Methods

`getPath`

Signature: `getPath()`

Gets the directory's path.

Return value

The path to the directory the Directory object is associated with.

`getProperties`

Signature: `getProperties()`

Gets the properties of the directory.

Return value

A Promise with an object containing the directory's properties defined as;

(string) <code>name</code>	the dictionary's name
(Date) <code>created</code>	the creation date
(Date) <code>modified</code>	the last modification date

`getContents`

Signature: `getContents()`

Gets the contents of the directory.

Return value

A Promise with an object containing lists for directories and files (see description 4.7.1).

`create`

Signature: `create(string newDirectory)`

`newDirectory` the name of the new directory

Creates a new directory in the file system in the Directory object's path.

Return value

A Promise with a Directory object for the new directory.

`delete`

Signature: `delete()`

Deletes the directory the object is associated with. The Directory object will be marked as deleted and subsequent method calls will be rejected. However, if any other entity creates the directory in the file system, calling the `fetch` method will reset the deleted status and the object will no longer reject methods.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

createFileObject

Signature: `createFileObject(string fileName)`

`fileName` the name of the file to create

Creates a new File object in the directory. The method does not create the file on the file system. It can be done by adding content to the object by calling `setContents` and then creating the file by calling the `save` method.

Return value

A File object.

rename

Signature: `rename(string newName)`

`newName` the new name of the directory

Renames the directory to the name provided.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

copy

Signature: `copy(string copyPath, boolean overwrite, boolean isRelativePath = true)`

`copyPath` the path of the copy

`overwrite` flag indicating whether to overwrite existing directory

`isRelativePath` flag indicating whether path is relative to original directory or absolute

Copies the directory to the path provided. If the `overwrite` flag is set to false and a directory or file with the name specified in `copyPath` exists the call will be rejected. If the flag `isRelativePath` is set to true the `copyPath` is expected to continue from the Directory objects path, otherwise the `copyPath` will be assumed to contain an absolute path.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

fetch

Signature: `fetch()`

Refreshes the contents of the Directory object.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

4.7.2 File Object

Description

The File object encapsulates a file in the controller's filesystem. The object can be used to retrieve the contents of the file.

Methods**getProperties**

Signature: `getProperties()`

Gets the properties of the directory.

Return value

A Promise with an object containing the file's properties defined as;

<code>(string) name</code>	the file's name
<code>(Date) created</code>	the creation date
<code>(Date) modified</code>	the last modification date
<code>(number) size</code>	the size of the file in bytes
<code>(boolean) isReadOnly</code>	flag indicating if file is read only

getContents

Signature: `getContents()`

Gets the contents of the file.

NOTE! After the contents have been retrieved, the content type can be determined by calling the method `getContentType`.

Return value

A Promise with an object containing the file's contents.

Example

```
const addTextToFile = async function(textToAdd){
  try{
    let myFile = await RWS.FileSystem.getFile("$HOME/logs/log.txt");
```

```
        let fileContent = await myFile.getContents();
        fileContent += `${textToAdd}\n`;
        myFile.setContents(fileContent);
        myFile.save(true);
    }
    catch(error) {
        console.log(`Failed to write log information.\n${error}`);
    }
}
```

getContentTypes

Signature: `getContentTypes()`

Gets the type of the contents. This is the content type string from the HTTP response.

Return value

A string with the content type, or empty string if not available.

setContents

Signature: `setContents(any newContents)`

`newContents` the new contents of the file

Sets the contents of the file object. After this has been called, the method `save` is used to store the content in the file on the file system.

Return value

A boolean with the status of the call.

fileExists

Signature: `fileExists()`

Checks if the file exists on the file system.

Return value

A Promise with a boolean with the result.

save

Signature: `save(boolean overwrite, boolean isBinary = false)`

`overwrite` flag indicating whether to overwrite existing directory

`isBinary` flag indicating whether binary or text mode should be used

Saves the contents to a file on the file system.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

delete

Signature: `delete()`

Deletes a file on the file system.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

rename

Signature: `rename(string newName)`

`newName` the new name of the directory

Renames a file on the file system to the name specified in `newName`.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

copy

Signature: `copy(string copyPath, boolean overwrite, boolean isRelativePath = true)`

`copyPath` the path of the copy

`overwrite` flag indicating whether to overwrite existing file

`isRelativePath` flag indicating whether path is relative to original file or absolute

Copies the file to the path provided. If the `overwrite` flag is set to true and a file with the name specified in `copyPath` exists that file will be overwritten. If the flag `isRelativePath` is set to true the `copyPath` is expected to continue from the File objects path, otherwise the `copyPath` will be assumed to contain an absolute path.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

Example

```
try {
    var file = RWS.FileSystem.createFileObject('$HOME/logs/log1.txt');
    await file.copy('backups/log1.txt', false, true);
} catch (error) {
    console.log(`Error when creating backup.\n${error}`);
}
```

Example

```
try {
    var file = RWS.FileSystem.createFileObject('$HOME/logs/log1.txt');
```

```

        await file.copy('$HOME/logs/backups/log1.txt, true, false);
    } catch (error) {
        console.log(`Error when creating backup.\n${error}`);
    }

```

fetch

Signature: `fetch()`

Refreshes the File object's content and content type.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

4.8 RWS.Elog Namespace

Description

The `RWS.Elog` namespace contains methods to read, handle and subscribe to controller events.

The namespace contains the following objects; Event and Domain. These are described closer in this section.

There are also a set of methods that are available directly in the `RWS.Elog` namespace.

Methods

clearElogAll

Signature: `clearElogAll()`

Clears all Elog domains on the controller.

NOTE! All events will be purged from the controller, and the operation cannot be undone.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

clearElog

Signature: `clearElog(number domainNumber)`

`domainNumber` the domain number

Clears a specified Elog domain on the controller.

NOTE! All events will be purged, and the operation cannot be undone.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

getBufferSize

Signature: `getBufferSize(number domainNumber)`

`domainNumber` the domain number

Gets the buffer size of a domain. The buffer size is the maximum number of events possible to store for the specified domain.

Return value

A Promise with a number.

getNumberOfEvents

Signature: `getNumberOfEvents(number domainNumber)`

`domainNumber` a domain number

Gets the actual number of events for a domain.

Return value

A Promise with a number.

getEvents

Signature: `getEvents(number domainNumber, string language = 'en')`

`domainNumber` the domain number

`language` a language specifier, valid values: 'en', 'de', 'es', 'sv', ...

Gets all events for a specified domain in a specified language.

To allow faster download, the Event objects' data must be retrieved by calling their `getContents()` method.

Return value

A Promise with a dictionary (JavaScript Object) of Event objects, where the key is the sequence number of the event.

Example

```
var commonEvents = await RWS.Elog.getEvents(0, 'de');
var events = [];
for (let item in commonEvents) {
    try{
        var e = await res[item].getContents();
        events.push(e);
    } catch(error){
        console.warn(`Item '${item}' failed to load.\n${error}`);
    }
}
```

getEventsPaged

Signature: `getEventsPaged(number domainNumber, string language = 'en', number count = 50, number page = 1)`

<code>domainNumber</code>	the domain number
<code>language</code>	a language specifier, valid values: 'en', 'de', 'es', 'sv', ...
<code>count</code>	the number of events per page
<code>page</code>	the page to get

Gets the specified number of events for a specified domain in a specified language. The events returned will start at the page number specified, where 1 is the first page.

To allow faster download, the Event objects' data must be retrieved by calling their `getContents()` method.

Return value

A Promise with a dictionary (JavaScript Object) of Event objects, where the key is the sequence number of the event.

getEvent

Signature: `getEvent(number sequenceNumber, string language = 'en')`

<code>sequenceNumber</code>	the sequence number of the event
<code>language</code>	a language specifier, valid values: 'en', 'de', 'es', 'sv', ...

Creates an Event object for a specified event, setting its preferred language but without retrieving its data.

Return value

An Event object.

getDomain

Signature: `getDomain(number domainNumber)`

<code>domainNumber</code>	the domain number
---------------------------	-------------------

Creates a Domain object for a specified domain.

Return value

A Domain object.

4.8.1 Event Object

Description

The Event object stores the information about an event in a language specified. The data is retrieved only when the `getContents()` method is called. As the information is static

subsequent calls to the `getContents()` method will not retrieve any data but return the information already held.

Methods

`getContents`

Signature: `getContents()`

Gets the data for an event. First time this method is called for the Event it will retrieve the data from the controller, subsequent calls returns the stored values.

Return value

A Promise with an object containing:

<code>(number) sequenceNumber</code>	events sequence number
<code>(string) eventType</code>	type of event, valid values: <code>'informational'</code> <code>'warning'</code> <code>'error'</code>
<code>(Date) timeStamp</code>	time of event
<code>(number) code</code>	event's code field
<code>(string) title</code>	event's title field
<code>(string) description</code>	event's description field
<code>(string) consequences</code>	event's consequences field
<code>(string) causes</code>	event's causes field
<code>(string) actions</code>	event's suggested actions field
<code>([object]) arguments</code>	event's optional information, each object contains <code>(string) type</code> <code>(string number) value</code>

`isValid`

Signature: `isValid()`

Verifies if the data in the Event is retrieved and complete. Return true if data is ready to use, otherwise false.

Return value

A boolean.

4.8.2 Domain Object

Description

The Domain object represents an Elog domain on the controller. It is used to access information and retrieve and subscribe to events.

Methods

getTitle

Signature: `getTitle()`

Used internally. Gets the title (part of a URL) of the subscribed resource.

Return value

A string containing the signal's title.

getDomainNumber

Signature: `getDomainNumber()`

Gets the domain number of the Domain object.

Return value

A number.

clearElog

Signature: `clearElog()`

Clears the domain on the controller.

NOTE! All events will be purged, and the operation cannot be undone.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

getBufferSize

Signature: `getBufferSize()`

Gets the buffer size of the domain. The buffer size is the maximum number of events possible to store for the specified domain.

Return value

A Promise with a number.

getNumberOfEvents

Signature: `getNumberOfEvents()`

Gets the actual number of events for the domain.

Return value

A Promise with a number.

getEvents

Signature: `getEvents(string language = 'en')`

`language` a language specifier, valid values: 'en', 'de', 'es', 'sv', ...

Gets all events for the domain in a specified language.

To allow faster download, the Event objects' data must be explicitly retrieved by calling their `getContents()` method.

Return value

A Promise with a dictionary (JavaScript object) of Event objects, where the key is the sequence number of the event.

getEventsPaged

Signature: `getEventsPaged(string language = 'en', number count = 50, number page = 1)`

`language` a language specifier, valid values: 'en', 'de', 'es', 'sv', ...

`count` the number of events per page

`page` the page to get

Gets the specified number of events for the domain in a specified language. The events returned will start at the page number specified, where 1 is the first page.

To allow faster download, the Event objects' data is not retrieved and must be retrieved by calling their `getContents()` method.

Return value

A Promise with a dictionary (JavaScript object) of Event objects, where the key is the sequence number of the event.

getResourceString

Signature: `getResourceString()`

Gets a unique resource URL for subscription.

Return value

A string containing a unique resource URL (as used by RWS.)

addCallbackOnChanged

Signature: `addCallbackOnChanged(function callback)`

`callback` a callback function to run when event occurs

Adds a callback function which will be called when the subscribed resource's state changes.

The callback can have one argument, which will contain the sequence number of the new event.

Return value

No return value.

onchanged

This is a function for internal use only. Do not use from external scripts.

subscribe

Signature: `subscribe()`

Starts the subscription to new events on the domain.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

Example

```
var myErrorCount = 0;
var elogDomain = null;

...

try{
    elogDomain = RWS.Elog.getDomain(RWS.Elog.DomainId.user);
    elogDomain.addCallbackOnChanged(async (newValue) => {
        let event = await RWS.Elog.getEvent(newValue);
        let contents = await event.getContents();
        if (contents.title === 'My Severe Error') {
            myErrorCount++;
        }
    });
    await elogDomain.subscribe();
} catch(error) {
    console.log(`Error occurred while subscribing to user elog.\n${error}`);
}
```

unsubscribe

Signature: `unsubscribe()`

Stops the subscription to the resource.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

4.9 RWS.UAS Namespace

Description

The `RWS.UAS` namespace, contains methods to read information in the user authentication system.

The namespace does not contain any objects, only a collection of methods.

Methods

`getUser`

Signature: `getUser()`

Gets basic information of the user currently logged in and using the app.

Return value

A Promise with an object containing:

<code>(string) alias</code>	the user's alias
<code>(string) name</code>	the user's name
<code>(string) locale</code>	the locale, possible values; 'internal', 'local' or 'remote'
<code>(string) application</code>	the name of the application that is executing this method
<code>(string) location</code>	the physical location of the user

`getGrants`

Signature: `getGrants()`

Gets a list of available grants on the system.

Return value

A Promise with a dictionary (JavaScript object) where the key is the grant's name ([reference below](#)) and the value is an object containing:

<code>(string) reference</code>	a name used for referring to this grant, e.g., when using the <i>hasGrant</i> method described below
<code>(string) name</code>	a reference to the grant's name element in the localized xml file (<code>uastext.xml</code> which is part of RobotWare and installed on the system)

(string) description a reference to the grant's description element in the localized xml file

hasGrant

Signature: `hasGrant(string grant)`

grant the grant to verify

Verifies if the user has a specific grant.

Return value

A Promise with a boolean indicating whether the user has the grant.

hasRole

Signature: `hasRole(string role)`

role the role to verify

Verifies if the user has a specific role.

Return value

A Promise with a boolean indicating whether the user has the role.

Example

Let's assume a role 'CanUseMyApp' exists on the controller and is required to run the app.

```
try {
  var hasRole = await RWS.UAS.hasRole('CanUseMyApp');
  if (hasRole === false) {
    var userInfo = await RWS.UAS.getUser();
    console.log(`User '${userInfo.name}' is not authorized to use app.`);
    myShutdownRoutine();
  }
} catch(error) {
  console.log(`Error while checking role, ${error}`);
  myShutdownRoutine();
}
```

4.10 RWS.ControlStation Namespace

Description

The way write access is handled by the OmniCore controller has changed from RobotWare 8 and forward, due to adherence to regulatory safety standards, i.e., ISO 10218-1:2025.

This domain replaces the Mastership and MotionMastership domains, and covers functionality needed by external, non-FlexPendant-based applications for acquiring write access and other related functionality.

Some functionality is usable from FlexPendant-based applications as well, such as reading or subscribing to control station-related information and states.

In general, for applications running on the FlexPendant, the operator must explicitly request and acquire write access using the common user interface of the FlexPendant itself.

For applications running externally, e.g., in a desktop web browser, the application is responsible for letting the user request and acquire write access using these APIs.

Automatic request/release of write access, as on RobotWare 7, is not supported directly by the App SDK when running on a RobotWare 8 (or later) system.

Calling any method in this domain on RobotWare 7 will raise an exception.

Methods

registerAsLocal

Signature: `registerAsLocal(string key)`

`key` A key used for proving local presence

Try to register the current session as part of a local control station.

This method should typically not be used by third-party applications, and the required key cannot be obtained using this SDK.

Applications, including third-party, running on the FlexPendant are automatically considered to be local clients, and need not register explicitly.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

registerAsRemote

Signature: `registerAsRemote(string name = <auto>, string id = <random>, string pincode = <random>, boolean autoReleaseWriteAccess = true)`

`name` Display name of the control station. If the default value `null` is used, a generic default name is used.

`id` Identifier for the control station. A 38-character string, e.g., {49fbd010-bbc9-4404-a11d-2c43317cc367}. If the default value `null` is used, a random identifier is used.

`pincode` A pincode used for joining an existing control station. An unsigned 32-bit number (0 - 4 294 967 295), represented as a string. If the default value `null` is used, a random pincode is used.

`autoReleaseWriteAccess` If `true`, the controller will release the write access held by this client when the last associated RWS session is lost. The default value is `true`.

Try to register the current session as part of a remote control station.

Registering is mandatory for external clients (not running on the FlexPendant) for write access to be granted on request.

More than one session/application can be part of a Control Station, sharing write access. Such cojoined applications should be physically close to ensure safe operations. To join an existing Control Station, use the same `id` and `pincode`.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

getId

Signature: `getId()`

Get the Control Station identifier associated with the current session.

Return value

A Promise with the identifier as a string, or the string `"none"` if the session is unregistered.

Contains a status if it is rejected.

requestWriteAccess

Signature: `requestWriteAccess()`

Try to acquire write access to the robot controller.

If a local control station (the FlexPendant) is present, "allow external control" must be enabled prior to making this request.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

releaseWriteAccess

Signature: `releaseWriteAccess()`

Release write access previously granted by the robot controller.

If a local control station (the FlexPendant) is present, "allow external control" must be enabled prior to making this request.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

disableExternalControl

Signature: `disableExternalControl()`

Disable possibility for remote control stations to acquire write access.

Can only be used by a local control station.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

allowMotionControl

Signature: `allowMotionControl(boolean allow = true)`

`allow` Boolean indicating whether to allow or disallow, default is `true`

Allow (or disallow) motion control for the calling control station. This is a safety measure that should not be automatically done by the application, only on explicit operator interaction.

After this method has been called, the controller will assume that the operator is aware that motion can be controlled by the control station.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

isMotionControlAllowed

Signature: `isMotionControlAllowed()`

Check whether motion control is allowed or not.

Return value

A Promise with a boolean indicating whether motion control is allowed or not. Contains a status if it is rejected.

getType

Signature: `getType()`

Get the current type of registered control station.

Return value

A Promise with one of the following strings

<code>none</code>	Not registered
<code>local</code>	Local control station
<code>remote</code>	Remote control station

Contains a status if it is rejected.

isLocalConnected

Signature: `isLocalConnected()`

Check whether a local control station, typically a FlexPendant, is connected to the controller.

Return value

A Promise with a boolean indicating whether a local control station is connected or not. Contains a status if it is rejected.

writeAccessReleaseAppeal

Signature: `writeAccessReleaseAppeal()`

Appeal to the current holder of write access to release the write access.

This is a fire-and-forget method that will not provide a response on whether the write access was released. There is no guarantee when, or even if, the write access will be released, that is up to the holding application and operator.

Use the monitor class of this namespace to subscribe to write access status to know when/if the write access is released.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

getWriteAccessStatus

Signature: `getWriteAccessStatus()`

Get the current write access status.

Return value

A Promise with an object with the following structure

<code>(string) name</code>	A string with the name of the control station currently holding write access. Might contain a string like "none" when no control station has access – however, do not use this to decide whether write access is held or not, instead refer to the <code>status</code> property.
<code>(string) id</code>	A string with the ID of the control station currently holding write access. Might contain a string like "none" when no control station has access – however, do not use this to decide whether write access is held or not, instead refer to the <code>status</code> property.
<code>(boolean) status</code>	A boolean indicating whether any control station is holding write access.
<code>(boolean) externalControlEnabled</code>	A boolean indicating whether remote control stations are allowed to be granted write access.

Contains a status if it is rejected.

getWriteAccessReleaseAppealChangeCounter

Signature: `getWriteAccessReleaseAppealChangeCounter()`

Get the total number of times the system has received an appeal for write access to be released. See the `writeAccessReleaseAppeal` method description for more information.

Return value

A Promise with a number.

Contains a status if it is rejected.

`getTPUSafetyProtocolStatus`

Signature: `getTPUSafetyProcotolStatus()`

Get the status of attached safety devices.

Return value

A Promise with a boolean, true when connected.

Contains a status if it is rejected.

`getMonitor`

Signature: `getMonitor(string resource)`

`resource` the resource to subscribe

Create a new Monitor (see section 4.10.1) for subscription on the provided resource. Valid resources are:

```
'write-access-status'  
'release-appeal-counter'  
'tpu-safety-protocol-connected'  
'local-is-connected'
```

Return value

A Monitor object.

4.10.1 Monitor Object

Description

To create a ControlStation monitor object use the factory method `getMonitor()` in the `RWS.Mastership` namespace.

The monitor object can subscribe to changes in control station state.

Methods

addCallbackOnChanged

Signature: `addCallbackOnChanged(function callback)`

`callback` a callback function to run when event occurs

Add a callback function which will be called when the subscribed state changes.

The callback routine can have one in-parameter, which is the new value of the subscribed resource. The type of the parameter is the same as from the corresponding method for fetching that value, e.g., if `writeAccessStatus` is the subscribed resource, the value of this parameter has the same type as the value returned by the `getWriteAccessStatus` method.

Return value

No return value.

subscribe

Signature: `subscribe(boolean raiseInitial = false)`

`raiseInitial` flag indicating whether an initial event is raised when subscription is registered

Start the subscription.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

unsubscribe

Signature: `unsubscribe()`

Stop the subscription.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

4.11 RWS.Mastership Namespace

Description

IMPORTANT: From RobotWare 8, the Mastership domain is obsolete and methods will either have no effect or raise exceptions. The App SDK still supports the domain on RobotWare 7 systems.

The `RWS.Mastership` namespace contains methods for requesting and releasing controller mastership (write access).

Normally, other parts of the RWS client library handle mastership automatically on a per-request basis. However, in some situations, there might be need for explicitly taking mastership, performing a series of operations, and releasing mastership again.

The mastership namespace uses a counter mechanism for keeping track of the number of requests that has been made from different parts of the code. As long as there are more requests than releases, mastership is kept. This means that it is important to release mastership when done with a sequence of operations.

NOTE: The methods below only handles **edit** mastership, which means that they cannot be used for motion related mastership. For motion mastership, see the `RWS.MotionMastership` namespace.

Methods

`request`

Signature: `request()`

Request **edit** mastership (write access) over the controller.

If mastership is already taken, the number of times the `release` method must be called to release mastership is increased by one.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

`release`

Signature: `release()`

Release **edit** mastership (write access) over the controller.

If mastership has been requested several times, an equal number of release calls must be performed.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

`getCurrent`

Signature: `getCurrent()`

Get an object representing the current mastership state on the controller.

The object has the following attributes:

<code>(string) alias</code>	Alias of session currently holding mastership. Undefined if mastership is not held.
-----------------------------	--

<code>(string) application</code>	Application name of session currently holding mastership. Undefined if mastership is not held.
<code>(string) uid</code>	UAS id of session currently holding mastership. Undefined if mastership is not held.
<code>(string) location</code>	Location of session currently holding mastership. Undefined if mastership is not held.
<code>(string) type</code>	Current mastership type. Can be “nomaster” if mastership is not held, “local” or “remote”.
<code>(string) heldbyme</code>	A boolean, <i>true</i> if mastership is held by the current user’s session, <i>false</i> when mastership is not held or when held by another session.

Return value

A Promise which is empty if it resolves and contains a mastership info object if it is rejected.

4.11.1 Monitor Object

Description

To create a Mastership monitor object use the factory method `getMonitor()` in the `RWS.Mastership` namespace.

The monitor object can subscribe to changes in mastership state.

Methods

addCallbackOnChanged

Signature: `addCallbackOnChanged(function callback)`

`callback` a callback function to run when event occurs

Adds a callback function which will be called when mastership state changes.

The callback routine can have one in-parameter, which is the new value of the subscribed resource. The value is a mastership info object representing the new state. The object’s format is the same as the one returned by the `RWS.Mastership.getCurrent` method.

Return value

No return value.

subscribe

Signature: `subscribe(boolean raiseInitial = false)`

`raiseInitial` flag indicating whether an initial event is raised when subscription is registered

Starts the subscription.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

unsubscribe

Signature: `unsubscribe()`

Stops the subscription.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

4.12 RWS.MotionMastership Namespace

Description

IMPORTANT: From RobotWare 8, the Mastership domain is obsolete and methods will either have no effect or raise exceptions. The App SDK still supports the domain on RobotWare 7 systems.

The `RWS.MotionMastership` namespace contain methods for requesting and releasing controller motion mastership (write access).

Normally, other parts of the RWS client library handle mastership automatically on a per-request basis. However, in some situations, there might be need for explicitly taking mastership, performing a series of operations, and releasing mastership again.

The motion mastership namespace uses a counter mechanism for keeping track of the number of requests that has been made from different parts of the code. As long as there are more requests than releases, mastership is kept. This means that it is important to release masterhip when done with a sequence of operations.

The counter used for ***motion*** mastership is completely independent from the counter used for ***edit*** mastership.

NOTE: The methods below only handles ***motion*** mastership, which means that they cannot be used for non-motion related mastership. For ***edit*** mastership, see the `RWS.Mastership` namespace.

Methods**request**

Signature: `request()`

Request ***motion*** mastership (write access) over the controller.

If motion mastership is already taken, the number of times the `release` method must be called to release motion mastership is increased by one.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

release

Signature: `release()`

Release *motion* mastership (write access) over the controller.

If motion mastership has been requested several times, an equal number of release calls must be performed.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

getCurrent

Signature: `getCurrent()`

Get an object representing the current *motion* mastership state on the controller.

The object has the following attributes:

<code>(string) alias</code>	Alias of session currently holding motion mastership. Undefined if motion mastership is not held.
<code>(string) application</code>	Application name of session currently holding motion mastership. Undefined if motion mastership is not held.
<code>(string) uid</code>	UAS id of session currently holding motion mastership. Undefined if motion mastership is not held.
<code>(string) location</code>	Location of session currently holding motion mastership. Undefined if motion mastership is not held.
<code>(string) type</code>	Current motion mastership type. Can be "nomaster" if motion mastership is not held, "local" or "remote".
<code>(string) heldbyme</code>	A boolean, <i>true</i> if motion mastership is held by the current user's session, <i>false</i> when motion mastership is not held or when held by another session.

Return value

A Promise which is empty if it resolves and contains a motion mastership info object if it is rejected.

4.12.1 Monitor Object

Description

To create a motion mastership monitor object use the factory method `getMonitor()` in the `RWS.MotionMastership` namespace.

The monitor object can subscribe to changes in motion mastership state.

Methods

`addCallbackOnChanged`

Signature: `addCallbackOnChanged(function callback)`

`callback` a callback function to run when event occurs

Adds a callback function which will be called when motion mastership state changes.

The callback routine can have one in-parameter, which is the new value of the subscribed resource. The value is a motion mastership info object representing the new state. The object's format is the same as the one returned by the `RWS.MotionMastership.getCurrent` method.

Return value

No return value.

`subscribe`

Signature: `subscribe(boolean raiseInitial = false)`

`raiseInitial` flag indicating whether an initial event is raised when subscription is registered

Starts the subscription.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

`unsubscribe`

Signature: `unsubscribe()`

Stops the subscription.

Return value

A Promise which is empty if it resolves and contains a status if it is rejected.

5 Components Library

5.1 Introduction

General information

The Components Library provides reusable HTML/CSS/JavaScript graphical components that can be re-used in an application for a look-and-feel like the FlexPendant shell, and for consistency between different user applications.

The Components Library is not a framework, i.e. it consists of a JavaScript API that can be called from the application code, not the other way around, as is the case for a typical framework. This means that it can be combined with or integrated with many other HTML/JavaScript frameworks or tools.

The application developer still needs to take care of the general layout, navigation and logic of the application – using basic common web technologies or by using a framework fit for the purpose.

Usage of the Components Library is entirely optional.

Installation

The Components Library files should be placed in a subdirectory under the application web root (where the `appinfo.xml` file is located) named `fp-components`.

Working with the components

Most components (but not all) works in a similar way. First, a handle object is created using JavaScript. This handle is used for manipulating the component, both at the time of its creation, and during the lifetime of the application.

The handle object typically has an `attachToId` and an `attachToElement` function, which is used to insert the component as a sub-tree of elements under an element, preferably an empty DIV, specified by the user using an element id or a direct reference to the element.

As a basic example, here is very small but complete application using a single button.

```
<!DOCTYPE html>
<html>
<head>
  <!-- Always use UTF-8 as character set! -->
  <meta charset="UTF-8">

  <!-- Import the common resources, this should always be
  done when using the Component library, and always
  before loading any other Component library resources. -->
  <script src="fp-components/fp-components-common.js"></script>

  <!-- Import the Button component resource. -->
  <script src="fp-components/fp-components-button-a.js"></script>
```

```
<script>

    var myButton;

    // Always wait until all resources are loaded
    // before using the library.
    window.addEventListener("load", function() {

        // Create a new Button component handle.
        myButton = new FPComponents.Button_A();
        // Set the text of the button.
        myButton.text = "Click me!";
        myButton.onclick = ()=>{
            // Add code here that should
            // run when the user clicks
            // on the button..
            // Here, we update the text
            // on the button.
            myButton.text = "Clicked!";
        };
        // Insert the button as elements under
        // the "myDiv" DIV element defined below.
        myButton.attachToId("myDiv");
    });
</script>
</head>

<body>
    <!-- This DIV element will be used as the root for
    the button. -->
    <div id="myDiv"></div>
</body>
</html>
```

In general, always use an empty DIV element as the base for an component.

Depending on the component, in some cases it makes sense to set the styles of this DIV itself, especially the `width` CSS attribute.

5.2 Common Resources

Common resources

All components described in the sections below requires functionality located in a common file. This file should always be imported before any other files from the Components Library:

```
<script src="fp-components/fp-components-common.js"></script>
```

5.3 Standard Colors

Using the standard colors

The component library includes a set of predefined colors. According to design guidelines (see separate document), since a limited palette is generally desirable, try to use these colors to the largest possible extent.

The colors are defined as CSS variables and are available after loading the common resources:

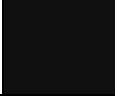
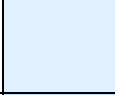
```
<script src="fp-components/fp-components-common.js"></script>
```

To use a color, use the `var` syntax in CSS. An example for setting the background color in a CSS rule:

```
background-color: var(--fp-components-GRAY-10);
```

List of available standard colors

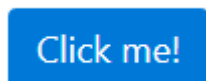
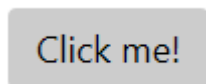
<code>--fp-color-STD-BACKGROUND</code>		White is used as the standard background color.
<code>--fp-color-PRIMARY-TEXT</code>		Use primary text color for all standard text, as well as headers.
<code>--fp-color-SECONDARY-TEXT</code>		The secondary text color can be used on a label when next to a data field.
<code>--fp-color-GRAY-10</code>		
<code>--fp-color-GRAY-20</code>		
<code>--fp-color-GRAY-30</code>		
<code>--fp-color-GRAY-40</code>		
<code>--fp-color-GRAY-50</code>		
<code>--fp-color-GRAY-60</code>		
<code>--fp-color-GRAY-70</code>		
<code>--fp-color-GRAY-80</code>		

--fp-color-GRAY-90		
--fp-color-GRAY-100		
--fp-color-BLUE-PRIMARY-HIGHLIGHT		The primary highlight color is used for highlighting. Things to highlight can be components and other elements. Same as --fp-color-BLUE-60
--fp-color-BLUE-10		
--fp-color-BLUE-20		
--fp-color-BLUE-30		
--fp-color-BLUE-40		
--fp-color-BLUE-50		
--fp-color-BLUE-60		
--fp-color-BLUE-70		
--fp-color-BLUE-80		
--fp-color-BLUE-90		
--fp-color-BLUE-100		
--fp-color-GREEN-OK		Status color: OK.
--fp-color-YELLOW-WARNING		Status color: Warning.
--fp-color-RED-DANGER		Status color: Danger / Error.

5.4 Button

Description

The Button component implements a simple clickable button. It is available in two variants – one basic (gray) and one highlighted (blue).



Requires

```
<script src="fp-components/fp-components-common.js"></script>
<script src="fp-components/fp-components-button-a.js"></script>
```

Example

```
var myButton = new FPComponents.Button_A();
myButton.text = "Click me!";
myButton.onclick = function () {
    // This function will execute when
    // the button is clicked.
};
myButton.attachToId("myDiv");
```

Handle constructor

```
var myButton = new FPComponents.Button_A();
```

Creates a handle to a new button.

Handle attributes

onclick

Type: `function ()`

Set this attribute to a callback function that should be called when the button is clicked.

enabled

Type: `boolean`

Setting this attribute to `false` will gray out the button and make it ignore any clicks. Setting this attribute to `true` (default) will enable the button, making it clickable.

text

Type: `string`

Set this attribute to the text that should be displayed on the button.

highlight

Type: `boolean`

Setting this attribute to `true` will make the button highlighted, i.e. *blue*. This is useful for marking an important button, having it stand out amongst other buttons.

Setting this attribute to `false` (default) will make the button use a normal appearance, i.e. *gray*.

icon

Type: `string`

Set this attribute to a string representing a path to an image to be used in order to display an icon to the left of the button text. If set to null (default) or empty string, the icon will not be displayed.

parent

Type: `Element`

After attaching the component to an element, this attribute will contain the parent element that the component has been attached to.

This attribute is read-only.

Methods**attachToId**

Signature: `attachToId ( string elementId )`

Creates the visual component as DOM tree elements under a parent element, specifying the parent element using an element ID (*elementId*).

Further updates to the component handle will affect this visual component.

IMPORTANT: Always use an empty div element. Do not attach the component to an element with existing children.

attachToElement

Signature: `attachToElement ( Element element )`

Same as `attachToId`, but takes an `Element` object directly as argument instead of an element ID.

5.5 Checkbox

Description

The Checkbox component implements a selectable checkbox.



Requires

```
<script src="fp-components/fp-components-common.js"></script>
<script src="fp-components/fp-components-checkbox-a.js"></script>
```

Example

```
var myCheckbox = new FPComponents.Checkbox_A();
myCheckbox.onclick = function (checked) {
    // This function will execute when
    // the checkbox is clicked.
    // The 'checked' argument contains a boolean
    // indicating whether the checkbox has been
    // checked (true) or unchecked (false).
};
myCheckbox.attachToId("myDiv");
```

Handle constructor

```
var myCheckbox = new FPComponents.Checkbox_A();
```

Creates a handle to a new checkbox.

Handle attributes

onclick

Type: `function ( boolean checked )`

Set this attribute to a callback function that should be called when the checkbox is clicked. The function should accept an argument, *checked*, which will be set to `true` if the checkbox was checked or `false` if it was unchecked. See example above.

enabled

Type: `boolean`

Setting this attribute to `false` will gray out the checkbox and make it ignore any clicks. Setting this attribute to `true` (default) will enable the button, making it clickable.

checked

Type: `boolean`

Set this attribute to `true` to check the checkbox, or `false` to uncheck it.

Setting this attribute directly will not trigger a callback to the `onclick` function.

scale

Type: `number`

Set this attribute to change the visual scale of the component.

A value of `1.0` corresponds to the "normal" base size used by the FlexPendant shell. A value of `1.3` means 30% larger, `2.0` double the size, and so on.

desc

Type: `string`

Set this attribute to a string in order to display that string as a description label to the right of the component. Clicking the description label will act like clicking the actual checkbox. If set to null (default) or empty string, the label will not be displayed.

parent

Type: `Element`

After attaching the component to an element, this attribute will contain the parent element that the component has been attached to.

This attribute is read-only.

Methods**attachToId**

Signature: `attachToId ( string elementId )`

Creates the visual component as DOM tree elements under a parent element, specifying the parent element using an element ID (*elementId*).

Further updates to the component handle will affect this visual component.

IMPORTANT: Always use an empty div element. Do not attach the component to an element with existing children.

`attachToElement`

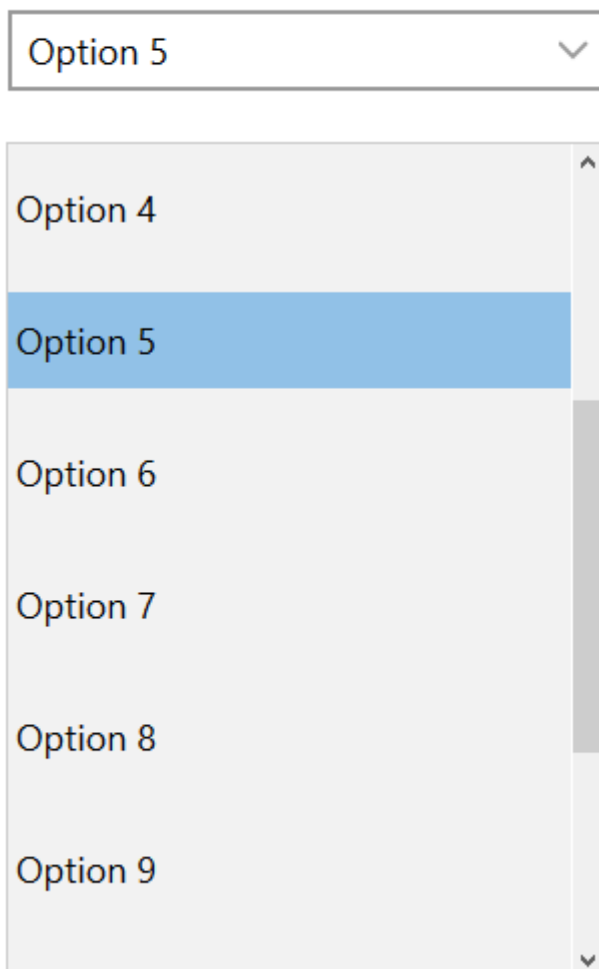
Signature: `attachToElement ( Element element )`

Same as `attachToId`, but takes an `Element` object directly as argument instead of an element ID.

5.6 Dropdown

Description

The Dropdown component implements a dropdown menu (also known as "combo-box"), allowing the user to select between available options.



Note: The scroll bar above is only visible on the virtual FlexPendant.

Requires

```
<script src="fp-components/fp-components-common.js"></script>
<script src="fp-components/fp-components-dropdown-a.js"></script>
```

Example

```
var myDropdown = new FPComponents.Dropdown_A();
myDropdown.model = {
    items: [
        "Option 1",
        "Option 2",
        "Option 3",
        "Option 4",
        "Option 5",
        "Option 6",
        "Option 7",
        "Option 8",
        "Option 9",
        "Option 10",
        "Option 11",
        "Option 12"
    ]
};
myDropdown.selected = 5;
myDropdown.onselection = function (index, obj) {
    // This function will execute when
    // an item in the dropdown menu is
    // selected by the user.
}
myDropdown.attachToId("myDiv");
```

Handle constructor

```
var myDropdown = new FPComponents.Dropdown_A();
```

Creates a handle to a new dropdown menu.

Handle attributes

onselection

Type: `function ( number index, object obj )`

Set this attribute to a callback function that should be called when the user selects an option in the dropdown menu. The function should accept two arguments, *index*, which will contain the selected index, and *obj*, which will contain the selected object from the model.

Indexing starts from 0.

See example above and the description for the `model` attribute.

enabled

Type: `boolean`

Setting this attribute to `false` will gray out the dropdown menu and make it ignore any clicks. Setting this attribute to `true` (default) will enable the dropdown menu, making it clickable.

leftTruncate

Type: `boolean`

If the text of the selected item does not fit within the component's width, the text will be clipped/truncated, and "..." will be added, either on the left side or the right side of the selected item's text. Setting this attribute to `false` (default) will truncate the selected item's text on the right side. Setting this attribute to `true` will truncate the selected item's text on the left side.

selected

Type: `number`

This attribute represents the currently selected index. It can be set to a new index to programmatically change selection. This will not cause the `onselection` callback to trigger.

It is possible to change selection while the menu part of the dropdown is shown. In this scenario, the selection will visibly change while the menu is kept open.

This attribute can have the value `null` when no selection is active.

model

Type: `object`

This attribute represents the available options in the dropdown menu.

The object should contain an attribute named `items`, which should contain a list of available options, where each option is represented by an object of any kind.

The string representation of each such object will be used when displaying the options within the menu. The option object will also be sent to the `onselection` callback function when the user has selected an option.

A typical option object could be a simple `string`, or a more complex custom object with a custom `toString` method.

Setting the model attribute will automatically refresh the dropdown menu options:

```
myDropdown.model = { ... some model ... }
```

... will cause a refresh.

However, simply modifying the current model:

```
myDropdown.model.items[0] = "New option title";
```

... will *not* cause a refresh, since the component will not notice that the model has changed. To force a refresh, set the model to itself after modifying it:

```
myDropdown.model.items[0] = "New option title";
```

```
myDropdown.model = myDropdown.model;
```

Example model object:

```
{
  items: [
    "My first option",
    "My second option"
  ]
}
```

desc

Type: `string`

Set this attribute to a string in order to display that string as a description label above the component. If set to null (default) or empty string, the label will not be displayed.

parent

Type: `Element`

After attaching the component to an element, this attribute will contain the parent element that the component has been attached to.

This attribute is read-only.

Methods

attachToId

Signature: `attachToId ( string elementId )`

Creates the visual component as DOM tree elements under a parent element, specifying the parent element using an element ID (*elementId*).

Further updates to the component handle will affect this visual component.

IMPORTANT: Always use an empty div element. Do not attach the component to an element with existing children.

attachToElement

Signature: `attachToElement ( Element element )`

Same as `attachToId`, but takes an `Element` object directly as argument instead of an element ID.

5.7 Input

Description

The Input component implements an input field with an attached on-screen keyboard.



Requires

```
<script src="fp-components/fp-components-common.js"></script>
<script src="fp-components/fp-components-input-a.js"></script>
```

Example

```
var myInput = new FPComponents.Input_A();
myInput.text = "Enter text here";
myInput.onchange = function ( text ) {
    // This function will execute when
    // the text has been modified by
    // the user.
}
myInput.attachToId("myDiv");
```

Handle constructor

```
var myInput = new FPComponents.Input_A();
```

Creates a handle to a new input field.

Handle attributes**onchange**

Type: `function ( string text )`

Set this attribute to a callback function that should be called when the input field is modified by the user. The function should accept an argument, `text`, which will be set to the new text content. See example above.

enabled

Type: `boolean`

Setting this attribute to `false` will gray out the input field and make it ignore any clicks. Setting this attribute to `true` (default) will enable the input field, making it possible for the user to click on it and change the value.

text

Type: `string`

This attribute represents the text content of the input field. It can be read to get the current content. Setting it will programmatically update the content and will not trigger the `onchange` callback function to be called.

label

Type: `string`

Descriptive label string that will be visible below the editor field on the keyboard when the input field is being edited. Should describe the value that the user is editing, preferably including any input limitations.

desc

Type: `string`

Set this attribute to a string in order to display that string as a description label above the component. If set to `null` (default) or empty string, the label will not be displayed.

regex

Type: Regular expression object

Standard JavaScript regular expression object used for validating and allowing the input.

Example:

```
myInput.regex = /^-?[0-9]+(\.[0-9]+)?$/;
```

will only allow input of floating-point numbers or integers.

Default value is `null`.

Can be used in combination with the `validator` argument.

Note that if the text of the input field is set programmatically using the `text` attribute, this input restriction does not apply.

validator

Type: `function ( string val )`

A callback function that will execute whenever the user is altering one or more characters in the keyboard's editable field.

The function signature should be:

```
function (val)
```

The `val` argument will be the current value (string) as it is being edited by the user. The callback function should return `true` when the value is acceptable and `false` when not acceptable.

Default value is `null`.

Can be used in combination with the `regex` argument.

Note that if the text of the input field is set programmatically using the `text` attribute, this input restriction does not apply.

variant

Type: `number`

Initial keyboard variant to be shown. Possible values are

`FP_COMPONENTS_KEYBOARD_ALPHA` or `FP_COMPONENTS_KEYBOARD_NUM`.

Default value is `null`.

NOTE! *This property is currently ignored and will not affect the appearance of the keyboard. This is due to limitations in built-in touch keyboard. It is possible that it will work in a future release.*

parent

Type: `Element`

After attaching the component to an element, this attribute will contain the parent element that the component has been attached to.

This attribute is read-only.

Methods

attachToId

Signature: `attachToId ( string elementId )`

Creates the visual component as DOM tree elements under a parent element, specifying the parent element using an element ID (*elementId*).

Further updates to the component handle will affect this visual component.

IMPORTANT: Always use an empty div element. Do not attach the component to an element with existing children.

attachToElement

Signature: `attachToElement ( Element element )`

Same as `attachToId`, but takes an `Element` object directly as argument instead of an element ID.

5.8 Radio Button

Description

The Radio button component implements a selectable radio button.



Requires

```
<script src="fp-components/fp-components-common.js"></script>
<script src="fp-components/fp-components-radio-a.js"></script>
```

Example

The following example shows two radio buttons, connected through callbacks.

```
var myRadioButton1 = null;
var myRadioButton2 = null;

myRadioButton1 = new FPComponents.Radio_A();
myRadioButton1.onclick = function () {
    // Uncheck the other radio button
    myRadioButton2.checked = false;
};
myRadioButton1.checked = true;
```

```
myRadioButton1.attachToId("myDiv1");

myRadioButton2 = new FPComponents.Radio_A();
myRadioButton2.onclick = function () {
    // Uncheck the other radio button
    myRadioButton1.checked = false;
};
myRadioButton2.attachToId("myDiv2");
```

Handle constructor

```
var myRadioButton = new FPComponents.Radio_A();
```

Creates a handle to a new radio button.

Handle attributes

onclick

Type: `function ( )`

Set this attribute to a callback function that should be called when the radio button is checked by the user. This callback function will not be called when the user clicks on a radio button that is already checked. See example above.

enabled

Type: `boolean`

Setting this attribute to `false` will gray out the radio button and make it ignore any clicks. Setting this attribute to `true` (default) will enable the radio button, making it clickable.

checked

Type: `boolean`

Set this attribute to `true` to check the radio button, or `false` to uncheck it.

Setting this attribute directly will not trigger a callback to the `onclick` function.

scale

Type: `number`

Set this attribute to change the visual scale of the component.

A value of `1.0` corresponds to the "normal" base size used by the FlexPendant shell. A value of `1.3` means 30% larger, `2.0` double the size, and so on.

desc

Type: `string`

Set this attribute to a string in order to display that string as a description label to the right of the component. Clicking the label will act like clicking the actual radio button. If set to null (default) or empty string, the label will not be displayed.

parent

Type: `Element`

After attaching the component to an element, this attribute will contain the parent element that the component has been attached to.

This attribute is read-only.

Methods**attachToId**

Signature: `attachToId ( string elementId )`

Creates the visual component as DOM tree elements under a parent element, specifying the parent element using an element ID (*elementId*).

Further updates to the component handle will affect this visual component.

IMPORTANT: Always use an empty div element. Do not attach the component to an element with existing children.

attachToElement

Signature: `attachToElement ( Element element )`

Same as `attachToId`, but takes an `Element` object directly as argument instead of an element ID.

5.9 Switch Button

Description

The Switch button component implements a selectable switch button.



Requires

```
<script src="fp-components/fp-components-common.js"></script>
```

```
<script src="fp-components/fp-components-switch-a.js"></script>
```

Example

```
var mySwitch = new FPComponents.Switch_A();
mySwitch.onchange = function (active) {
    // This function will execute when
    // the switch has been clicked by
    // the user.
}
mySwitch.attachToId("myDiv");
```

Handle constructor

```
var mySwitch = new FPComponents.Switch_A();
```

Creates a handle to a new switch button.

Handle attributes

onchange

Type: function (boolean active)

Set this attribute to a callback function that should be called when the switch button is modified by the user. The function should accept an argument, `active`, which will be set to `true` when the switch has been activated and `false` when it has been deactivated. See example above.

enabled

Type: boolean

Setting this attribute to `false` will gray out the switch button and make it ignore any clicks. Setting this attribute to `true` (default) will enable the switch button, making it clickable.

active

Type: boolean

Set this attribute to `true` to make the switch button activated, or `false` to deactivate it. Setting this attribute directly will not trigger a callback to the `onchange` function.

scale

Type: `number`

Set this attribute to change the visual scale of the component.

A value of `1.0` corresponds to the "normal" base size used by the FlexPendant shell. A value of `1.3` means 30% larger, `2.0` double the size, and so on.

desc

Type: `string`

Set this attribute to a string in order to display that string as a description label to the right of the component. Clicking the label will act like clicking the actual switch button. If set to null (default) or empty string, the label will not be displayed.

parent

Type: `Element`

After attaching the component to an element, this attribute will contain the parent element that the component has been attached to.

This attribute is read-only.

Methods**attachToId**

Signature: `attachToId ( string elementId )`

Creates the visual component as DOM tree elements under a parent element, specifying the parent element using an element ID (*elementId*).

Further updates to the component handle will affect this visual component.

IMPORTANT: Always use an empty div element. Do not attach the component to an element with existing children.

attachToElement

Signature: `attachToElement ( Element element )`

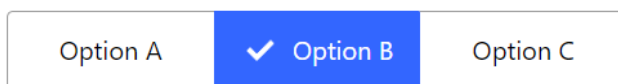
Same as `attachToId`, but takes an `Element` object directly as argument instead of an element ID.

5.10 Toggle

Description

The toggle component works as a group of buttons that allows the user to select only one or multiple values, depending on configuration of toggle component.

Each item in the toggle component can be configured separately to have different text, and different icons for both toggled and untoggled states.



Requires

```
<script src="fp-components/fp-components-common.js"></script>
<script src="fp-components/fp-components-toggle-a.js"></script>
```

Example

The following example creates a toggle component with three items. The first item has no icons, the second item has one icon for untoggled and a different one for toggled state. The last item has the same icon for both untoggled and toggled states.

```
var myToggle = new FPComponents.Toggle_A();
myToggle.model = [
  { text: "Option A" },
  { text: "Option A", icon: "option.png", activeIcon: "check.png" },
  { text: "Option A", icon: "option.png" }
]
myToggle.attachToId("myDiv");
```

Handle constructor

```
var myToggle = new FPComponents.Toggle_A();
```

Creates a handle to a new toggle component.

Handle attributes

multi

Type: `boolean`

Set this attribute to `true` to allow selection of multiple options in toggle. When set to `true` will ignore `singleAllowNone` setting, since all toggles will be toggled independently from each other.

Set this attribute to `false` (default) to allow selection of only one option at a time.

singleAllowNone

Type: `boolean`

Set this attribute to `true` to allow removing all selections and leaving toggle component without selected option.

Set this attribute to `false` (default) to force at least one option to be selected. Only works if **multi** is set to `false` as well.

onclick

Type: `function ( object state )`

Set this attribute to a callback function that should be called when any button in the component is toggled. This callback function will not be called when the user clicks on a button that does not change state, for example when clicking already toggled button and `singleAllowNone` is `true`. The function will return a state object containing which buttons changed and what they changed to, as well as current state of all buttons.

Example of state object:

```
{
  all: [false, true, false],
  changed: [ [1, true], [2, false] ]
}
```

the **all** property represents all toggle buttons and their current state, so in the example above the first button is untoggled, second is toggled and third is untoggled.

The **changed** property represents what changed when the user clicked a button, it's an array containing one array per changed button. Each button array with index of button changed, and what state it changed to. In the example above it says that button with index 1 changed to `true`, and button with index 2 changed to `false`, which can be verified in the `all` array as well.

model

Type: `object`

This attribute represents the buttons in the toggle component.

Contains a list of objects where each object represents a button.

The following attributes can be used on each element object:

Attribute	Description
<code>text</code>	Mandatory attribute. Text displayed on button
<code>icon</code>	Optional attribute. The path (relative to the web root, i.e. where the WebApp manifest is located) to an image to be displayed on the button. The optimal size for the icon image is 24x24 pixels, but it will be automatically resized.
<code>toggledIcon</code>	Optional attribute. Same requirements as icon attribute. When set will display the icon only when button is in toggled state. Will override icon attribute when button is toggled if both are set.

Methods

`attachToId`

Signature: `attachToId ( string elementId )`

Creates the visual component as DOM tree elements under a parent element, specifying the parent element using an element ID (*elementId*).

Further updates to the component handle will affect this visual component.

IMPORTANT: Always use an empty div element. Do not attach the component to an element with existing children.

`attachToElement`

Signature: `attachToElement ( Element element )`

Same as `attachToId`, but takes an `Element` object directly as argument instead of an element ID.

`setToggled`

Signature: `setToggled ( number index, boolean toggled, boolean noLimitations=false )`

Will set button with specified index to the provided toggled state. This operation will not cause onclick callbacks to be fired when used.

By default, this method will follow the limitations on toggled state (`multi`, `singleAllowNone`). Setting the optional `noLimitations` argument to `true` will force the toggle regardless of usage limitations.

getToggledList

Signature: `getToggledList ( )`

Gets array containing state of all buttons in order.

isToggled

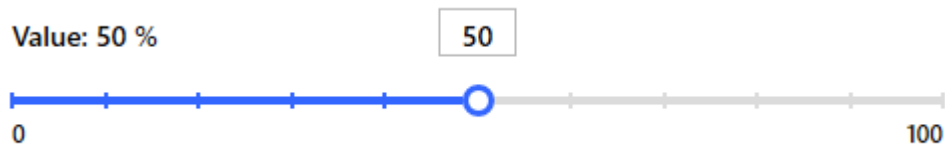
Signature: `isToggled ( number index )`

Returns true if button with provided index is toggled and returns false otherwise. if index is not valid (out of range, or not a number) will return undefined.

5.11 Slider

Description

The Slider component implements an analog slider that displays and lets the user set a value. It is highly configurable and has support for labels and tick marks.



Requires

```
<script src="fp-components/fp-components-common.js"></script>
<script src="fp-components/fp-components-slider-a.js"></script>
```

Example

```
var slider = new FPComponents.Slider_A();
slider.tickStep = 10;
slider.displayTicks = true;
slider.min = 0;
slider.max = 100;
slider.value = 20;
slider.enabled = true;
slider.label = "Value: ";
```

```
slider.unit = " %";  
slider.width = 200;  
slider.attachToId("slider-container-2");  
slider.ondrag = function(value) {  
    // Do something  
}  
slider.onrelease = function(value) {  
    // Do something  
}  
slider.attachToId("slider-placeholder");
```

Handle constructor

```
var mySlider = new FPComponents.Slider_A();
```

Creates a handle to a new Slider.

Handle attributes

max

Type: `number`

Set this attribute to the upper bound of the slider.

Default value is 100.

min

Type: `number`

Set this attribute to the lower bound of the slider.

Default value is 0.

label

Type: `string`

This attribute represents the base content of the built-in label.

Default value is an empty string.

displayLabel

Type: `boolean`

his attribute decides if the built-in label should be displayed.

Default value is `true`.

displayValue

Type: `boolean`

Setting this attribute to `true` will display the current value in the built-in label.

Default value is `true`.

unit

Type: `string`

Set this attribute to the unit to be displayed after values.

This attribute is only used if the `displayValue` attribute is set to `true`.

Default value is an empty string.

value

Type: `number`

Set this attribute to change the current value of the slider.

Default value is `0`.

tickStep

Type: `number`

This attribute sets the frequency of the tick marks on the slider.

Default value is `1`.

displayTicks

Type: `bool`

Setting this attribute to `true` will display tick marks on the slider.

The frequency of the tick marks is set on the `tickStep` attribute.

Default value is `false`.

numberOfDecimals

Type: `number`

This attribute controls the number of decimals to displayed by the slider component.

Default value is `0`.

enabled

Type: `bool`

Setting this to `true` will enable the slider for input.

Setting this to `false` will disable the slider for input and indicate that it is disabled visually.

Default value is `true`.

ondrag

Type: `function ( number value )`

Set this attribute to a callback function that should be called when thumb is moved.

The function should accept an argument, value which is the current value of the slider.

onrelease

Type: `function ( number value )`

Set this attribute to a callback function that should be called when thumb is released.

The function should accept an argument, value which is the current value of the slider.

width

Type: `number`

Set this attribute to the desired width of the slider component, in pixels.

Note that this value is including space required for labels. This is handled automatically by the component.

Default value is 200.

parent

Type: `Element`

After attaching the component to an element, this attribute will contain the parent element that the component has been attached to.

This attribute is read-only.

Methods

attachToId

Signature: `attachToId ( string elementId )`

Creates the visual component as DOM tree elements under a parent element, specifying the parent element using an element ID (*elementId*).

Further updates to the component handle will affect this visual component.

IMPORTANT: Always use an empty div element. Do not attach the component to an element with existing children.

attachToElement

Signature: `attachToElement ( Element element )`

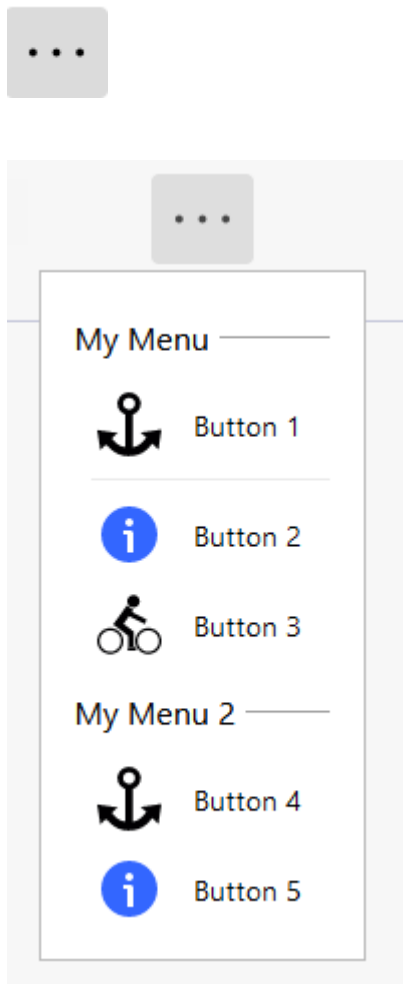
Same as `attachToId`, but takes an `Element` object directly as argument instead of an element ID.

5.12 Context Menu

Description

The Context menu component implements a context menu presenting the user with a configurable set of buttons.

The component takes the form of a single button that, when clicked, brings up the menu itself in close vicinity of the button.



Requires

```
<script src="fp-components/fp-components-common.js"></script>
<script src="fp-components/fp-components-contextmenu-a.js"></script>
```

Example

```
var myContextMenu = new FPComponents.Contextmenu_A();
myContextMenu.model = {
  content: [
    {
      type: "label",
      label: "My Menu"
    },
    {
      type: "button",
      label: "Button 1",
      icon: "anchor.svg",
      onclick: function () {
        // This function will execute when
        // the button is pressed by
        // the user.
      },
      enabled: true
    },
    {
      type: "gap"
    },
    {
      type: "button",
      label: "Button 2",
      icon: "info.svg",
      onclick: function () {
        // This function will execute when
        // the button is pressed by
        // the user.
      },
      enabled: true
    },
    {
      type: "button",
```

```
        label: "Button 3",
        icon: "cyclist.svg",
        onclick: function () {
            // This function will execute when
            // the button is pressed by
            // the user.
        },
        enabled: true
    },
    {
        type: "label",
        label: "My Menu 2"
    },
    {
        type: "button",
        label: "Button 4",
        icon: "anchor.svg",
        onclick: function () {
            // This function will execute when
            // the button is pressed by
            // the user.
        },
        enabled: true
    },
    {
        type: "button",
        label: "Button 5",
        icon: "info.svg",
        onclick: function () {
            // This function will execute when
            // the button is pressed by
            // the user.
        },
        enabled: true
    },
]
};
```

```
myContextMenu.attachToId("MyDiv");
```

Handle constructor

```
var myContextMenu = new FPComponents.Contextmenu_A();
```

Creates a handle to a new Context Menu.

Handle attributes

model

Type: object

This attribute represents the available options in the context menu.

The object should contain an attribute named `content`, which should contain a list of menu items, where each item is represented by an object.

Setting the model attribute will automatically refresh the context menu options:

```
myContextMenu.model = { ... some model ... }
```

... will cause a refresh.

However, simply modifying the current model:

```
myContextMenu.model.content[0] = {type: "gap"};
```

... will *not* cause a refresh, since the component will not notice that the model has changed. To force a refresh, set the model to itself after modifying it:

```
myContextMenu.model.content[0] = {type: "gap"};
```

```
myContextMenu.model = myContextMenu.model;
```

Example model object:

```
{
  content: [
    {
      type: "label",
      label: "My Menu"
    },
    {
      type: "button",
      label: "Button 1",
      icon: "someimage.png",
      onclick: function () {
        // This function will execute when
```

```

        // the button is pressed by
        // the user.
    },
    enabled: true
}
]
}

```

The available attributes for the objects are:

Attribute	Description
type	Mandatory attribute. Can be either "label", "button" or "gap".
label	Valid for types "label" and "button" only. The text that should be displayed on the item.
onclick	Valid for type "button" only. A callback function that should be called when the button is clicked.
icon	Valid for type "button" only. The path (relative to the web root, i.e. where the WebApp manifest is located) to an image to be displayed as an icon on the button. The optimal size for the icon image is 32x32 pixels, but it will be automatically resized. Can be omitted.
enabled	Valid for type "button" only. If set to <code>true</code> (default), this attribute has no effect. If set to <code>false</code> , the button will be disabled, i.e. it will be "grayed-out" and cannot be clicked. Default value is <code>true</code> .

enabled

Type: `boolean`

Setting this attribute to `false` will gray out the button that brings up the context menu and make it ignore any clicks. Setting this attribute to `true` (default) will enable the button, making it possible to click.

Methods

attachToId

Signature: `attachToId ( string elementId )`

Creates the visual component as DOM tree elements under a parent element, specifying the parent element using an element ID (*elementId*).

Further updates to the component handle will affect this visual component.

IMPORTANT: Always use an empty div element. Do not attach the component to an element with existing children.

attachToElement

Signature: `attachToElement ( Element element )`

Same as `attachToId`, but takes an `Element` object directly as argument instead of an element ID.

5.13 Digital Indicator

Description

The Digital indicator component implements an indicator that can show a digital state, typically for envisaging the state of a digital I/O signal.



Requires

```
<script src="fp-components/fp-components-common.js"></script>
<script src="fp-components/fp-components-digital-a.js"></script>
```

Example

```
var myDigital = new FPComponents.Digital_A();
myDigital.onclick = function () {
    // This function will execute when
    // the digital indicator has been clicked by
```

```
// the user.  
myDigital.active = !(myDigital.active);  
}  
myDigital.attachToId("myDiv");
```

Handle constructor

```
var myDigital = new FPComponents.Digital_A();
```

Creates a handle to a new Digital indicator.

Handle attributes

onclick

Type: `function ( )`

Set this attribute to a callback function that should be called when the digital indicator is clicked.

active

Type: `boolean`

Set this attribute to `true` to make the digital indicator activated (1), or `false` to deactivate it (0).

desc

Type: `string`

Set this attribute to a string in order to display that string as a description label to the right of the component. Clicking the label will act like clicking the actual digital indicator. If set to null (default) or empty string, the label will not be displayed.

parent

Type: `Element`

After attaching the component to an element, this attribute will contain the parent element that the component has been attached to.

This attribute is read-only.

Methods

attachToId

Signature: `attachToId ( string elementId )`

Creates the visual component as DOM tree elements under a parent element, specifying the parent element using an element ID (*elementId*).

Further updates to the component handle will affect this visual component.

IMPORTANT: Always use an empty div element. Do not attach the component to an element with existing children.

attachToElement

Signature: `attachToElement ( Element element )`

Same as `attachToId`, but takes an `Element` object directly as argument instead of an element ID.

5.14 Analog Level Meter

Description

The Analog level meter component implements a value meter that can show an analog state, typically for envisaging the state of an analog I/O signal or other non-digital value sources.



Requires

```
<script src="fp-components/fp-components-common.js"></script>
<script src="fp-components/fp-components-levelmeter-a.js"></script>
```

Example

```
var myLevelmeter = new FPComponents.Levelmeter_A();
myLevelmeter.min = 0;
myLevelmeter.max = 100;
myLevelmeter.lim1 = 50;
myLevelmeter.lim2 = 70;
myLevelmeter.unit = "°C";
myLevelmeter.width = 250;
myLevelmeter.value = 33;
myLevelmeter.attachToId("myDiv");
```

Handle constructor

```
var myLevelmeter = new FPComponents.Levelmeter_A();
```

Creates a handle to a new Analog level meter.

Handle attributes**width**

Type: `number`

Set this attribute to the desired component width in pixels.

Default is 200 pixels.

max

Type: `number`

Set this attribute to the upper bound of the meter.

Default is 100.

min

Type: `number`

Set this attribute to the lower bound of the meter.

Default is 0.

lim1

Type: `number`

Set this attribute to the value limit for the first (yellow) warning section.

Default is 100.

lim2

Type: `number`

Set this attribute to the value limit for the second (red) warning section.

Default is 100.

unit

Type: `string`

Set this attribute to the unit to be displayed after values.

Default is empty string.

value

Type: `number`

Set this attribute to the current value to be displayed.

Default is 0.

parent

Type: `Element`

After attaching the component to an element, this attribute will contain the parent element that the component has been attached to.

This attribute is read-only.

Methods

attachToId

Signature: `attachToId ( string elementId )`

Creates the visual component as DOM tree elements under a parent element, specifying the parent element using an element ID (*elementId*).

Further updates to the component handle will affect this visual component.

IMPORTANT: Always use an empty div element. Do not attach the component to an element with existing children.

attachToElement

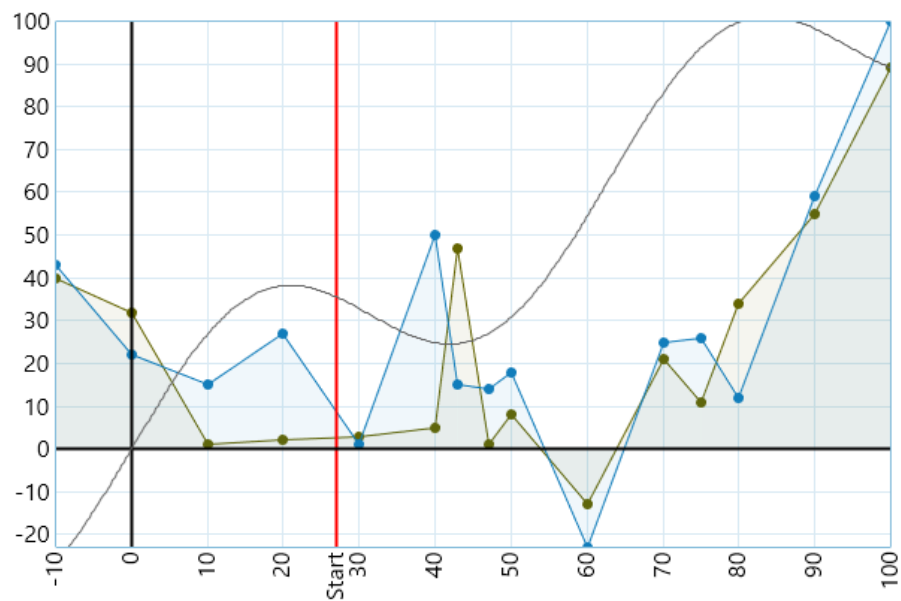
Signature: `attachToElement ( Element element )`

Same as `attachToId`, but takes an `Element` object directly as argument instead of an element ID.

5.15 Line Chart

Description

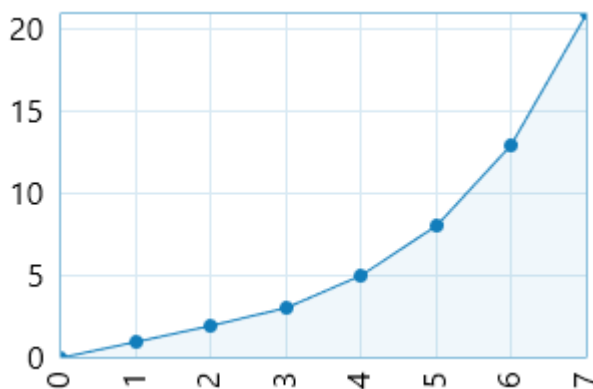
The Line chart component implements a configurable line chart with support for multiple 2-axis data series and mathematical functions.



Requires

```
<script src="fp-components/fp-components-common.js"></script>
<script src="fp-components/fp-components-linechart-a.js"></script>
```

Example



```
var myLineChart = new FPComponents.Linechart_A();
myLineChart.width = 300;
myLineChart.height = 400;

myLineChart.model = [
```

```

{
    points: [
        [0, 0],
        [1, 1],
        [2, 2],
        [3, 3],
        [4, 5],
        [5, 8],
        [6, 13],
        [7, 21]
    ]
}

];

myLineChart.attachToId("myDiv");

```

Handle constructor

```
var myLineChart = new FPCComponents.Linechart_A();
```

Creates a handle to a new line chart.

Handle attributes

width

Type: `number`

Set this attribute to the desired width of the line chart component, in pixels.

Note that this value is including space required for labels. This is handled automatically by the component.

Default value is 640.

height

Type: `number`

Set this attribute to the desired height of the line chart component, in pixels.

Note that this value is including space required for labels. This is handled automatically by the component.

Default value is 480.

xMax

Type: `number`

Set this attribute to the upper limit for x-axis range. A value of `null` will cause the component to automatically set the limit.

Default value is `null`.

xMin

Type: `number`

Set this attribute to the lower limit for the x-axis range. A value of `null` will cause the component to automatically set the limit.

Default value is `null`.

yMax

Type: `number`

Set this attribute to the upper limit for y-axis range. A value of `null` will cause the component to automatically set the limit.

Default value is `null`.

yMin

Type: `number`

Set this attribute to the lower limit for the y-axis range. A value of `null` will cause the component to automatically set the limit.

Default value is `null`.

xStep

Type: `number`

Set this attribute to the size of each background column. A value of `null` will cause the component to automatically set the size. A value of 0 or less will disable background columns.

Default value is `null`.

yStep

Type: `number`

Set this attribute to the size of each background row. A value of `null` will cause the component to automatically set the size. A value of 0 or less will disable background rows.

Default value is `null`.

xLabels

Type: `array`

This attribute holds custom labels positioned on the x-axis. The model is an array containing any number of labels. Each label is represented by another array, containing the label text and the x value.

Example:

```
myLineChart.xLabels = [  
    ["A", 1],  
    ["B", 4.5]  
];
```

yLabels

Type: `array`

This attribute holds custom labels positioned on the y-axis. The model is an array containing any number of labels. Each label is represented by another array, containing the label text and the y value.

Example:

```
myLineChart.yLabels = [  
    ["A", 1],  
    ["B", 4.5]  
];
```

xAutoLabelStep

Type: `number`

Set this attribute to the distance between each automatic label on the x-axis. Preferably, this value should be a multiple of the `xStep` attribute.

A value of `null` will cause this attribute to be automatic, i.e. same as the `xStep` attribute.

A value of 0 or less will disable automatic labels on the x axis.

Note that mixing automatic labels and custom labels (using the `xLabels` attribute) might cause layout collisions. In this case, custom labels take precedence.

Default value is `null`.

yAutoLabelStep

Type: `number`

Set this attribute to the distance between each automatic label on the y-axis. Preferably, this value should be a multiple of the `yStep` attribute.

A value of `null` will cause this attribute to be automatic, i.e. same as the `yStep` attribute.

A value of 0 or less will disable automatic labels on the y axis.

Note that mixing automatic labels and custom labels (using the `yLabels` attribute) might cause layout collisions. In this case, custom labels take precedence.

Default value is `null`.

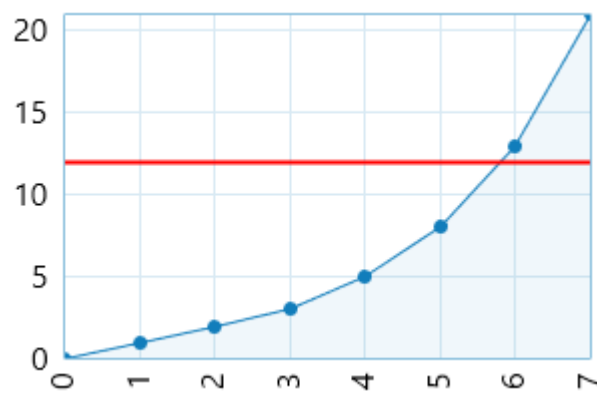
`model`

Type: `array`

This attribute represents the various items that will be displayed in the line chart, such as data sets and functions.

The model is an array which contains any number of objects. Each object contains various attributes which decide what should be displayed.

The following example contains two objects, a data set and a marker on the y axis:



```
myLineChart.model = [
  {
    // First object
    points: [
      [0,0],
      [1,1],
      [2,2],
      [3,3],
      [4,5],
      [5,8],
      [6,13],
      [7,21]
    ],
  },
  {
    // Second object
    yMarker: 12,
    thickness: 2,
    red: 255,
    green: 0,
  }
]
```

```

        blue: 0
    }
];

```

The available attributes for the objects are:

Attribute	Description
points	A data set with any number of points. Represented by an array containing single points, also arrays, where the first element is the x value and the second the y value. Example: <pre>points: [[102,673], // Point 1 [132,822], // Point 2 [155,223] // Point 3]</pre> The points will be automatically sorted by x value.
fill	A boolean indicating whether a data set graph should be filled (true) or simply a line (false). Only valid together with the points attribute. Default value is true.
dots	A boolean indicating whether the individual points in a data set graph should be indicated by a small dots (true) or not (false). Only valid together with the points attribute. Default value is true.
thickness	A number representing the thickness of the data set graph line, marker or function graph line. The unit is pixels. Default value is 1.
red, green, blue	If all three color attributes are present, use this RGB color value when drawing the object. Each attribute is a number from 0 to 255.
hidden	A boolean indicating whether this object should be displayed (false) or not (true).
xMarker	One or more x values that should be marked with a straight line. Can be either a number or an array with several numbers. Examples: <pre>xMarker : 17</pre> or

	<code>xMarker : [17, 28, 53]</code>
<code>yMarker</code>	One or more y values that should be marked with a straight line. Can be either a number or an array with several numbers.
<code>xFunc</code>	A function, $y = f(x)$. This function will be called repeatedly do draw a graph from <code>xMin</code> to <code>xMax</code> . The function should be of the signature <code>function (x)</code> and should return the y value for any given x value. Example: <pre>xFunc : function (x) { return Math.sin(x); }</pre>
<code>yFunc</code>	A function, $x = f(y)$. This function will be called repeatedly do draw a graph from <code>yMin</code> to <code>yMax</code> . The function should be of the signature <code>function (y)</code> and should return the x value for any given y value. Example: <pre>yFunc : function (y) { return Math.sin(y); }</pre>
<code>xFuncStep</code>	The resolution (x value step size) for resolving a <code>xFunc</code> function. Should be a positive number.
<code>yFuncStep</code>	The resolution (y value step size) for resolving a <code>yFunc</code> function. Should be a positive number.

Setting the model attribute will automatically refresh the line chart:

```
myLineChart.model = [ ... some model ... ]
```

... will cause a refresh.

However, simply modifying the current model:

```
myLineChart.model[2].hidden = true;
```

... will *not* cause a refresh, since the component will not notice that the model has changed. To force a refresh, set the model to itself after modifying it:

```
myLineChart.model[2].hidden = true;
```

```
myLineChart.model = myLineChart.model;
```

or call the `repaintLater()` method described below.

parent

Type: `Element`

After attaching the component to an element, this attribute will contain the parent element that the component has been attached to.

This attribute is read-only.

Methods

attachToId

Signature: `attachToId ( string elementId )`

Creates the visual component as DOM tree elements under a parent element, specifying the parent element using an element ID (*elementId*).

Further updates to the component handle will affect this visual component.

IMPORTANT: Always use an empty div element. Do not attach the component to an element with existing children.

attachToElement

Signature: `attachToElement ( Element element )`

Same as `attachToId`, but takes an `Element` object directly as argument instead of an element ID.

repaintLater

Signature: `repaintLater ( )`

Will repaint this component asynchronously at some point in the future.

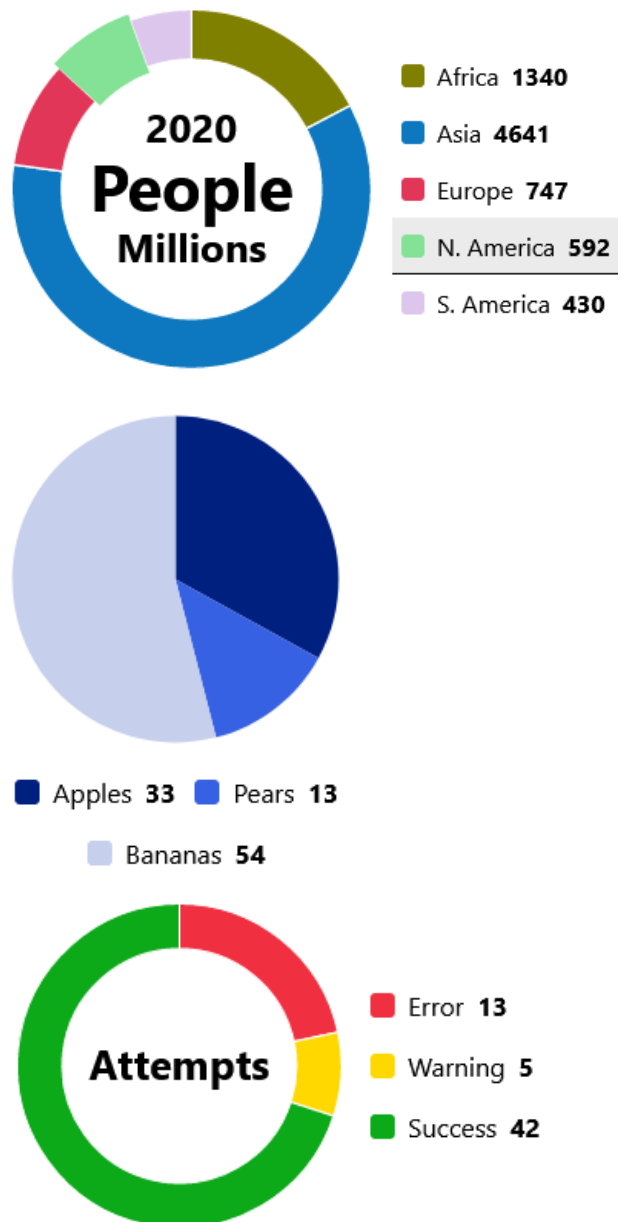
Most attribute changes will automatically cause this method to be called, but it can be used after changing the internals of complex attributes like `model` or `xLabels` without explicitly setting the attributes on the components handle.

Calling this method several times may result in a single update.

5.16 Pie Chart

Description

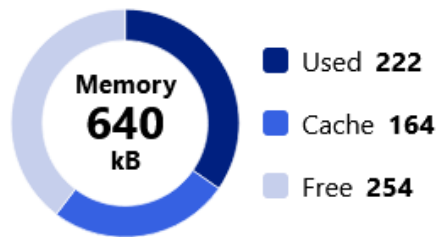
The Pie chart component implements a configurable pie- or donut chart with support for automatic color distribution and selectable sectors for easy reading of values.



Requires

```
<script src="fp-components/fp-components-common.js"></script>
<script src="fp-components/fp-components-piechart-a.js"></script>
```

Example



```
var myDonut = new FPComponents.Piechart_A();

myDonut.model = [
    [222, 'Used' ],
    [164, 'Cache' ],
    [254, 'Free' ],
];

myDonut.topText = "Memory";
myDonut.centerText = "640";
myDonut.bottomText = "kB";
myDonut.size = 150;

myDonut.attachToId("myDiv");
```

Handle constructor

```
var myPieChart = new FPComponents.Piechart_A();
```

Creates a handle to a new pie/donut chart.

Handle attributes

size

Type: `number`

Set this attribute to the desired width/height of the graphical part of the component, in pixels.

Note that this value is not including space required for labels. The space required by the labels are dependent on number of labels and their names and values.

Default value is 300.

donut

Type: `boolean`

Set this attribute to `true` for the graphical part to use the Donut chart visual style. Set it to `false` to use the Pie chart style instead.



Default value is `true`.

showLabels

Type: `boolean`

Set this attribute to `true` for the labels to be visible. Set it to `false` to hide them.

Default value is `true`.

labelsBelow

Type: `boolean`

Set this attribute to `true` for the labels to be positioned underneath the graphical chart. Set it to `false` for the labels to be positioned on the side.

Default value is `false`.

model

Type: `Array`

This attribute represents the data to be displayed in the chart.

The model is an array of arrays where each inner array represents a single value.

The format of each inner array is either

[`<number value>`, `<string description>`]

or

[`<number value>`, `<string description>`, `<string color>`]

If the color is omitted, it will instead be automatically decided by the component. See the `hue` and `multiHue` attributes for more related information.

Example:

```
myDonut.model = [
    [222, 'Used' ],
    [164, 'Cache' ],
    [254, 'Free' ],
];
```

or

```
myDonut.model = [
    [222, 'Used',  '#FF2222' ],
    [164, 'Cache', '#22FF22' ],
    [254, 'Free',  '#2222FF' ],
];
```

Setting the `model` attribute will automatically refresh the chart:

```
myDonut.model = [ ... some model ... ]
```

... will cause a refresh.

However, simply modifying the current model:

```
myDonut.model[2][0] = 150;
```

... will *not* cause a refresh, since the component will not notice that the model has changed. To force a refresh, set the model to itself after modifying it:

```
myDonut.model[2][0] = 150;
```

```
myDonut.model = myDonut.model;
```

Default value is an empty array.

hue

Type: `number`

Unless the sector colors are specified directly, this attribute can be set to a hue value within the HSV/HSL color space, i.e., a value between 0 and 360 – the angle on the color wheel.

The hue will be used for automatically picking sector colors when no color is specified in `model`.

Some example hues:

0	Red	
30	Orange	
60	Yellow	

90	Chartreuse	
120	Green	
150	Emerald	
180	Cyan	
210	Azure	
240	Blue	
270	Violet	
300	Magenta	
330	Crimson	
360	Red	

Default value is

225	Cobalt blue	
-----	-------------	--

multiHue

Type: `boolean`

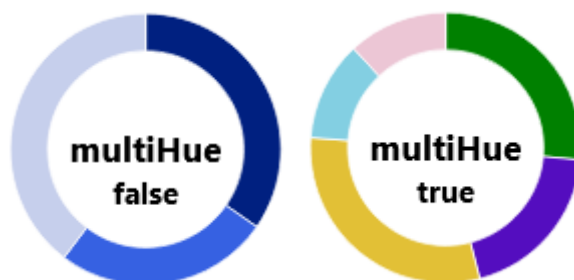
When no color is specified for sectors in `model`, the component will automatically select colors.

If this attribute is set to `false`, all auto-selected sector colors will have the same hue, specified by the `hue` attribute. They will differ only in saturation and lightness.

The single hue mode is works best when there are few sectors.

Setting the `multiHue` attribute to `true` enables the multi hue mode. In this mode, the sector hues will have different hues. The base hue (set using the `hue` attribute) will be used for the first sector, while other sectors will have other hues, as far away from each other as possible. Changing the `hue` attribute will affect all sectors.

The multi hue mode should typically be used when there are a larger number of sectors.



The default value is `false`.

centerText

Type: `string`

Set this attribute to a string that should be shown in the middle of the donut chart. This text is not visible in pie chart mode.

The text size is automatically adjusted to fit inside the donut.

Default value is an empty string.

topText

Type: `string`

Set this attribute to a string that should be above the center text of the donut chart. This text is not visible in pie chart mode.

The text size is automatically adjusted to fit inside the donut.

Default value is an empty string.

bottomText

Type: `string`

Set this attribute to a string that should be below the center text of the donut chart. This text is not visible in pie chart mode.

The text size is automatically adjusted to fit inside the donut.

Default value is an empty string.

selected

Type: `number`

This attribute represents the currently selected sector. The value is either `null` (for no selection) or the numeric index of a sector in the `model` array.

The initial value is `null`, but it will be changed when the user selects a sector.

onselection

Type: `function (number index, number value)`

Set this attribute to a function that should be called when the sector selection changes.

The function should accept two arguments, the index of the selected sector in the `model` array, and the numeric value of that sector.

Default value is `null`.

parent

Type: `Element`

After attaching the component to an element, this attribute will contain the parent element that the component has been attached to.

This attribute is read-only.

Methods

`appendData`

Signature: `appendData ( number value, string label, string color=null )`

Shorthand method for adding another sector to the chart. The sector will be added to the `model` array.

The arguments corresponds to the fields in each sector entry within the `model` array.

`clearData`

Signature: `clearData ( )`

This method is equivalent to setting the `model` attribute to an empty array, i.e., the chart is cleared from sectors.

`attachToId`

Signature: `attachToId ( string elementId )`

Creates the visual component as DOM tree elements under a parent element, specifying the parent element using an element ID (*elementId*).

Further updates to the component handle will affect this visual component.

IMPORTANT: Always use an empty `div` element. Do not attach the component to an element with existing children.

`attachToElement`

Signature: `attachToElement ( Element element )`

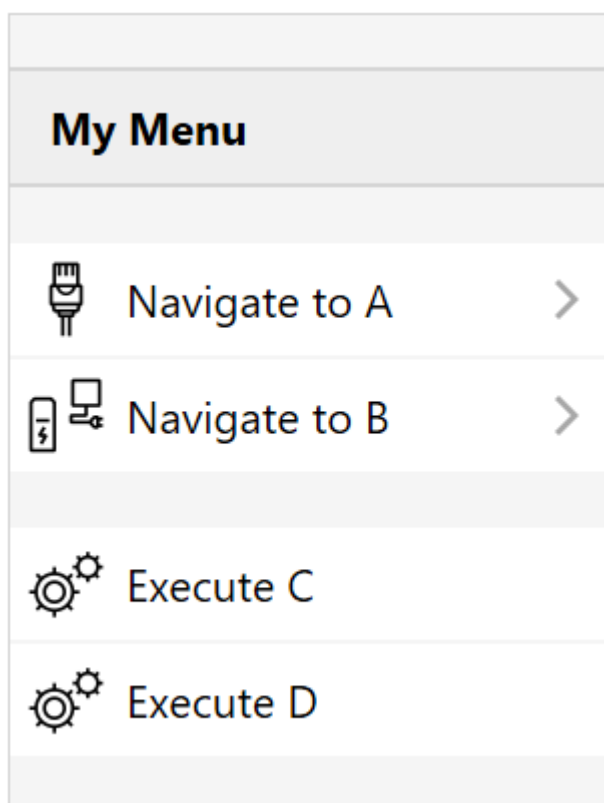
Same as `attachToId`, but takes an `Element` object directly as argument instead of an element ID.

5.17 Menu

Description

The `Menu` component implements a configurable menu, typically used for app navigation and main actions. It can be embedded in a `Fold-in` component (see below), some other custom component or simply put directly on the main view.

It has built-in support for buttons with optional icons, dividers and labels.



Requires

```
<script src="fp-components/fp-components-common.js"></script>
<script src="fp-components/fp-components-menu-a.js"></script>
```

Example

```
var myMenu = new FPComponents.Menu_A();
myMenu.model = {
  content: [
    {
      type: "gap"
    },
    {
      type: "label",
      label: "My Menu"
    },
    {
```

```
        type: "gap"
    },
    {
        type: "button",
        label: "Navigate to A",
        icon: "icons/A.png",
        arrow: true,
        onclick: function () {
            // Code to execute
        }
    },
    {
        type: "button",
        label: "Navigate to B",
        icon: "icons/B.png",
        arrow: true,
        onclick: function () {
            // Code to execute
        }
    },
    {
        type: "gap"
    },
    {
        type: "button",
        label: "Execute C",
        icon: "icons/C.png",
        arrow: false,
        onclick: function () {
            // Code to execute
        }
    },
    {
        type: "button",
        label: "Execute D",
        icon: "icons/D.png",
        arrow: false,
```

```
        onclick: function () {
            // Code to execute
        }
    },
    {
        type: "gap"
    }
]
};
myMenu.attachToId("myDiv");
```

Handle constructor

```
var myMenu = new FPComponents.Menu_A();
```

Creates a handle to a new menu.

Handle attributes

model

Type: object

This attribute represents the elements in the menu.

The object should contain an attribute named `content`, which should contain a list of elements, where each element is represented by an object.

```
myMenu.model =
{
    content: [
        { ... element 1 ... },
        { ... element 2 ... },
        { ... element 3 ... },
        { ... element 4 ... }
    ]
}
```

The following attributes can be used on each element object:

Attribute	Description
<code>type</code>	Mandatory attribute. Can be either <code>"label"</code> , <code>"gap"</code> , or <code>"button"</code> .
<code>onclick</code>	Valid for type <code>"button"</code> only. A callback function that should be called when the button is clicked.
<code>label</code>	Valid for type <code>"label"</code> and <code>"button"</code> only. The text that should be displayed on the element.
<code>icon</code>	Valid for type <code>"button"</code> only. The path (relative to the web root, i.e. where the WebApp manifest is located) to an image to be displayed on the button. The optimal size for the icon image is 40x40 pixels, but it will be automatically resized.
<code>arrow</code>	Valid for type <code>"button"</code> only. If set to <code>true</code> , an arrow will be displayed on the right side of the button. Use this to indicate that the clicking the button will cause UI navigation, rather than potentially affecting the system directly. If set to <code>false</code> (default), no arrow will be displayed.
<code>flash</code>	Valid for type <code>"button"</code> only. If set to <code>true</code> , the button will flash. Use this to draw attention when the user needs to be aware of something, e.g. an alarm or a finished task. If set to <code>false</code> (default), the button will not flash.
<code>flashColor</code>	Valid for type <code>"button"</code> only. Set to any valid CSS color string to change which background color the button will flash to if <code>flash</code> is set to <code>true</code> as well. See section 5.3 for standard colors to use. Default color is set to <code>--fp-color-YELLOW-WARNING</code>

enabled	Valid for type "button" only. If set to <code>true</code> (default), this attribute has no effect. If set to <code>false</code>, the button will be disabled, i.e. it will be "grayed-out" and cannot be clicked.
---------	--

Setting the model attribute will automatically refresh the menu:

```
myMenu.model = { ... some model ... }
```

... will cause a refresh.

However, simply modifying the current model:

```
myMenu.model.content[0].label = "New button label";
```

... will *not* cause a refresh, since the component will not notice that the model has changed. To force a refresh, set the model to itself after modifying it:

```
myMenu.model.content[0].label = "New button label";
```

```
myMenu.model = myMenu.model;
```

See example above.

parent

Type: `Element`

After attaching the component to an element, this attribute will contain the parent element that the component has been attached to.

This attribute is read-only.

Methods

attachToId

Signature: `attachToId ( string elementId )`

Creates the visual component as DOM tree elements under a parent element, specifying the parent element using an element ID (*elementId*).

Further updates to the component handle will affect this visual component.

IMPORTANT: Always use an empty div element. Do not attach the component to an element with existing children.

attachToElement

Signature: `attachToElement ( Element element )`

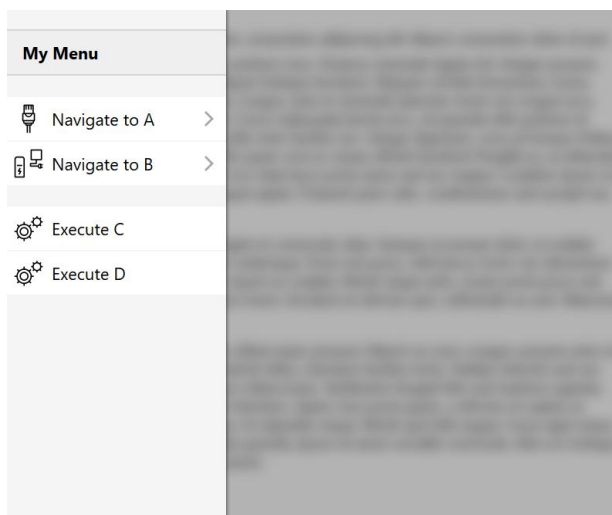
Same as `attachToId`, but takes an `Element` object directly as argument instead of an element ID.

5.18 Fold-in

Description

The Fold-in component implements an area which can be temporarily shown on the left side of the screen, on top of the rest of the UI.

It can be populated with any content. In the image below the Menu component example from section 5.16 is used.



Requires

```
<script src="fp-components/fp-components-common.js"></script>
<script src="fp-components/fp-components-foldin-a.js"></script>
```

Example

```
var myFoldin = new FPComponents.Foldin_A();
myFoldin.contentId = "foldinId";
myFoldin.width = 350;
myFoldin.attachToBody();

...

document.getElementById("foldinId").textContent = "My content";
myFoldin.show();
```

In this example, the content DOM element created by the component, is fetched by calling the standard JavaScript `document.getElementById` function, using the string identifier provided as `contentId`.

To fill the Fold-in component with more complex content, many approaches exist, such as

- Creating content in JavaScript and appending that content using the standard JavaScript `append` method, available on the element.
- Moving a part of the existing DOM tree into the content element using the same `append` method. This sub-tree could be from plain HTML.
- Using the `attachToId` method available on components from the Components Library. This is especially useful for creating a Menu component inside a Fold-in.

Handle constructor

```
var myFoldin = new FPComponents.Foldin_A();
```

Creates a handle to a new Fold-in.

Handle attributes

width

Type: `string` or `number`

This attribute represents the width of the Fold-in. Examples:

```
myFoldin.width = "350px";    // 350 pixels
myFoldin.width = 350         // 350 pixels
myFoldin.width = "40vw";    // 40 % of screen width
```

contentId

Type: `string`

This attribute represents the element ID that will be used for the content div element in this Fold-in. After attaching the Fold-in to the document body, this element ID can be used for adding content to the Fold-in. See example and general explanation above.

parent

Type: `Element`

After attaching the component to an element, this attribute will contain the parent element that the component has been attached to.

This attribute is read-only.

Methods

attachToBody

Signature: `attachToBody ( )`

Creates the visual component as DOM tree elements directly under the document body.

Further updates to the component handle will affect this visual component.

The Fold-In applies a blur filter to all sibling elements under the document body when shown. If performance is an issue, consider putting the body content in a wrapping `<div>`.

The blur filter will not affect text nodes directly under the body. To avoid this issue, put any such nodes in `<div>`, `<p>` or `<span>` blocks.

show

Signature: `show ( )`

Makes the Fold-in visible.

hide

Signature: `hide ( )`

Makes the Fold-in hidden.

5.19 Popup Dialog

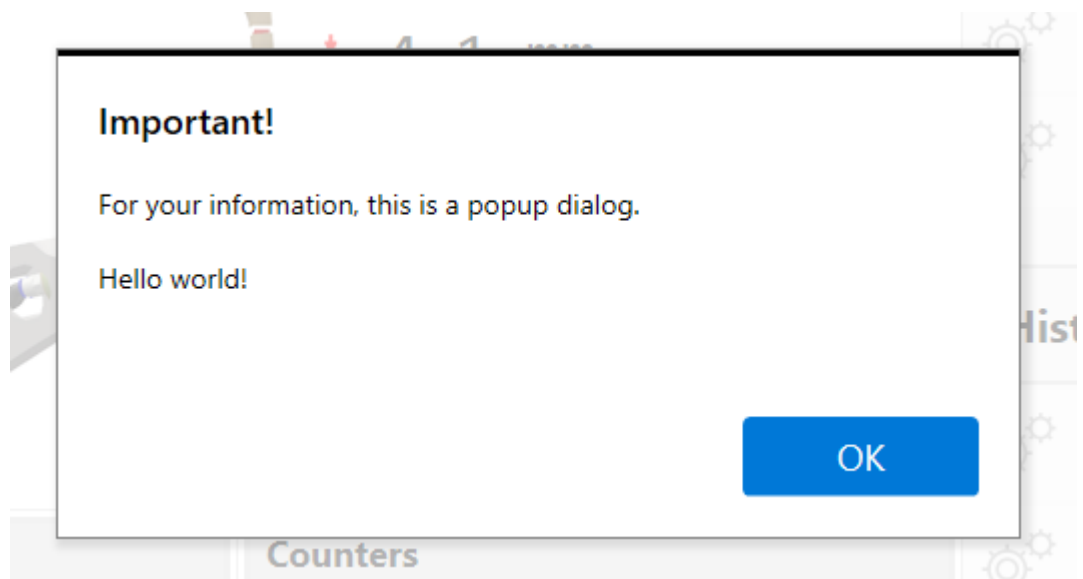
Description

A popup dialog is a modal component that can be used to in a simple manner provide the user with information, typically error/info messages or other occasional messages.

Another possibility is to have a simple "OK/Cancel" confirmation dialog.

It's easy to change the style of the dialog by setting the style to either warning or danger, which will add icons and change the color of the top border.

There's also support for additional customization options.



Requires

```
<script src="fp-components/fp-components-common.js"></script>
<script src="fp-components/fp-components-button-a.js"></script>
<script src="fp-components/fp-components-popup-a.js"></script>
```

Example 1

```
FPComponents.Popup_A.message(
    "Important!",
    [
        "For your information, this is a popup dialog.",
        "",
        "Hello world!"
    ]
);
```

Example 2

```
FPComponents.Popup_A.message(
    "Warning!", "Something went wrong", function (action) {
        // callback for clicking OK button
    }, FPComponents.Popup_A.STYLE.WARNING);
```

Example 3

```
FPComponents.Popup_A.confirm(
    "Heads up!", "Should we continue?",
```

```
function (action) {  
    if (action == FPComponents.Popup_A.OK) {  
        console.log("Let's go!");  
    } else if (action == FPComponents.Popup_A.CANCEL) {  
        console.log("Abort mission!");  
    }  
};
```

Example 4

```
FPComponents.Popup_A.confirm(  
    "Heads up!", "Should we continue?",  
    function (action) {  
        switch (action) {  
            case FPComponents.Popup_A.OK:  
                console.log("Let's go!");  
                break;  
            case FPComponents.Popup_A.CANCEL:  
                console.log("Abort mission!");  
                break;  
        }  
    }  
);
```

Example 5

```
var model = {  
    header: [  
        { type: "text", text: "This is a header"}  
    ],  
    content: [  
        { type: "text", text: "This is a line in a message" },  
        { type: "text", text: "This will be on another line" },  
    ],  
    footer: [  
        {  
            type: "button",  
            text: "Custom button",  
            highlight: true,  
            closeDialog: false,  
            action: "myAction"  
        }  
    ]  
}
```

```
FPComponents.Popup_A.custom(model, function(action){
    // implement callback for button clicks.
});
```

Usage

Three functions are available directly on the `Popup_A` object, there is no need to instantiate a new object.

First, there is the `message` function. This variant produces a simple popup with just an 'OK' button:

```
FPComponents.Popup_A.message(string topic, string/string[]
message, function callback=null, string style)
```

To produce a confirmation popup, with both 'OK' and 'Cancel' buttons, use the `confirm` function:

```
FPComponents.Popup_A.confirm(string topic, string/string[]
message, function callback=null, string style)
```

Argument	Description
<code>topic</code>	A string, describing the topic of the message.
<code>message</code>	The actual message. Can be either a simple string or, if several lines are required, an array where each element is a string with a message line. Example 1: "A message" Example 2: ["Message line 1", "Message line 2"]
<code>callback</code>	A function that will be called when the user dismisses by pressing the 'OK' or 'Cancel' button. This argument is optional, and the default value is <code>null</code>. The function signature should be <pre>function (action)</pre> where <i>action</i> is a string describing the action. The strings are predefined and can hence be compared using the <code>==</code> or <code>===</code> operator. The possible values for <i>action</i> are <pre>FPComponents.Popup_A.OK</pre> and <pre>FPComponents.Popup_A.CANCEL</pre>

style	A string determining the predefined style of the popup. This argument is optional, and the default value is null. The possible values for <i>style</i> are <code>FPComponents.Popup_A.STYLE.INFORMATION</code> <code>FPComponents.Popup_A.STYLE.WARNING</code> <code>FPComponents.Popup_A.STYLE.DANGER</code> Information style will add a blue information icon to popup. Warning style will add a warning icon to popup. Danger style will add a danger icon to popup

Custom popups

To produce a custom popup, without any default content, use the custom function. The custom function takes a model as parameter, which is detailed below.

In addition to text and buttons, the model accepts any kind of HTML DOM element (with children) for a completely custom setup.

```
FPComponents.Popup_A.custom(object model, function callback=null)
```

The model used by custom popup must always have three properties called header, content and footer, which will add elements to corresponding sections of the popup.

```
var model = {
  header: [...objects],
  content: [...objects],
  footer: [...objects],
}
```

See full example in examples section above.

Attribute	Description
type	Mandatory attribute. Can be either "text", "button" or "element". Each different object type represents text messages, buttons, or DOM elements (for completely custom content).
text	Optional attribute. Valid for types "button" and "text" only.

	The text that should be displayed on text and button objects. For multiline text, use several text objects.
<code>action</code>	Valid for type "button" only. The action that should be passed to the callback function when the button is clicked. This action can be any value.
<code>highlight</code>	Valid for type "button" only. Set to <code>true</code> to use highlight styling for button, same as button component. Default is <code>false</code> .
<code>closeDialog</code>	Valid for type "button" only. Set this to <code>false</code> to prevent dialog from closing when the button is clicked. Default is <code>true</code> .
<code>element</code>	Valid for type "element" only. Any kind of HTML DOM element which will be inserted as-is. Can contain any number of children for a completely custom setup.

Closing programatically

Sometimes it is useful to be able to close a popup from code. A typical scenario is when using a heavily customized popup using the "element" object type described above.

To close the currently active dialog, use this method:

```
FPComponents.Popup_A.close(action)
```

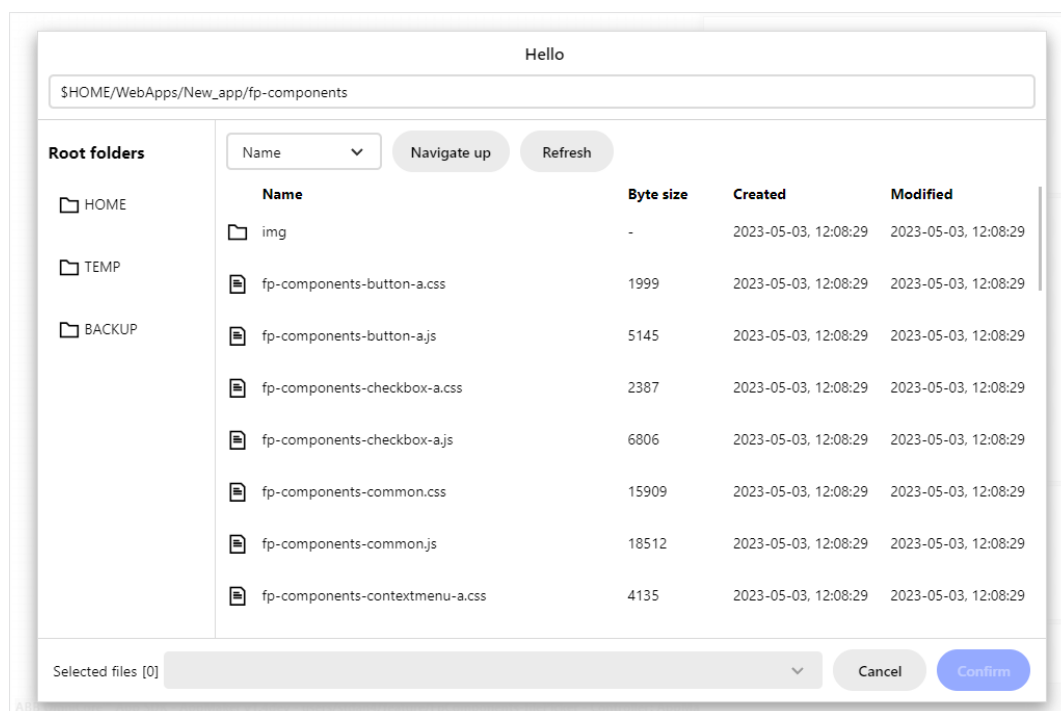
The action parameter can be of any type and will be passed on to the callback function when closing.

5.20 File System Dialog

Description

A file system dialog is a modal component that is used to provide an interactive method for selecting files or folders from the controller's file system.

The dialog can be customized with different options, such as to use a custom topic, use a file name filter, select single or multiple file system objects, and select which type of file system object to be target by the dialog (files or folders).



Requires

```
<script src="fp-components/fp-components-common.js"></script>
<script src="fp-components/fp-components-button-a.js"></script>
<script src="fp-components/fp-components-dropdown-a.js"></script>
<script src="fp-components/fp-components-filesystemdialog-
a.js"></script>
```

Please note that this component uses the Robot Web Services Client Library.

Example 1

```
let filePath = await FPComponents.Filesystemdialog_A.select();
```

Example 2

```
let filePath = await FPComponents.Filesystemdialog_A.select(
  FPComponents.Filesystemdialog_A.SelectionMode.Single,
  FPComponents.Filesystemdialog_A.FileSystemObjectType.File,
  "A custom dialog topic",
  [".txt", ".png", ".js", ".css"],
  FPComponents.Filesystemdialog_A.ROOT_FOLDER_HOME);
```

Example 3

```
let folderPathArr = await FPComponents.Filesystemdialog_A.select(
    "multi",
    "folder",
    null,
    "$BACKUP/myFolder");
```

Usage

Just like the Popup component, there is no need to instantiate a new object. The `select` function can be called directly from the component class object, without the need of a constructed instance.

```
FPComponents.Filesystemdialog_A.select(selectionMode="single",
fileSystemObjectType="file", string textHeading=null, string[]
validFileEndings=null, string defaultFolderPath="$HOME")
```

Argument	Description
selectionMode	Denotes if single or multiple file system objects can be selected from the dialog. This argument is optional, and the default value is "single". Valid inputs are "single" or "multi".
fileSystemObjectType	Denotes the file system object type, that is to be targeted by the selection in the dialog (file or folder). This argument is optional, and the default value is "file". Valid inputs are "file" or "folder".
textHeading	The optional heading/topic text of the dialog. This argument is optional, and the default value is null.
validFileEndings	An array of strings, denoting valid file name endings. This argument is optional, and the default value is null. Only file names on the VC/RC that ends with at least one of the strings in this array will be displayed by the dialog. This filter is ignored when set to null (dialog displays all file names).
defaultFolderPath	The default folder path of the dialog.

	This argument is optional, and the default value is null. If path is invalid, this path will automatically be set to \$HOME. The dialog considers the following folder paths as valid: \$HOME, \$TEMP, \$BACKUP, or any of their sub folders.
--	--

The `select` method returns a Promise. The Promise result depends on the following scenarios:

Use cases	Promise result
User cancels the dialog	null
User confirms selection, with one or more file system objects selected.	An array of strings, where each string represents a chosen file or folder.

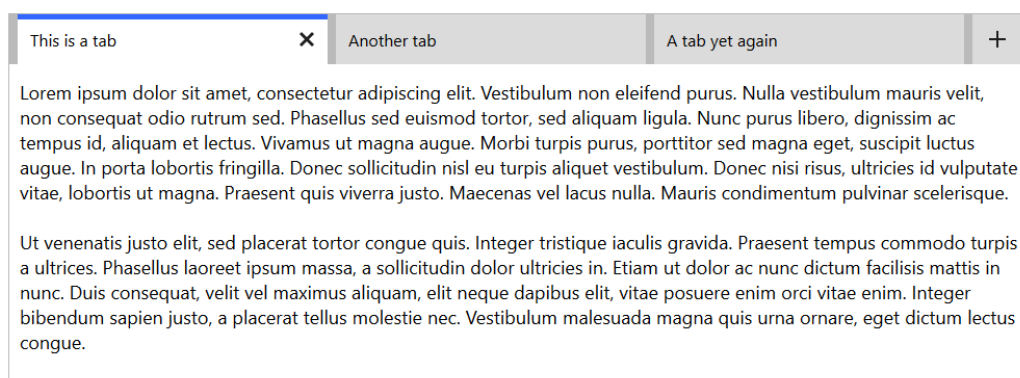
5.21 Tab Container

Description

The Tab container component implements a view-switching area with a visual tab bar at the top.

It can be populated with any content and can be configured for most use cases. Any number of tabs are supported.

Optionally, the component can be used as a generic view-switcher without any visible tab bar.



Requires

```
<script src="fp-components/fp-components-common.js"></script>
```

```
<script src="fp-components/fp-components-tabcontainer-a.js"></script>
```

Example 1

The following example produces a simple full-window tab container with two tabs.

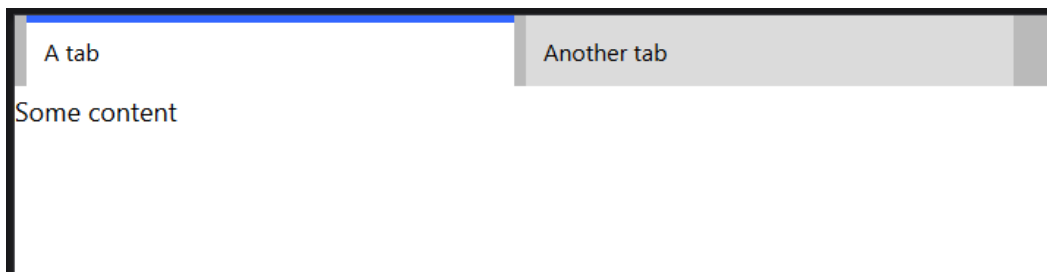
Note that it does not matter where the content div elements are placed, they will be moved into the tab container when the `addTab` method is used.

```
var myTabContainer = new FPComponents.Tabcontainer_A();
myTabContainer.addTab("A tab", "contentDiv1");
myTabContainer.addTab("Another tab", "contentDiv2");
myTabContainer.attachToId("myDiv");

...

<body>
  <div id="myDiv" style="height: 100vh"></div>

  <div id="contentDiv1">Some content</div>
  <div id="contentDiv2">Some more content</div>
</body>
```



Handle constructor

```
var myFoldin = new FPComponents.Tabcontainer_A();
```

Creates a handle to a new Tab container

Tab identifier objects

Many of the attributes and methods available on the Tab container component handle uses *tab identifier objects* as a way of referring to the tabs themselves.

When creating a tab using the `addTab` method, a tab identifier object is returned.

IMPORTANT: When comparing tab identifier objects, make sure to use the JavaScript *identity operator*, i.e. `===`, **not** the *equality operator*, i.e. `==`.

Limitations

IMPORTANT: Elements that are inside a tab that is not active are not part of the DOM tree. This means that they cannot be searched for using methods like `document.getElementById` or `document.querySelector`.

It also means that after adding an element to a tab, while the tab is inactive, the id name attribute for that element or any of its' children cannot be used for attaching components using `attachToId` methods, since those methods are searching the DOM for the element.

To get around this limitation, it is possible to either..

- Attach the components to the element before creating the tab
- Save a reference to the element before creating the tab, and then use the `attachToElement` methods instead of `attachToId`.

Handle attributes

`activeTab`

Type: `Object`

This attribute represents the currently active tab in the form of a tab identifier object.

Set this attribute to another tab identifier object to switch tab/view programmatically.

`onchange`

Type: `function ( Object oldTabId, Object newTabId )`

Set this attribute to a callback function that should be called when a new tab becomes active, i.e. the container has switched view to another tab.

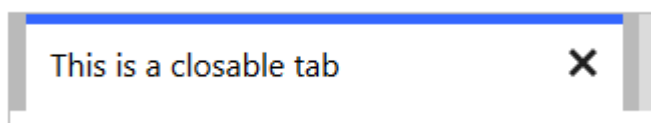
The `oldTabId` and `newTabId` parameters are both tab identifier objects which respectively identify the previously active tab and the currently active tab. It is possible for one of the parameters to be null, e.g. when a first tab is created, or when the last tab is removed.

`userTabClosing`

Type: `boolean`

Set this attribute to `true` to allow the user to close tabs using a button on each active tab.

Default value is `false`.



onuserclose

Type: `boolean function ( Object tabId )`

Set this attribute to a callback function that should be called when the user clicks on a tab's "close" button. The parameter `tabId` is the tab identifier object for the tab being closed.

If this attribute is not set, the tab will be closed immediately.

If it is set to a function, that function will be called, and if the function returns `true`, the tab will be removed directly.

If the function returns `false`, the tab will not be removed, and it is up to the developer to later close the tab using the `removeTab` method. This is useful for implementing user interaction when closing the tab, e.g. "Do you want to save?" or "Do you really want to close?".

onplus

Type: `function ( )`

Set this attribute to a callback function that should be called when the user clicks on the plus ("add-tab") button. Clicking the button has no effect other than this callback function being called.

Typically, this callback function will dynamically build a new view and then add it as a new tab using the `addTab` method.

Setting this attribute to a function will cause the plus button to appear in the tab bar.

**plusEnabled**

Type: `boolean`

Set this attribute to `false` to disable and gray out the plus ("add-tab") button, whenever visible.



Default value is `true`.

hiddenTabs

Type: `boolean`

Set this attribute to `true` to completely hide the tab bar. This means that only the tab content will be visible.

This is useful for using the Tab container component as a generic view-switcher.

Default value is `false`.

tabIdList

Type: `array`

This attribute represents a list of objects identifying the current tabs in the Tab container.

The order of the list is the same as the order of the tabs. The list is generated each time it is read from the handle object, which means that a read list will not change unless read again.

Example, closing the third tab:

```
myTabContainer.removeTab(myTabContainer.tabIdList[2]);
```

This attribute is read-only.

parent

Type: `Element`

After attaching the component to an element, this attribute will contain the parent element that the component has been attached to.

This attribute is read-only.

Methods

addTab

Signature: `addTab ( string title, string/Element content, boolean makeActive=false )`

Creates a new tab and adds it to the Tab container.

The `title` string parameter specifies the tab label.

The `content` parameter specifies the content element of the tab.

The parameter can be either the id of an element existing in the DOM tree, or an `Element` object directly.

If the element is a child of a parent element, it will be removed from the parent before being placed within the Tab container. This means that if tab content is declared as plain HTML (with an *id* attribute), it can be placed anywhere within the document before being added to the Tab container, since it will be moved automatically.

The optional `makeActive` parameter can be set to `true` to automatically make the tab active after adding it.

The `addTab` method returns a tab identifier object for the new tab. This can be used for referring to the tab at a later stage.

IMPORTANT: See “Limitations” section above.

removeTab

Signature: `removeTab ( Object tabId )`

Removes the specified tab from the Tab container.

The `tabId` parameter should be a tab identifier object referring to the tab.

getTabTitle

Signature: `getTabTitle ( Object tabId )`

Returns the title of the specified tab.

The `tabId` parameter should be a tab identifier object referring to the tab.

setTabTitle

Signature: `setTabTitle ( Object tabId, string title )`

Updates the title of the specified existing tab.

The `tabId` parameter should be a tab identifier object referring to the tab.

attachToId

Signature: `attachToId ( string elementId )`

Creates the visual component as DOM tree elements under a parent element, specifying the parent element using an element ID (*elementId*).

Further updates to the component handle will affect this visual component.

IMPORTANT: Always use an empty div element. Do not attach the component to an element with existing children.

attachToElement

Signature: `attachToElement ( Element element )`

Same as `attachToId`, but takes an `Element` object directly as argument instead of an element ID.

5.22 Hamburger Menu

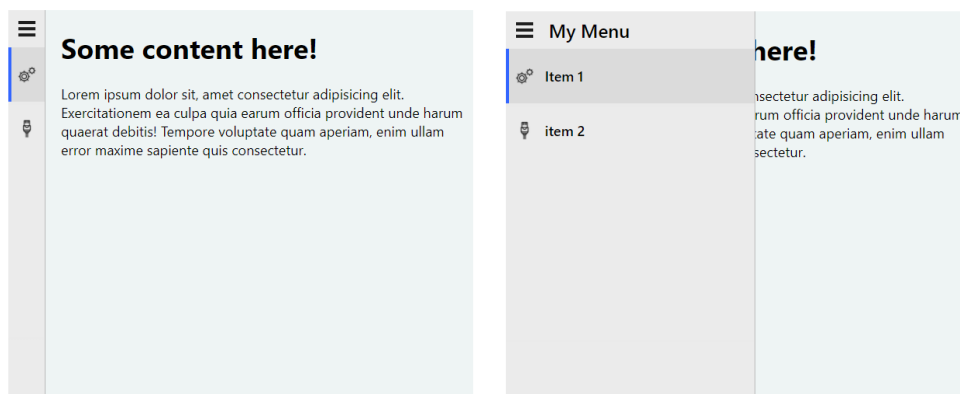
Description

The Hamburger Menu component implements a view-switching area with a hamburger menu. The hamburger menu can be configured to either always be visible or to only display a hamburger icon on the top left of the screen.

When configured to always be visible, the visible part will be only wide enough to display the icons of the menu and will push aside the content.

When configured to not be visible, the hamburger icon will be placed on top of the content, and on the top left. In this case it is up to the app developer to keep that area free of other elements.

The hamburger menu can be populated with any content and can be configured for most use cases. Any number of views are supported.



Requires

```
<script src="fp-components/fp-components-common.js"></script>
<script src="fp-components/fp-components-hamburgermenu-a.js"></script>
```

Example

This example creates a full screen hamburger menu and has the second item initially active. Note that the location of the content elements does not matter, they will be pulled into the hamburger menu's content area when the model is set.

```
var myHamburgermenu = new FPComponents.Hamburgermenu_A();
myHamburgermenu.addView("Item 1", "contentDiv1", "option1.png", false);
myHamburgermenu.addView("Item 2", "contentDiv2", "option2.png", true);
myHamburgermenu.attachToId("myMenuContainer");
...

<body>
  <div id="myMenuContainer" style="height: 100vh; width: 100vw"></div>

  <div id="contentDiv1">Some content</div>
  <div id="contentDiv2">Some more content</div>
</body>
```

Handle constructor

```
var myHamburgermenu = new FPComponents.Hamburgermenu_A();
```

Creates a handle to a new hamburger menu component

Limitations

IMPORTANT: Elements that are inside a view that is not active are not part of the DOM tree. This means that they cannot be searched for using methods like `document.getElementById` or `document.querySelector`.

It also means that after adding an element to a view, while the view is inactive, the id name attribute for that element or any of its' children cannot be used for attaching components using `attachToId` methods, since those methods are searching the DOM for the element.

To get around this limitation, it is possible to either..

- Attach the components to the element before creating the view
- Save a reference to the element before creating the view, and then use the `attachToElement` methods instead of `attachToId`.

Handle attributes

`alwaysVisible`

Type: `boolean`

Set this attribute to `true` (default) to have the hamburger menu always be visible. In this state the hamburger menu will be wide enough for the icons of each menu item to be visible when closed. When the menu is opened, it will be full width.

Set this attribute to `false` to hide the menu when closed, and only display the hamburger menu icon.

`title`

Type: `string`

Set this attribute to any string to display a title next to the hamburger menu icon when the menu is open.

Set this attribute to `false` (default) to hide the menu when closed, and only display the hamburger menu icon.

`onchange`

Type: `function ( Object oldViewId, Object newViewId )`

Set this attribute to a callback function that should be called when a new view becomes active, i.e. the container has switched view to another content element.

The `oldViewId` and `newViewId` parameters are both view identifier objects which respectively identify the previously active view and the currently active view. It is possible for one of the parameters to be null, e.g. when the first view becomes active.

`activeView`

Type: `Object`

This attribute represents the currently active view in the form of a view identifier object. Set this attribute to another view identifier object to switch view programmatically.

viewIdList

Type: `array`

This attribute represents a list of objects identifying the current views in the Hamburger menu.

The order of the list is the same as the order of the menu items. The list is generated each time it is read from the handle object, which means that a read list will not change unless read again.

Example, setting the third menu item and it's view as active:

```
myHamburgerMenu.activeView = myHamburgerMenu.viewIdList[2];
```

This attribute is read-only.

parent

Type: `Element`

After attaching the component to an element, this attribute will contain the parent element that the component has been attached to.

This attribute is read-only.

Methods

attachToId

Signature: `attachToId ( string elementId )`

Creates the visual component as DOM tree elements under a parent element, specifying the parent element using an element ID (*elementId*).

Further updates to the component handle will affect this visual component.

IMPORTANT: Always use an empty div element. Do not attach the component to an element with existing children.

attachToElement

Signature: `attachToElement ( Element element )`

Same as `attachToId`, but takes an `Element` object directly as argument instead of an element ID.

getViewButtonLabel

Signature: `getViewButtonLabel ( Object id )`

Returns the label of the specified views menu item.

The `id` parameter should be a view identifier object referring to the view.

setViewButtonLabel

Signature: `getViewButtonLabel ( Object id, string label )`

Sets the label of the specified views menu item to the provided label.

The `id` parameter should be a view identifier object referring to the view

addView

Signature: `addView ( string label, string/Element contentElement, string icon, boolean active=false )`

Creates a new view with corresponding menu item and adds it to the Hamburger menu.

The `label` string parameter specifies the menu item label.

The `contentElement` parameter specifies the content element of the view to be displayed.

The parameter can be either the `id` of an element existing in the DOM tree, or an `Element` object directly.

If the element is a child of a parent element, it will be removed from the parent before being placed within the Hamburger menu. This means that if tab content is declared as plain HTML (with an `id` attribute), it can be placed anywhere within the document before being added to the Hamburger menu, since it will be moved automatically.

The `icon` parameter specifies a path to an icon to be used in the menu item. The optimal size for the icon is 24x24 px but it will be automatically resized.

The optional `active` parameter can be set to `true` to automatically make the view active after adding it.

The `addView` method returns a view identifier object for the new view. This can be used for referring to the view at a later stage.

IMPORTANT: See “Limitations” section above.

removeView

Signature: `removeView ( Object id )`

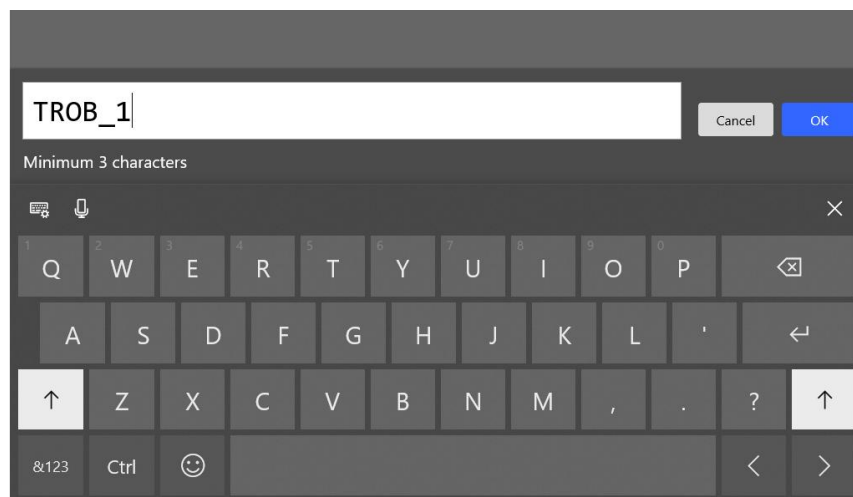
Removes the referenced view from hamburger menu component, meaning the menu item is removed and if the view was active, the content is removed as well.

The `id` parameter should be a view identifier object referring to the view.

5.23 On-screen Keyboard

Description

The on-screen keyboard can be used when text input from the user is required, but the Input field component (which has integrated support for this keyboard) cannot be used.



Requires

```
<script src="fp-components/fp-components-common.js"></script>
```

Example

```
fpComponentsKeyboardShow( function(text) {  
    // This function will execute when  
    // the user has finished editing.  
}, "Change me" );
```

Usage

```
fpComponentsKeyboardShow ( function callback, string initial=null,  
string label=null, object variant=null, RegExp regex=null,  
function validator=null )
```

Argument	Description
<code>callback</code>	A callback function that will execute when the user has finished the input. The function signature should be: <pre>function (text)</pre> If the user finishes input and clicks on the “OK” button, <code>text</code> will be a string containing the new value. If the user cancels input (by clicking on the close button or by clicking on the shaded background), <code>text</code> will be <code>null</code>.
<code>initial</code>	Initial string that should be edited by the user. Can be omitted, default value is <code>null</code>.
<code>label</code>	Descriptive label string that will be visible below the editor field on the keyboard. Should describe the value that the user is editing, preferably including any input limitations. Can be omitted, default value is <code>null</code>.
<code>variant</code>	Initial keyboard variant to be shown. Possible values are <code>FP_COMPONENTS_KEYBOARD_ALPHA</code> or <code>FP_COMPONENTS_KEYBOARD_NUM</code>. Can be omitted, default value is <code>null</code>. NOTE! <i>This argument is currently ignored and will not affect the appearance of the keyboard. This is due to limitations in built-in touch keyboard. It is possible that it will work in a future release.</i>
<code>regex</code>	Standard JavaScript regular expression object used for validating and allowing the input. Example: <pre>/^-?[0-9]+(\.[0-9]+)?\$/</pre> will only allow input of floating-point numbers or integers. Can be omitted, default value is <code>null</code>. Can be used in combination with the <code>validator</code> argument.

<code>validator</code>	A callback function that will execute whenever the user is altering one or more characters in the editable field. The function signature should be: <pre>function (val)</pre> The <code>val</code> argument will be the current value (string) as it is being edited by the user. The callback function should return <code>true</code> when the value is acceptable and <code>false</code> when not acceptable. Can be omitted, default value is <code>null</code>. Can be used in combination with the <code>regex</code> argument.
------------------------	--

5.24 Console Log Overlay

Description

When developing for the FlexPendant, it can be useful to access the standard JavaScript console log. Normally, this is not possible, since – unlike a common web browser – the FlexPendant has no integrated native console.

The Console log overlay component can be used to view the log (`console.log`, `console.info`, `console.error`). Since the overlay jacks into the default logger in JavaScript, it is likely that any third-party code will have its output printed here.

The overlay should only be used during development.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Mauris consectetur dolor id sem molestie elementum. Vestibulum ac pretium risus. Vivamus venenatis ligula nisl. Integer posuere sapien et dui feugiat viverra. Sed aliquet tristique tincidunt. Aliquam vel felis fermentum, luctus augue nec, ullamcorper justo. Donec congue, ante et venenatis placerat, lorem est congue arcu, lobortis congue neque lacus id odio. Fusce malesuada lacinia arcu, vel gravida nibh pulvinar id. Vestibulum rutrum ultricies mi, id mollis enim facilisis nec. Integer dignissim, urna vel tempor finibus, metus enim ultricies odio, sed sagittis quam urna ac massa. Morbi hendrerit fringilla ex, eu bibendum neque facilisis ac. Suspendisse eget orci vitae lacus porta varius sed nec magna. Curabitur ipsum orci, lobortis ac erat vitae, finibus consequat sapien. Praesent justo odio, condimentum sed suscipit nec, lacinia non nisi.

Curabitur faucibus eros odio, ac feugiat mi commodo vitae. Quisque accumsan dolor ut sodales tempus. Praesent tincidunt porttitor scelerisque. Proin nisi purus, vehicula ac tortor vel, elementum egestas purus. Fusce scelerisque in mauris eu sodales. Morbi neque ante, ornare porta purus sed, vehicula iaculis lectus. Vivamus ipsum lorem, tincidunt at ultricies quis, sollicitudin eu sem. Maecenas molestie erat a fringilla congue.

Vivamus fermentum metus eu dolor ullamcorper posuere. Mauris eu nunc congue, posuere ante non, fermentum eros. Donec feugiat hendrerit tellus, interdum facilisis tortor. Nullam lobortis erat nec urna pellentesque, at hendrerit ipsum ullamcorper. Vestibulum feugiat felis sed maximus egestas. Nullam maximus, quam nec blandit interdum, sapien risus porta quam, a ultricies mi sapien ac neque. Quisque ut malesuada augue, id vulputate neque. Morbi quis felis augue. Fusce eget massa vel nibh vestibulum ultrices. Phasellus gravida, ipsum sit amet convallis commodo, felis orci tristique justo, non imperdiet justo ante ac mauris.

Ac auctor augue mauris augue. Luctus venenatis lectus magna fringilla urna porttitor rhoncus dolor. Felis imperdiet proin fermentum leo vel orci porta non pulvinar. Euismod elementum nisi quis eleifend. Nisl purus in mollis nunc sed id semper risus. Sapien eget mi proin sed libero enim sed faucibus turpis. Eleifend donec pretium vulputate sapien nec sagittis. Enim neque volutpat ac tincidunt vitae semper quis lectus. Nunc consequat interdum varius sit amet mattis vulputate. Tristique senectus et netus et malesuada. Turpis egestas maecenas pharetra convallis posuere morbi. Neque laoreet suspendisse interdum consectetur. Mattis vulputate enim nulla aliquet porttitor lacus luctus accumsan tortor. Et ultrices neque ornare aenean euismod elementum nisi quis eleifend. Ullamcorper sit amet risus nullam eget. Amet massa vitae tortor condimentum lacinia quis vel

In the example above, note the gray square in the lower left corner. It will be visible when the overlay is enabled. Clicking on it will bring up the overlay:



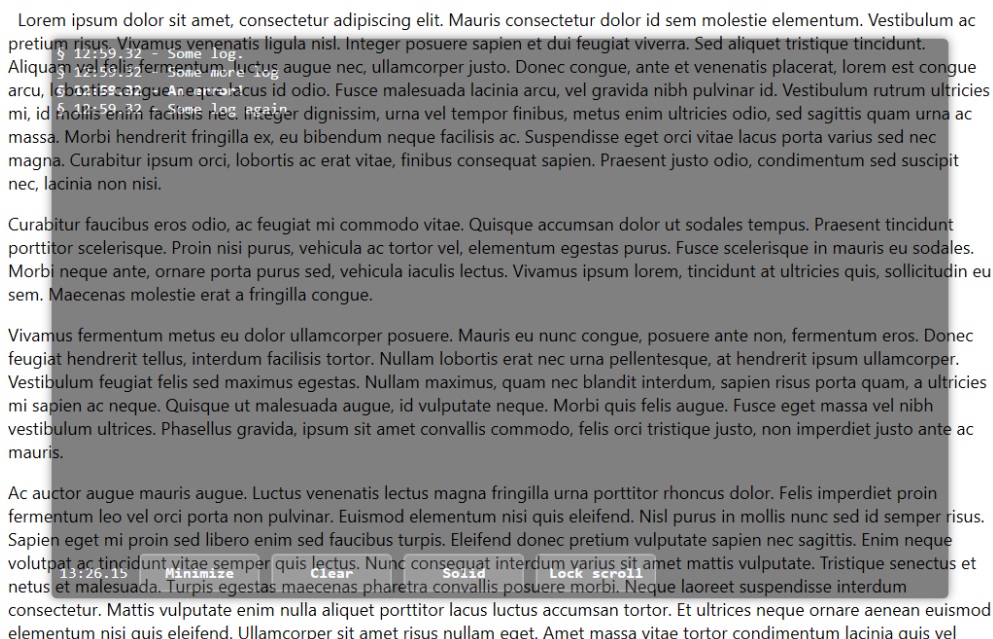
The overlay features a scrollable log, a reference clock, and a few buttons:

The **Minimize** button will hide the overlay and once again display the launcher square in the lower left corner.

The **Clear** button will clear the log.

The **Lock scroll** button will stop automatic scrolling until clicked again.

The **Ghost** button will cause the overlay to be semi-transparent. When this mode is active, it is possible to “click through” the overlay. This is useful for interacting with the UI and monitor the log at the same time. See image below.



```
<script src="fp-components/fp-components-common.js"></script>
```

```
// Run this early in the application when debugging.
fpComponentsEnableLog();
```

fpComponentsEnableLog ()

6 AppMaker

Regarding AppMaker

Previous versions of OmniCore App SDK included *AppMaker*, a visual tool for quickly designing and generating code for extensible OmniCore App SDK-based apps.

As of release 1.5, AppMaker has been discontinued.

If this causes any inconvenience, please reach out to ABB support to discuss your options for moving forward.